



**ESCUELA POLITÉCNICA
SUPERIOR DE CÓRDOBA**
Universidad de Córdoba



TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

Sistema de Vigilancia Distribuido Basado en Red de Radars Cooperativos

Autor

Alejandro Millan de Lara

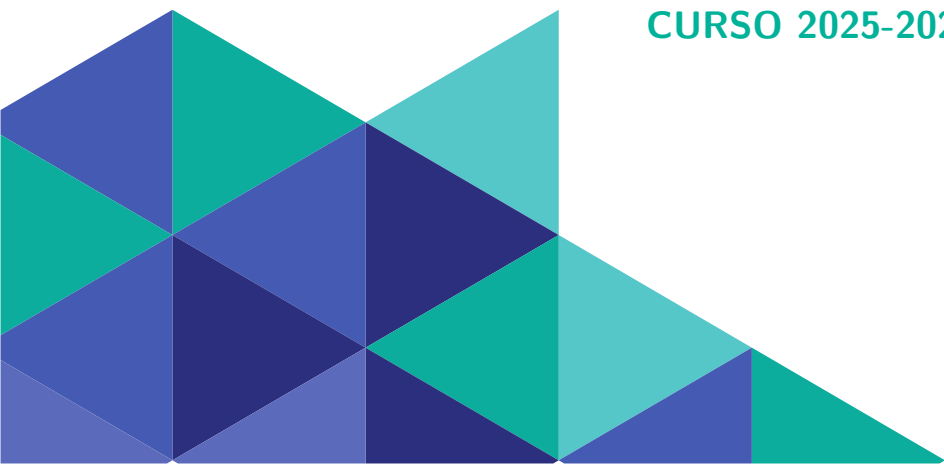
Directores

**Dr. Fernando León García
Cristina Martínez Ruedas**

CURSO 2025-2026



UNIVERSIDAD D CÓRDOBA



Agradecimientos

En primer lugar, quiero dedicar este proyecto a mis padres, quienes han sido el pilar fundamental de mi vida. Gracias a los valores que me habéis inculcado, he podido forjar la persona que soy hoy y encontrar la determinación para superar cada obstáculo. A mi madre, que en paz descanse; tu recuerdo ha sido mi brújula para no perder el rumbo y tu influencia sigue presente en cada uno de mis logros. A mi padre, por ser mi mayor referente y mi red de seguridad; sin tus enseñanzas y tu apoyo incondicional en los momentos de mayor debilidad, jamás habría llegado hasta aquí.

A mis hermanas, quiero darles las gracias por su paciencia inagotable y por acompañarme siempre. Habéis sido mi refugio en la etapa más difícil de mi vida. Cuando las fuerzas flaqueaban y la idea de abandonar la carrera parecía la única salida, fuisteis vosotras quienes me disteis el impulso y la motivación necesarios para seguir adelante y cruzar esta meta.

A mis amigos, gracias por ser mi vía de escape. Aunque el tiempo y las responsabilidades hacen que no podamos vernos tan a menudo como nos gustaría, cada encuentro nos permite disfrutar al máximo. A vuestro lado he vivido los mejores momentos de mi vida, y espero que esa conexión nos acompañe siempre.

No puedo olvidar a mis compañeros de clase, que con el paso de los años se han convertido en grandes amigos. Vuestra presencia ha sido vital a lo largo de toda la etapa universitaria. Gracias por las horas de esfuerzo compartido, por la ayuda técnica en las asignaturas más exigentes y por el apoyo mutuo cuando el camino se hacía cuesta arriba. Habéis conseguido que la dureza de esta carrera sea una experiencia mucho más llevadera y memorable.

Por último, quiero expresar mi gratitud a mis tutores, Fernando y Cristina, por hacer posible la realización de este Trabajo de Fin de Grado. Vuestra orientación, vuestras ideas y, sobre todo, vuestra infinita paciencia han sido determinantes para poder desarrollar un proyecto que realmente se ajusta a mis preferencias. Sin vuestra guía constante y dedicación, materializar este trabajo no habría sido posible. A todos los que de una forma u otra habéis formado parte de este proceso, y con un recuerdo muy especial a mi madre, mi más sincero y profundo agradecimiento.

Sistema de Vigilancia Distribuido Basado en Red de Radares Cooperativos

Resumen

Este Trabajo de Fin de Grado presenta el diseño, implementación y validación de un sistema distribuido de radares cooperativos sobre plataformas embebidas de bajo coste (ESP32 con sensores ultrasónicos). A diferencia de los sistemas de barrido secuencial tradicionales que sufren de altas latencias o interferencias cruzadas (crosstalk), este proyecto desarrolla una arquitectura de control centralizada basada en WiFi y un protocolo de comunicación binario sobre TCP/IP optimizado para tiempo real.

El sistema desarrollado implementa un algoritmo de Reutilización Espacial Dinámica sobre una red TDMA (Acceso Múltiple por División de Tiempo). El servidor central es capaz de calcular en tiempo real las trayectorias de los haces de los sensores, permitiendo la ejecución paralela de mediciones en nodos no adyacentes y serializando únicamente aquellos con riesgo de colisión acústica. Los resultados experimentales, validados mediante una arquitectura híbrida de hardware físico y gemelos digitales, demuestran la viabilidad de coordinar múltiples agentes para maximizar la frecuencia de muestreo global sin comprometer la integridad de los datos.

Palabras Clave: Radares Cooperativos, Sistemas Distribuidos, TDMA Dinámico, Reutilización Espacial, ESP32, Interferencias Ultrasónicas.

Distributed Surveillance System Based on a Cooperative Radar Network

Abstract

This Final Degree Project presents the design, implementation, and validation of a distributed cooperative radar system on low-cost embedded platforms (ESP32 with ultrasonic sensors). Unlike traditional sequential scanning systems that suffer from high latencies or crosstalk interference, this project develops a centralized control architecture based on WiFi and a binary communication protocol over TCP/IP optimized for real-time operation.

The developed system implements a Dynamic Spatial Reuse algorithm over a TDMA (Time Division Multiple Access) network. The central server calculates the sensor beam trajectories in real-time, allowing for parallel measurement execution in non-adjacent nodes and serializing only those with a risk of acoustic collision. The experimental results, validated through a hybrid architecture of physical hardware and digital twins, demonstrate the viability of coordinating multiple agents to maximize the global sampling frequency without compromising data integrity.

Keywords: Cooperative Radars, Distributed Systems, Dynamic TDMA, Spatial Reuse, ESP32, Ultrasonic Interference.

Índice general

1. Introducción y Contexto	1
1.1. Historia y evolución de los sistemas de detección y vigilancia	1
1.1.1. Historia en proyectos de vigilancia distribuida	2
1.2. Estado actual y relevancia	3
1.3. Motivación del proyecto	3
1.4. Enfoque y metodología	4
1.5. Estructura de la memoria	4
2. Objetivos	6
2.1. Objetivo Principal	6
2.2. Objetivos Secundarios	6
2.3. Alcance del Sistema	7
2.4. Restricciones y condicionantes	8
2.5. Criterios de Éxito	8
3. Antecedentes	9
3.1. El sensor ultrasónico como unidad de detección	9
3.2. El problema central: la interferencia acústica entre nodos	10
3.3. Enfoques para la eliminación del <i>crosstalk</i>	10
3.3.1. Separación en frecuencia y codificación de canal	10
3.3.2. Barrido secuencial (<i>Round Robin</i>)	11
3.3.3. Contención con detección de portadora (CSMA/CA)	11
3.3.4. TDMA Dinámico con reutilización espacial	11
3.4. Enfoque adoptado y aportación de este proyecto	13
4. Análisis del problema	14
4.1. Descripción del problema	14
4.1.1. El problema real	14
4.1.2. Problema técnico	15
4.2. Justificación del enfoque adoptado	15
4.3. Identificación de Requisitos	16
4.3.1. Requisitos Funcionales (RF)	16
4.3.2. Requisitos No Funcionales (RNF)	17
4.4. Restricciones	19
4.4.1. Factores Estratégicos	19
4.4.2. Factores Técnicos	19
5. Diseño	21
5.1. Diseño Hardware	21
5.1.1. Selección y Justificación de las Decisiones de Diseño Hardware	21
5.1.2. Especificaciones Hardware	26

5.1.3.	Diagrama de conexiones	29
5.1.4.	Elementos Estructurales de Soporte en impresión 3D	30
5.2.	Diseño Software	34
5.2.1.	Visión General de la Arquitectura	34
5.2.2.	Topología de Red y Distribución Física	36
5.2.3.	Modelo de Datos: Protocolo de Comunicación TCP	40
5.2.4.	Modelo Procedimental: La Supertrama TDMA	51
5.2.5.	Modos de Operación y Localización Cooperativa	52
5.3.	Diseño del Firmware	54
5.3.1.	Máquina de Estados Finita del nodo	54
5.3.2.	Arquitectura de tareas	57
5.3.3.	Modelo de Datos	58
5.3.4.	Control de hardware	64
5.4.	Interfaz gráfica de usuario	66
5.4.1.	Descripción general	66
5.4.2.	Campo de visualización radar	68
5.4.3.	Panel de control: Modo Diseño	69
5.4.4.	Panel de control: Modo Telemetría	72
5.4.5.	Reanudación de sesión	74
6.	Pruebas	76
6.1.	Prueba 1: Verificación del Ciclo de Supertrama y Detección de Objetos	76
6.1.1.	Descripción de los Componentes Utilizados	76
6.1.2.	Escenario de Prueba	77
6.1.3.	Funcionamiento del Sistema	77
6.1.4.	Requisitos a Cumplir	78
6.1.5.	Resultados Obtenidos	79
6.1.6.	Evaluación de Resultados Obtenidos	81
6.2.	Prueba 2: Resolución de Conflictos de Ranura y Localización Cooperativa	82
6.2.1.	Descripción de los Componentes Utilizados	82
6.2.2.	Funcionamiento del Sistema	82
6.2.3.	Escenario de Prueba	83
6.2.4.	Requisitos a Cumplir	84
6.2.5.	Resultados Obtenidos	84
6.2.6.	Evaluación de Resultados Obtenidos	86
6.3.	Prueba 3: Detección de Desconexión y Recuperación de Estado Angular	87
6.3.1.	Descripción de los Componentes Utilizados	87
6.3.2.	Funcionamiento del Sistema	87
6.3.3.	Escenario de Prueba	88
6.3.4.	Requisitos a Cumplir	88
6.3.5.	Resultados Obtenidos	89
6.3.6.	Evaluación de Resultados Obtenidos	91
7.	Conclusiones	92
7.1.	Conclusiones Generales	92

7.2. Grado de Cumplimiento de los Objetivos	92
7.2.1. Validación de Requisitos Funcionales	93
7.2.2. Validación de Requisitos No Funcionales	94
7.2.3. Conclusión sobre los Objetivos	95
7.3. Retos Técnicos y Limitaciones	95
7.4. Retos Técnicos y Limitaciones	95
7.5. Líneas de Trabajo Futuro	97
Referencias	99
A. Fabricación del Nodo Radar	102
A.1. Montaje de la Estructura Mecánica	102
A.2. Conexiones Electrónicas	103
A.3. Grabación del Firmware	105
B. Instalación y Configuración del Sistema	106
B.1. Requisitos de Software	106
B.2. Arranque del Servidor y el Visualizador	106
B.3. Ubicación Física de los Nodos	106
B.4. Configuración de Coordenadas y Zonas de Alerta	108
C. Guía de Operativa	109
C.1. Modo Diseño	109
C.2. Modo Telemetría	109
C.3. Reanudación de Sesión	109

Índice de figuras

1.	Esquema de conexiones del nodo radar.	27
2.	Diagrama de conexiones del nodo radar.	29
3.	Esquema del diseño tridimensional.	31
4.	Vistas de planta y alzado del ensamblaje final del nodo periférico con cotas generales (mm).	31
5.	Evolución de los prototipos preliminares del nodo: A) Iteración 1, B) Iteración 2 y C) Iteración 3.	32
6.	Diseño de la Versión Final del nodo.	33
7.	Nodo radar completamente ensamblado.	33
8.	Diagrama de la estructura por capas del sistema.	35
9.	Diagrama de flujo del bucle de orquestación del servidor central.	36
10.	Representación lógica del modelo cliente-servidor en estrella.	38
11.	Diagrama de flujo del descubrimiento automático del servidor.	39
12.	Abstracción geométrica de la distribución perimetral, orientaciones acústicas y áreas de intersección (<i>crosstalk</i>).	40
13.	Diagrama de Secuencia UML — Protocolo de comunicación TDMA entre el nodo ESP32 y el servidor central durante una sesión operativa completa.	45
14.	Diagrama de módulos UML del servidor central (<code>server_central.py</code>).	50
15.	Localización cooperativa por trilateración: estimación de posición global combinando las mediciones polares de dos nodos adyacentes en zona de solapamiento.	52
16.	Modos de operación del nodo sensor: SEARCH y TRACK con sus subestados y condiciones de transición.	53
17.	Diagrama de flujo de la FSM del nodo sensor ESP32.	55
18.	Arquitectura AMP del nodo sensor: distribución de tareas por núcleo y mecanismo de comunicación IPC.	57

19.	Diagrama de módulos UML del firmware del nodo ESP32.	64
20.	Diagrama de secuencia del ciclo EXECUTE_SLOT: desde la recepción del comando del servidor hasta la entrega del resultado.	66
21.	Modo Diseño de la interfaz gráfica de usuario.	67
22.	Modo Telemetría de la interfaz gráfica de usuario.	68
23.	Selección interactiva de nodo en el canvas.	69
24.	Efecto del zoom sobre la representación de zonas.	70
25.	Panel de configuración de parámetros de la interfaz.	71
26.	Tarjeta de nodo en estado de seguimiento activo.	72
27.	Canvas con detecciones activas por zona de alerta.	73
28.	Estimación de posición por localización cooperativa: dos nodos en modo TRACK sobre el mismo objeto en zona de solapamiento.	74
29.	Cuadro de diálogo de reanudación de sesión anterior.	75
30.	Configuración del despliegue en el Modo Diseño del visualizador: posición de los tres nodos sobre el plano, sectores de cobertura angulares y zonas de solapamiento entre pares adyacentes.	78
31.	Panel de visualización en Modo Telemetría con los tres nodos activos en modo búsqueda, mostrando el ángulo de apuntamiento actual de cada nodo antes de la introducción del objeto.	80
32.	Panel de visualización en Modo Telemetría tras la detección: nodo en modo seguimiento con el marcador de posición del objeto sobre el plano y las lecturas de distancia y ángulo activas en la tarjeta lateral.	81
33.	Modo Diseño del visualizador con las zonas sucias de los pares N1-N2 y N2-N3 señaladas.	83
34.	Registro del servidor durante las supertramas SF62 a SF64 con el par N1-N2 activo en zona sucia.	85
35.	Panel de visualización en Modo Telemetría con localización cooperativa activa: detecciones individuales de N1 y N2 (puntos rojos) y posición combinada estimada (círculo amarillo).	86
36.	Panel de visualización con los tres nodos activos antes de la desconexión de N2.	88

37.	Registro del servidor: desconexión de N2 con strikes 6/7 y 7/7 más mensaje de expulsión (izquierda) y reconexión en SF 260 con ángulo restaurado a $-27,5^\circ$ (derecha).	89
38.	Panel de visualización con N2 marcado como desconectado y N1 y N3 operando sin interrupción.	90
39.	Panel de visualización con los tres nodos activos tras la reconexión de N2, que retoma el barrido desde el ángulo restaurado.	91
40.	Nodo radar completamente ensamblado, vista frontal: sensor HC-SR04 en el cabezal rotatorio y electrónica de control en las placas del chasis.	104
41.	Nodo radar completamente ensamblado, vista trasera: cableado de los componentes electrónicos y estructura del chasis.	104
42.	Despliegue físico de los tres nodos radar sobre la superficie de vigilancia en disposición triangular.	107

Índice de tablas

1.	Comparativa de técnicas de gestión de acceso al medio en redes de sensores ultrasónicos.	12
2.	Requisitos funcionales (RF) del sistema.	17
3.	Requisitos no funcionales (RNF) del sistema.	18
4.	Comparativa técnica de plataformas de control.	23
5.	Comparativa técnica de motores.	25
6.	Componentes hardware de cada nodo.	28
7.	Estructura de la cabecera de trama (<i>Packet Header</i>).	41
8.	Diccionario de mensajes del protocolo.	42
9.	Mapa de bytes de MSG_HELLO_ACK.	43
10.	Mapa de bytes de MSG_SLOT_ASSIGN.	44
11.	Mapa de bytes de MSG_DATA_REPORT.	44
12.	Descripción del estado central de ArbitroServer (<code>server_central.py</code>).	46
13.	Descripción del orquestador de supertramas (<code>server_central.py</code>).	47
14.	Descripción del localizador cooperativo (<code>server_central.py</code>).	48
15.	Descripción del gestor de persistencia (<code>server_central.py</code>).	49
16.	Configuración de tareas FreeRTOS del nodo sensor.	57
17.	Descripción del módulo <code>comms_protocol.h</code>	59
18.	Descripción del módulo <code>config.h</code>	60
19.	Descripción del módulo <code>SystemManager</code> (<code>system.h</code>).	61
20.	Descripción del módulo <code>TaskComms</code> (<code>task_comms.cpp</code>).	62
21.	Descripción del módulo <code>RadarHardware/TaskRadar</code> (<code>task_radar.cpp/.h</code>).	63
22.	Requisitos funcionales evaluados en la Prueba 1.	79

23.	Requisitos funcionales evaluados en la Prueba 2.	84
24.	Requisitos funcionales evaluados en la Prueba 3.	89
25.	Grado de cumplimiento de los Requisitos Funcionales (RF).	93
26.	Grado de cumplimiento de los Requisitos No Funcionales (RNF).	94

Desde las primeras redes de escucha acústica utilizadas en conflictos armados del siglo XX hasta los avanzados sistemas de vigilancia distribuida que actualmente supervisan infraestructuras críticas, el problema fundamental se mantiene inalterado: detectar, localizar y seguir objetos o personas en un entorno específico, con la mayor fiabilidad posible y el menor coste operativo. Lo que ha cambiado drásticamente en las últimas décadas es el contexto tecnológico en el que se aborda este problema. La miniaturización de los componentes electrónicos, la disminución del coste de los sensores y la universalización de las comunicaciones inalámbricas han democratizado el acceso a capacidades de detección que, hasta los años noventa, estaban reservadas para aplicaciones militares o industriales de alto presupuesto [1, 2]. Este cambio en el umbral de accesibilidad ha creado un nuevo espacio de diseño en el que investigadores, ingenieros y estudiantes pueden desarrollar sistemas funcionales de vigilancia distribuida utilizando componentes comerciales de bajo coste, generando un cuerpo de conocimiento aplicado de extraordinaria relevancia para áreas tan diversas como la robótica de servicio, la seguridad perimetral, la gestión de afluentes o la domótica avanzada [3, 4].

El presente Trabajo de Fin de Grado se enmarca en esta tendencia y propone el diseño, implementación y validación de un sistema de vigilancia distribuida basado en una red de sensores de barrido angular cooperativos, capaz de detectar objetos en un espacio compartido y estimar su posición mediante la combinación de las medidas de múltiples nodos. La memoria que sigue documenta el proceso completo, desde la revisión del contexto tecnológico y la definición de objetivos hasta la validación experimental del prototipo construido.

1.1 Historia y evolución de los sistemas de detección y vigilancia

El fundamento físico que sustenta la tecnología radar, es decir, la emisión de una señal, su propagación a través del espacio, la reflexión en un obstáculo y el análisis del eco resultante para deducir la posición del objeto, fue descrito teóricamente por primera vez por Heinrich Hertz en 1888, cuando demostró de manera experimental que las ondas electromagnéticas podían reflejarse en superficies metálicas de forma similar a como lo hace la luz visible [5]. No obstante, la conversión de este principio en un sistema de detección práctico demandó décadas de desarrollo, impulsadas en gran medida por las necesidades de defensa aérea que se volvieron urgentes en los días previos a la Segunda Guerra Mundial. La cadena CHAIN HOME, implementada por el Reino Unido a partir de 1937, representó la primera red operativa de detección temprana de aeronaves mediante radar y fue crucial en el desarrollo de la Batalla de Inglaterra [5, 6]. La experiencia en el campo de batalla aceleró el avance técnico de manera notable: en menos de diez años se pasó de sistemas rudimentarios de onda continua

a radares de impulsos, de antenas fijas a sistemas de exploración angular, y de la detección cualitativa de presencia a la estimación cuantitativa de distancia, velocidad y rumbo.

La posguerra dio lugar a una diversificación gradual de las aplicaciones civiles. El radar meteorológico, la gestión del tráfico aéreo y la radionavegación marítima se transformaron en infraestructuras esenciales de la civilización contemporánea, y su evolución propició innovaciones conceptuales de gran relevancia: el efecto Doppler aplicado a la identificación de objetivos en movimiento [6], el radar de apertura sintética (SAR) para la captura de imágenes de alta resolución desde plataformas aéreas o espaciales [7], y el radar FMCW (*Frequency Modulated Continuous Wave*) para la medición de distancias cortas y medias en entornos terrestres [6]. Cada una de estas tecnologías respondía a necesidades específicas de aplicación, pero todas compartían una arquitectura conceptual común: un emisor, un receptor y un algoritmo de procesamiento que extrae información geométrica del tiempo de vuelo o de la variación de frecuencia de la señal reflejada.

1.1.1 Historia en proyectos de vigilancia distribuida

La implementación del principio radar en la vigilancia terrestre de áreas siguió un trayecto similar al de las grandes instalaciones militares, caracterizado por la búsqueda de sistemas más compactos, económicos y de fácil despliegue. Las primeras redes de sensores para la vigilancia terrestre, que se desarrollaron durante la Guerra de Vietnam mediante dispositivos sísmicos y acústicos dispersos desde aeronaves, anticiparon conceptualmente lo que décadas después se conocería como redes de sensores inalámbricos [8, 9]: un conjunto de nodos autónomos, distribuidos en el espacio, que recogen información local y la envían a un punto de agregación para su procesamiento conjunto. La diferencia entre esos sistemas y los modernos era principalmente tecnológica; la electrónica de los años sesenta no permitía el procesamiento local ni las comunicaciones bidireccionales de bajo consumo, pero la arquitectura distribuida, la diversidad espacial y la fusión de datos como estrategia para superar las limitaciones de un único sensor ya estaban presentes como principios de diseño [9].

La revolución de los sistemas microelectrónicos en la década de 1990, junto con la posterior aparición de las comunicaciones inalámbricas de corto alcance estandarizadas (IEEE 802.15.4, Bluetooth, Wi-Fi), facilitó el desarrollo de las redes de sensores inalámbricos como una disciplina académica e industrial claramente definida. El trabajo pionero de Akyildiz et al. [1], publicado en 2002, organizó los fundamentos teóricos del área y estableció los modelos de referencia que han orientado la investigación en las dos décadas siguientes. En el ámbito de la vigilancia, la disponibilidad de sensores de bajo costo como los ultrasónicos, infrarrojos o de imagen, combinada con la capacidad de comunicación inalámbrica de los nodos, abrió la posibilidad de soluciones que, por primera vez, eran económicamente viables para aplicaciones no militares: vigilancia de perímetros industriales, control de afluencia en espacios públicos, detección de intrusiones en entornos domésticos [3, 4, 10].

1.2 Estado actual y relevancia

El contexto tecnológico actual muestra una convergencia de elementos que favorece el avance de sistemas de vigilancia distribuidos que utilizan sensores de bajo costo. En lo que respecta a los sensores de proximidad, los dispositivos que miden la distancia mediante el tiempo de vuelo, tanto ultrasónicos como ópticos, han alcanzado un grado de madurez y accesibilidad que los hace viables para proyectos de pequeña envergadura sin sacrificar las capacidades funcionales [11, 12]. La precisión, el alcance y la robustez de estos sensores han experimentado un progreso constante en los últimos años, ampliando el rango de aplicaciones en las que pueden ser utilizados con garantías de fiabilidad [13].

En el ámbito de la computación embebida, la llegada de plataformas de desarrollo de bajo costo con conectividad inalámbrica integrada ha eliminado la necesidad de hardware especializado [2, 14]. Este cambio ha democratizado el acceso a arquitecturas distribuidas que anteriormente requerían conocimientos y recursos típicos del ámbito profesional, permitiendo que proyectos académicos aborden problemas de coordinación multi-nodo con el mismo rigor conceptual que los sistemas industriales, aunque a una escala de laboratorio.

Desde la óptica de las aplicaciones, la necesidad de sistemas de vigilancia autónomos, no invasivos y de escaso mantenimiento está en constante aumento en diversos sectores: la logística de almacenes automatizados, los entornos de atención a personas mayores, la gestión inteligente de edificios y la robótica de servicio crean escenarios en los que la detección y localización de presencia sin infraestructura óptica, y, por ende, sin las implicaciones de privacidad vinculadas a las cámaras, resulta especialmente valiosa [3, 4]. En este marco, los sistemas que se fundamentan en sensores acústicos o de radiofrecuencia están ocupando un nicho en expansión, favorecido por su resistencia a las condiciones de iluminación, su bajo consumo relativo y su facilidad de integración en entornos preexistentes [15, 16].

1.3 Motivación del proyecto

La motivación para llevar a cabo este trabajo proviene de la identificación de una limitación esencial en los sistemas de vigilancia de nodo único: un solo sensor de barrido angular, por muy preciso que sea, proporciona únicamente una medida de distancia desde una perspectiva específica, sin la capacidad de determinar la posición absoluta del objeto detectado ni de resolver las ambigüedades geométricas que surgen cuando varios objetos se encuentran a distancias similares en diferentes direcciones. La disposición de múltiples nodos en el espacio, con posiciones y orientaciones conocidas, transforma cualitativamente las capacidades del sistema: la información de distancia de cada nodo puede integrarse para estimar la posición del objeto mediante técnicas de trilateración, y la diversidad espacial de las mediciones disminuye la probabilidad de ocultación y amplía la cobertura total del área vigilada [15, 17].

No obstante, la operación conjunta de varios sensores acústicos en un entorno compartido

plantea un desafío que no tiene paralelo en los sistemas de nodo único: la interferencia entre sensores. Cuando dos nodos emiten pulsos acústicos de manera simultánea o superpuesta en el tiempo, el eco que recibe cada uno de ellos puede incluir contribuciones del pulso del otro, lo que genera lecturas incorrectas que afectan la fiabilidad del sistema en su totalidad [11, 13]. Este inconveniente, que en la literatura se aborda comúnmente a través de la separación temporal de las emisiones [16, 17], demanda un mecanismo de coordinación explícito que asegure que los nodos con potencial de interferencia mutua no operen al mismo tiempo. El diseño de dicho mecanismo de coordinación, junto con la validación de su efectividad en un prototipo real, constituye el núcleo técnico de este estudio.

La selección de este problema como tema de un Trabajo de Fin de Grado se justifica, además, por su naturaleza como ejercicio de síntesis de competencias: el sistema resultante combina conocimientos de electrónica analógica y digital, sistemas operativos de tiempo real, protocolos de comunicación en red, algoritmos de estimación y validación experimental, abarcando de manera natural el espectro de materias del Grado en Ingeniería Informática [14].

1.4 Enfoque y metodología

El proyecto ha adoptado una metodología de desarrollo iterativo enfocada en la integración progresiva de subsistemas. A partir de un análisis de requisitos que define el alcance funcional del sistema, se ha llevado a cabo el diseño y la construcción de cada componente de hardware y software de manera incremental, verificando en cada fase la conformidad del subsistema con los requisitos correspondientes antes de su integración con el resto. Este enfoque minimiza el riesgo técnico asociado a la complejidad del sistema global al aislar los problemas en el nivel donde se originan, y permite una adaptación continua del diseño a los resultados obtenidos en las pruebas intermedias.

La validación final del sistema se ha organizado en pruebas funcionales que complementan el conjunto de requisitos identificados, siguiendo las prácticas habituales en proyectos de ingeniería de sistemas embebidos distribuidos [18, 10]. Cada prueba establece un escenario específico, define de antemano los criterios de éxito y documenta los resultados obtenidos de manera cuantitativa, lo que facilita la evaluación objetiva del grado de cumplimiento de los requisitos y la delimitación precisa de las capacidades del prototipo desarrollado.

1.5 Estructura de la memoria

La presente memoria se organiza en siete capítulos. El capítulo 2 establece los objetivos del proyecto y el alcance del sistema. El capítulo 3 revisa el estado del arte en sistemas de vigilancia distribuida, detección por tiempo de vuelo y coordinación de redes de sensores. El capítulo 4 formaliza los requisitos funcionales y no funcionales que condicionan el diseño. El

capítulo 5 describe en detalle la arquitectura del sistema, las especificaciones hardware, el protocolo de comunicación y los mecanismos de coordinación y estimación implementados. El capítulo 6 presenta las pruebas de validación y analiza los resultados. El capítulo 7 expone las conclusiones y las líneas de trabajo futuro.

El presente Trabajo de Fin de Grado nace de la constatación de que los sistemas de detección basados en sensores ultrasónicos individuales presentan limitaciones estructurales que impiden su escalabilidad en aplicaciones de vigilancia perimetral. Un único sensor cubre un sector angular reducido y no puede monitorizar simultáneamente distintas regiones del espacio. La solución natural, desplegar varios sensores en paralelo, introduce un problema físico nuevo: las ondas acústicas emitidas por un nodo pueden ser recibidas por el sensor de otro, generando lecturas espurias que invalidan la medida. Este fenómeno, conocido como interferencia acústica o *crosstalk*, no tiene solución de hardware sencilla y obliga a abordarlo desde la coordinación entre nodos.

El objetivo de este trabajo no es únicamente construir un sistema multisensor, sino diseñar e implementar la lógica de coordinación que hace posible que varios nodos radar operen de forma cooperativa, sin interferirse mutuamente, y sean capaces de combinar sus medidas para obtener información espacial que ninguno de ellos podría proporcionar de forma aislada.

2.1 Objetivo Principal

Diseñar, desarrollar y validar un sistema de vigilancia distribuido formado por una red de nodos radar cooperativos sobre plataformas embebidas de bajo coste, capaz de cubrir un área perimetral de forma continua, coordinar el acceso al medio acústico para evitar interferencias entre nodos, y estimar la posición de objetos detectados de forma conjunta. El sistema debe operar en tiempo real sobre una infraestructura inalámbrica estándar y ser verificado mediante un prototipo físico funcional.

2.2 Objetivos Secundarios

- **Desarrollo del firmware del nodo radar:** Cada nodo debe operar de forma autónoma, ejecutando simultáneamente el control mecánico del sensor, la adquisición de medidas y la comunicación con el servidor central. El objetivo es que esta concurrencia sea robusta y predecible, de forma que ninguna tarea interfiera con las demás ni comprometa la precisión del posicionamiento angular o la fiabilidad de las medidas.
- **Diseño de un protocolo de comunicaciones eficiente:** La coordinación de la red exige que los nodos y el servidor intercambien información de control y medidas dentro

de márgenes temporales estrictos. El objetivo es definir un protocolo de mensajería de baja latencia que soporte la sincronización del ciclo de medida, la detección de pérdida de conexión y la reincorporación de un nodo sin pérdida de su estado previo.

- **Desarrollo del servidor central de orquestación:** El servidor debe concentrar la lógica de coordinación de la red: organizar los turnos de medida de cada nodo, resolver los conflictos de acceso al medio derivados de la geometría del sistema, mantener el estado global de la red y calcular estimaciones de posición cooperativas a partir de las medidas de varios nodos. El objetivo es que esta lógica opere en tiempo real y sea tolerante a la incorporación y pérdida de nodos durante el funcionamiento.
- **Desarrollo de la interfaz gráfica de visualización:** El sistema debe ofrecer una representación visual en tiempo real del estado de la red y de las detecciones registradas, de forma que un operador pueda interpretar de un vistazo la situación del área monitorizada. El objetivo es que esta interfaz se actualice de forma continua y refleje fielmente el estado interno del sistema.
- **Construcción del prototipo físico y diseño del banco de pruebas:** La validación del sistema requiere un entorno experimental reproducible que permita verificar su comportamiento bajo distintas condiciones de operación: funcionamiento normal, situaciones de interferencia acústica y pérdida de conectividad de nodos. El objetivo es disponer de un prototipo físico funcional y de un conjunto de pruebas que cubran los requisitos funcionales definidos para el sistema.
- **Elaboración de la memoria académica:** Documentar de forma completa y rigurosa las decisiones de diseño, el desarrollo del sistema, los resultados obtenidos y las posibles líneas de trabajo futuro, de modo que la memoria constituya por sí sola una referencia suficiente para reproducir o extender el trabajo realizado.

2.3 Alcance del Sistema

El prototipo desarrollado en este trabajo comprende una red de tres nodos radar operando de forma coordinada sobre una superficie plana en entorno de interior. Aunque la arquitectura diseñada es escalable en número de nodos, el alcance de validación experimental se limita a esta configuración de tres unidades, que es suficiente para demostrar los mecanismos de coordinación y localización cooperativa. El sistema emplea sensores ultrasónicos para la detección de objetos, por lo que su aplicabilidad queda circunscrita a entornos sin fuertes corrientes de aire, superficies altamente absorbentes o distancias superiores a las que el sensor puede medir con fiabilidad. No se persigue precisión centimétrica en la localización, el cual sería un objetivo que requeriría sensores de mayor coste, sino demostrar la viabilidad de la coordinación espacial y temporal entre nodos como mecanismo para eliminar interferencias y mejorar la cobertura del área monitorizada.

2.4 Restricciones y condicionantes

El desarrollo está sujeto a varias restricciones que condicionan las decisiones de diseño a lo largo de todo el trabajo. En primer lugar, el coste del hardware de cada nodo debe mantenerse en un rango accesible para validar la escalabilidad económica del enfoque, lo que limita la selección a componentes comerciales de bajo coste disponibles en el mercado de electrónica de prototipado. En segundo lugar, la infraestructura de comunicaciones se restringe a una red WiFi estándar, descartando protocolos industriales de mayor fiabilidad pero menor disponibilidad y mayor coste. En tercer lugar, el sistema no puede apoyarse en ninguna infraestructura externa de posicionamiento: la única referencia espacial disponible son las posiciones fijas y conocidas de los propios nodos. Por último, el alcance del trabajo está acotado temporalmente al marco de un Trabajo de Fin de Grado, lo que impone límites sobre el número de iteraciones de diseño y el grado de optimización alcanzable.

2.5 Criterios de Éxito

El trabajo se considerará satisfactorio si el prototipo implementado cumple los siguientes criterios verificables: los nodos se conectan al servidor de forma autónoma y retoman su estado angular tras una reconexión sin intervención manual; el servidor asigna turnos de medida diferenciados a los nodos en situación de interferencia geométrica sin que se registren lecturas espurias atribuibles al *crosstalk* acústico; el sistema produce estimaciones de posición cooperativas cuando la geometría lo permite; la interfaz gráfica refleja el estado de la red en tiempo real con una latencia inferior a un ciclo de medida; y el conjunto del sistema supera sin fallos los escenarios de prueba diseñados para validar los requisitos funcionales definidos en el capítulo correspondiente.

El propósito de este capítulo es exponer el estado actual del arte desde la perspectiva particular del problema que este Trabajo de Fin de Grado busca resolver: la coordinación de varios sensores de medición de distancia de bajo coste para que funcionen simultáneamente y sin interferencias en un entorno perimetral. Para lograr esto, el capítulo se organiza en torno a una progresión intencionada: primero se presenta la naturaleza física del problema que toda solución debe enfrentar, luego se examinan los enfoques que la literatura y la industria han utilizado para abordarlo, y finalmente se concluye identificando la estrategia más adecuada para los requisitos del presente proyecto, destacando en qué se diferencia de las aproximaciones anteriores.

3.1 El sensor ultrasónico como unidad de detección

Para comprender adecuadamente los antecedentes de este proyecto, es conveniente explicar primero el funcionamiento del elemento sensor en el que se basa. Un transductor ultrasónico de bajo costo opera bajo el principio del tiempo de vuelo: emite un pulso acústico de alta frecuencia y mide el intervalo de tiempo hasta que el eco reflejado por un objeto regresa al receptor. Dado que la velocidad del sonido en el aire es aproximadamente 343 m/s, la distancia al obstáculo se calcula directamente a partir de la mitad del tiempo de tránsito [7]. Este principio es análogo al del radar electromagnético clásico [6] y proporciona una precisión aceptable para aplicaciones de corto y medio alcance, con un costo de hardware muy reducido.

El uso de este tipo de sensores para la detección de obstáculos se popularizó en el ámbito de la robótica reactiva desde la década de 1980 [19] y, desde entonces, se ha extendido a entornos de vigilancia perimetral civil, protección de infraestructuras agrícolas [20] y sistemas de vallado virtual [21]. Sin embargo, la cobertura angular de un único transductor es limitada. Para monitorear un área amplia con una resolución angular suficiente, es necesario disponer de varios nodos sensores en distintas posiciones del perímetro, formando lo que la literatura denomina una Red de Sensores Inalámbricos [3].

Es precisamente en este paso, del sensor aislado a la red de sensores, donde surge el problema central que este capítulo analiza.

3.2 El problema central: la interferencia acústica entre nodos

Cuando múltiples transductores ultrasónicos funcionan simultáneamente en el mismo espacio físico, se genera un fenómeno de interferencia acústica conocido en la literatura como *crosstalk* o interferencia cruzada. El mecanismo es fácil de comprender: al emitir un nodo su pulso acústico, las ondas no solo se dirigen hacia el objetivo deseado, sino que se dispersan en todas las direcciones del espacio. Si otro nodo receptor está activo en ese momento, puede captar ese eco ajeno e interpretarlo erróneamente como la reflexión de su propio disparo. Esto resulta en lecturas de distancia incorrectas que comprometen completamente la fiabilidad del sistema [11].

Es crucial señalar que el problema no es de naturaleza software, sino física: las ondas acústicas se propagan sin depender de ninguna lógica de control. Borenstein y Koren [11] caracterizaron experimentalmente este fenómeno en el ámbito de la robótica móvil y demostraron que sus efectos son sistemáticos y reproducibles, además de que se intensifican a medida que aumenta la densidad de nodos en el área monitoreada. En un entorno como el de este proyecto, donde varios nodos radar operan sobre una maqueta de dimensiones reducidas, la cercanía física entre ellos hace que el *crosstalk* sea inevitable si no se gestiona de manera explícita.

3.3 Enfoques para la eliminación del *crosstalk*

La literatura científica y la práctica industrial han abordado este problema desde tres perspectivas diferenciadas. A continuación se analizan sus principios, ventajas y limitaciones. Para vigilar un área perimetral con varios sensores es necesario que estos operen de forma coordinada y, en la medida de lo posible, simultánea. Esta simultaneidad es precisamente la que genera el problema del *crosstalk* descrito en la sección anterior, y es el motivo por el que la gestión del acceso al medio acústico resulta un componente crítico del diseño.

3.3.1 Separación en frecuencia y codificación de canal

El primer enfoque consiste en asignar a cada nodo una frecuencia de emisión distinta o una firma de codificación única, de modo que cada receptor sea capaz de distinguir los ecos propios de los ajenos. Esta estrategia, estudiada por Moebus y Zoubir [12], permite en principio que todos los nodos operen de forma completamente simultánea sin interferencias.

Sin embargo, su implementación en plataformas de bajo coste tiene un problema de raíz: los transductores piezoeléctricos económicos operan en una frecuencia fija y no son reprogramables. Dotar a cada nodo de una frecuencia diferenciada exige transductores especializados y circuitos analógicos de filtrado dedicados, lo que incrementa el coste y la complejidad del hardware de forma incompatible con los objetivos de este proyecto [12].

3.3.2 Barrido secuencial (*Round Robin*)

El segundo enfoque, y el más extendido en implementaciones de bajo coste, consiste en serializar temporalmente las emisiones. El sistema garantiza que en ningún instante haya más de un nodo activo, esperando a que el eco del sensor anterior se disipe completamente antes de activar el siguiente. Este método, conocido como barrido secuencial o *Round Robin*, elimina el *crosstalk* de forma efectiva mediante una solución puramente software [13].

El problema es que la latencia de actualización de la red crece de forma lineal con el número de nodos. Si cada ciclo de emisión-recepción requiere un tiempo mínimo T , una red de N nodos necesita $N \cdot T$ para completar un ciclo completo de mediciones. Teniendo en cuenta que la velocidad del sonido limita físicamente la duración mínima de cada ciclo, el tiempo de respuesta total de la red se vuelve inaceptable para la detección en tiempo real en cuanto el número de nodos crece [13].

3.3.3 Contención con detección de portadora (CSMA/CA)

Un tercer enfoque, ampliamente utilizado en redes de datos inalámbricas como el estándar WiFi, es el acceso múltiple por detección de portadora con prevención de colisiones (CSMA/-CA). En este modelo, cada nodo escucha el canal antes de proceder a transmitir y, si detecta que está ocupado, aplica un retardo aleatorio antes de intentarlo nuevamente.

La dificultad de trasladar este mecanismo al ámbito acústico radica en que la velocidad de propagación del sonido en el aire es aproximadamente un millón de veces menor que la de las ondas electromagnéticas. Esto significa que cuando un nodo verifica que el canal está libre y decide emitir, el pulso de otro nodo puede haber sido enviado instantes antes, pero aún no ha llegado al receptor. El resultado es una colisión tardía que la detección de portadora no puede evitar [11]. Por esta razón, CSMA/CA no es una solución viable en redes de sensores acústicos.

3.3.4 TDMA Dinámico con reutilización espacial

Los tres enfoques anteriores presentan una limitación inherente: suponen que todos los nodos de la red interfieren entre sí en todo momento. Esta premisa es conservadora y, en numerosos escenarios, resulta innecesariamente restrictiva.

Para comprender la alternativa, es útil introducir el concepto de TDMA (Acceso Múltiple por División de Tiempo). En un esquema TDMA, el tiempo se segmenta en intervalos discretos denominados *slots*. Cada nodo tiene asignado uno o varios *slots* en los que puede transmitir, mientras que los demás permanecen en modo de escucha. Esto asegura que las transmisiones sean completamente disjuntas en el tiempo, eliminando el riesgo de colisiones de manera determinista y sin requerir mecanismos de contención [18]. En su forma más elemental, el TDMA es equivalente al barrido secuencial previamente analizado y hereda sus mismas limitaciones

en cuanto a latencia.

La mejora se presenta al incorporar conocimiento espacial en la asignación de *slots*. La literatura sobre redes de comunicaciones se refiere a este principio como reutilización espacial [22]: dos nodos cuyos conos de emisión no se superponen no pueden interferirse entre sí, sin importar si transmiten simultáneamente. Si el árbitro central tiene conocimiento de la posición y la orientación de cada nodo, puede determinar qué pares son físicamente incompatibles y cuáles pueden operar en paralelo sin riesgo.

Con esta información, los *slots* dejan de asignarse de manera fija y estática, y se asignan en función de la compatibilidad espacial calculada en tiempo real. Los nodos que no interfieren comparten un *slot* y transmiten simultáneamente; solo los pares incompatibles reciben *slots* separados [22, 18]. El resultado es una frecuencia de actualización global de la red notablemente superior a la del barrido secuencial, con garantía determinista de ausencia de *crosstalk*.

La Tabla 1 recoge un resumen comparativo de las características de los cuatro enfoques en relación con los requisitos del presente proyecto.

Característica	Barrido cial	Secuen- cial	Contención (CS- MA/CA)	TDMA Dinámico
Riesgo de <i>cross-talk</i>	Nulo		Alto	Nulo
Escalabilidad de la red	Muy baja		Media	Alta
Frecuencia de actualización	Lenta (crecimiento lineal)		Variable e impredecible	Rápida y optimizada
Complejidad del software central	Baja		Media	Alta
Sincronización temporal	No requerida		Relajada	Estricta
Eficiencia del medio acústico	Muy baja		Media	Alta
Tolerancia a la latencia	Pobre		Pobre bajo alta carga	Excelente (determinista)

Tabla 1: Comparativa de técnicas de gestión de acceso al medio en redes de sensores ultrasónicos.

De los cuatro enfoques analizados, el TDMA Dinámico con reutilización espacial es el único que satisface simultáneamente la ausencia de *crosstalk*, la respuesta en tiempo real y un coste de hardware reducido. Es, por tanto, la estrategia adoptada en el presente proyecto.

3.4 Enfoque adoptado y aportación de este proyecto

Del análisis previo se concluye que el único enfoque que satisface simultáneamente los tres requisitos del proyecto, es decir, la ausencia de *crossstalk*, la respuesta en tiempo real y un coste reducido, es el TDMA dinámico con reutilización espacial.

La contribución específica de este estudio en relación con los antecedentes revisados radica en la implementación de este esquema de manera íntegra a través de software, utilizando hardware de consumo masivo y sin la necesidad de componentes analógicos especializados. La coordinación se realiza mediante un servidor central que actúa como árbitro de la red: recibe el ángulo de apuntado actual de cada nodo antes de cada ciclo de medición, calcula qué pares son espacialmente compatibles y asigna los *slots* de emisión en consecuencia, todo ello sobre una red TCP/IP estándar mediante un protocolo de comunicación diseñado específicamente para este proyecto y detallado en el Capítulo 5.

Ninguno de los trabajos revisados combina reutilización espacial dinámica, coordinación puramente software y una plataforma de bajo coste en un sistema de vigilancia perimetral operativo. Es en esta combinación donde se encuentra la novedad técnica del presente trabajo.

Este capítulo examina en profundidad la esencia del problema que el sistema necesita resolver, diferencia el problema auténtico de los desafíos de ingeniería que surgen de él, justifica el enfoque elegido para tratarlo y formaliza los requisitos y limitaciones que han orientado el diseño.

4.1 Descripción del problema

Este capítulo descompone dicho problema en sus dos niveles fundamentales y examina los desafíos técnicos que surgen de cada uno.

4.1.1 El problema real

Como se mencionó en la sección 1.3, el desafío principal de este proyecto radica en la coordinación de múltiples sensores ultrasónicos económicos para que funcionen simultáneamente sin generar interferencias acústicas. El problema se articula en dos niveles distintos.

El primero es el objetivo de aplicación: supervisar un perímetro físico con varios nodos distribuidos que operen en tiempo real. Un solo transductor abarca un ángulo de apertura limitado, por lo que la vigilancia de un área completa requiere una red de sensores ubicados en diferentes posiciones.

El segundo es la dificultad técnica que convierte este objetivo en un reto no trivial: como se analizó en el Capítulo 3, la operación simultánea de varios transductores en el mismo espacio provoca interferencia acústica cruzada. El verdadero problema no es, por tanto, solo vigilar el área, sino hacerlo de manera confiable a pesar de que los propios sensores pueden interferirse entre sí.

El sistema debe responder a la pregunta: *¿cómo coordinar una red de sensores económicos para que monitoricen un perímetro de manera simultánea, sin interferencias mutuas y centralizando la información en tiempo real?*

4.1.2 Problema técnico

Dar respuesta a esa pregunta se traduce en cuatro retos técnicos concretos que estructuran todo el diseño posterior:

- **Selección de la plataforma hardware-software:** El primer reto consiste en elegir los componentes físicos (microcontrolador, sensor de distancia, actuador angular) y el entorno de desarrollo adecuados para los objetivos del proyecto. Esta elección debe equilibrar coste, capacidad de procesamiento en tiempo real, compatibilidad entre componentes y disponibilidad de herramientas de desarrollo accesibles.
- **Concurrencia en el firmware del nodo:** Cada nodo debe realizar simultáneamente varias tareas con restricciones temporales estrictas: controlar el movimiento angular del motor, emitir el pulso acústico en el instante preciso indicado por el servidor, medir el tiempo de vuelo mediante interrupciones de hardware y transmitir el resultado por red. Estas tareas no pueden bloquearse entre sí, lo que exige una arquitectura de firmware basada en un sistema operativo de tiempo real que gestione las tareas de forma concurrente.
- **Arquitectura del servidor de orquestación:** El servidor central debe mantener en todo momento el estado de cada nodo (posición, orientación y estado de conexión) y gestionar dinámicamente su presencia en la red. Dado que la comunicación se realiza sobre WiFi, cuya disponibilidad no está garantizada, el servidor debe detectar desconexiones y reintegrar nodos sin interrumpir la operación del resto del sistema. Este requisito surge de la naturaleza distribuida y no cableada de la red, y no puede ignorarse si se quiere un sistema robusto en condiciones reales.
- **Gestión del medio acústico:** Este es el núcleo algorítmico del proyecto. El servidor debe calcular en cada ciclo qué pares de nodos son espacialmente incompatibles, es decir, cuyos conos de emisión se solapan y por tanto podrían interferirse, y asignar los *slots* temporales de emisión en consecuencia: los nodos compatibles disparan en paralelo; los incompatibles, en *slots* separados. Esta lógica implementa el esquema TDMA dinámico con reutilización espacial descrito en el Capítulo 3.

4.2 Justificación del enfoque adoptado

Una vez identificados los desafíos, es fundamental justificar por qué la solución elegida es la más apropiada dentro de las limitaciones del proyecto, y no simplemente asumir que lo es.

Como se concluye en el análisis comparativo del Capítulo 3, los enfoques alternativos para la eliminación del *crosstalk* presentan limitaciones que los hacen inviables en este contexto: la separación en frecuencia requiere hardware analógico especializado que no se ajusta al requisito de bajo coste, el barrido secuencial genera una latencia que aumenta linealmente y dificulta la respuesta en tiempo real, y el mecanismo de contención CSMA/CA no se adapta

al dominio acústico debido a la lentitud de propagación del sonido. El TDMA dinámico con reutilización espacial es el único de los enfoques analizados que satisface simultáneamente los tres requisitos del proyecto: ausencia de interferencias, tiempo de respuesta aceptable y coste reducido.

La implementación de este esquema recae completamente en el servidor de orquestación, por una razón estructural: los nodos no pueden detectar por sí mismos si el canal acústico está ocupado la baja velocidad del sonido hace inviable cualquier mecanismo de escucha previa, como se explicó en la sección 3.3.3, por lo que la decisión de cuándo puede emitir cada nodo debe ser tomada por un elemento central que tenga una visión global del sistema. Esta arquitectura de control centralizado no representa una limitación del diseño, sino una consecuencia directa de las propiedades físicas del medio acústico.

Como resultado, los nodos periféricos funcionan como actuadores esclavos: llevan a cabo las órdenes del servidor en los momentos precisos que este les indica, sin autonomía propia sobre el acceso al medio.

4.3 Identificación de Requisitos

Los requisitos que se detallan a continuación establecen el comportamiento esperado del sistema en términos observables y verificables. Cada requisito funcional se relaciona directamente con uno o varios objetivos específicos definidos en el Capítulo 2.

Antes de enumerar los requisitos, se identifican los actores del sistema y el alcance de su interacción:

- **El Nodo Sensor (Sistema Periférico):** Componente físico que lleva a cabo las acciones mecánicas de escaneo y la medición acústica de distancias.
- **El Árbitro Central (Sistema Core):** Servidor responsable de recibir datos de telemetría, calcular intersecciones geométricas y emitir las órdenes de disparo a los nodos.
- **El Entorno (Contexto):** Agente pasivo que proporciona los obstáculos físicos sobre los cuales rebotan las ondas ultrasónicas.
- **El Administrador (Usuario):** Responsable de la activación, asignación de coordenadas topológicas en el mapa y supervisión a través de la interfaz gráfica.

4.3.1 Requisitos Funcionales (RF)

Los requisitos funcionales especifican el comportamiento que el sistema debe exhibir en respuesta a entradas y condiciones específicas. La tabla siguiente recoge las funcionalidades

esenciales que el sistema debe satisfacer para cumplir con los objetivos del Capítulo 2.

Código	Descripción
RF01	El sistema operará bajo una arquitectura distribuida en la que el servidor central gestionará de forma autónoma el despliegue de los barridos de cada nodo, garantizando la cobertura del área monitorizada y la eliminación de interferencias acústicas cruzadas sin requerir intervención manual.
RF02	Cada nodo memorizará su última posición angular al cerrar la sesión y la restaurará al reconectarse, evitando movimientos bruscos de recalibración y reanudando el barrido desde el ángulo exacto.
RF03	Los nodos filtrarán internamente las lecturas fuera del rango útil configurado antes de reportarlas al servidor, descartando mediciones espurias producidas por ausencia de eco.
RF04	El servidor calculará las proyecciones geométricas de los haces acústicos en tiempo real y asignará <i>slots</i> de emisión dinámicos: disparos simultáneos para nodos no interferentes y <i>slots</i> separados para los que apunten a zonas de solapamiento.
RF05	El sistema implementará un mecanismo de control de latencia bidireccional: el servidor cerrará la conexión de un nodo que no responda tras un número determinado de ciclos; el nodo se reiniciará lógicamente si no recibe comandos del servidor en el tiempo estipulado.
RF06	Cuando varios nodos detecten un mismo obstáculo en zonas de solapamiento, el servidor ejecutará un algoritmo de trilateración para estimar su posición global, asignará el seguimiento concentrado al nodo con mayor certidumbre y liberará al resto para que retomen el barrido de exploración.
RF07	La interfaz gráfica recibirá tramas de estado desde el servidor y representará visualmente, con una actualización inferior a 1 segundo, la posición estática de cada nodo, su orientación angular en tiempo real y las coordenadas de los obstáculos detectados con indicación de su nivel de criticidad.
RF08	El sistema modificará dinámicamente el régimen de barrido de cada nodo en función de la proximidad del obstáculo detectado: al identificar un objeto en el perímetro de criticidad máxima, el nodo transitará de un barrido de arco completo a un barrido concentrado de alta frecuencia hasta que el objeto abandone esa zona.
RF09	El nodo contará con un chasis modular y estandarizado que aloje e inmovilice los componentes electrónicos, garantice la alineación del eje de rotación y facilite el ensamblaje y el mantenimiento sin herramientas especializadas.

Tabla 2: Requisitos funcionales (RF) del sistema.

4.3.2 Requisitos No Funcionales (RNF)

Los requisitos no funcionales establecen las restricciones y condiciones de calidad bajo las cuales el sistema debe operar.

Código	Descripción
RNF01	El diseño del firmware optimizará el uso de recursos del microcontrolador para reducir el consumo energético operativo, estableciendo las bases para un eventual despliegue autónomo mediante fuentes de alimentación portátiles.
RNF02	La arquitectura de software, tanto en el firmware como en el servidor, seguirá un principio de alta cohesión y bajo acoplamiento, de forma que mejoras algorítmicas futuras puedan integrarse sin necesidad de refactorizar la estructura general del sistema.
RNF03	El sistema se construirá exclusivamente con componentes de hardware comercial de uso general, evitando dependencias de arquitecturas propietarias y garantizando la máxima accesibilidad tecnológica.
RNF04	La totalidad del código fuente y la especificación del protocolo de comunicación se publicarán en repositorios públicos bajo licencia de código abierto, asegurando la auditabilidad del sistema.
RNF05	La pila de comunicaciones y el lazo de control del servidor operarán bajo restricciones estrictas de latencia, empleando optimizaciones a nivel de socket para garantizar que el <i>overhead</i> de red no afecte la frecuencia de muestreo global ni desestabilice la sincronización temporal.
RNF06	La topología hardware, los esquemas de interconexión eléctrica y el diseño mecánico de los soportes estructurales contarán con documentación técnica suficiente para garantizar la replicabilidad del prototipo por terceros.
RNF07	El entorno de software estará acompañado de una guía de despliegue que documente dependencias, cadenas de compilación del firmware y configuración del servidor, asegurando la portabilidad del sistema en plataformas heterogéneas.
RNF09	La escalabilidad horizontal del sistema debe ser económicamente sostenible, optimizando el coste unitario de cada nodo para facilitar la expansión del perímetro vigilado sin incurrir en costes de nivel industrial.
RNF10	La integración electromecánica de los nodos debe ser realizable con herramientas manuales estándar y conexiones prefabricadas, sin necesidad de soldadura de componentes de montaje superficial ni maquinaria de precisión industrial.

Tabla 3: Requisitos no funcionales (RNF) del sistema.

Nota sobre RNF01: La validación energética no se ha llevado a cabo utilizando instrumentación de laboratorio debido a los plazos del proyecto. Su cumplimiento se justifica de manera cualitativa mediante la elección de hardware de bajo consumo y la implementación de un RTOS que bloquea tareas inactivas, previniendo el consumo innecesario de ciclos de CPU, y se establece como una línea de trabajo para futuras iteraciones.

4.4 Restricciones

El desarrollo de este sistema no es el resultado de un proceso de diseño abierto, sino que se basa en la aplicación de restricciones específicas sobre el espacio de soluciones posibles. A continuación, se detallan los factores que han influido en las decisiones de diseño.

4.4.1 Factores Estratégicos

Estos factores surgen de la esencia académica del proyecto y establecen el marco de trabajo.

- **Plazo de desarrollo:** La naturaleza académica del proyecto establece un plazo de entrega innegociable que prioriza la rapidez en el desarrollo sobre la optimización extrema del código. Esto justifica la elección de lenguajes de alto nivel para el servidor y de bibliotecas consolidadas para la conectividad WiFi del microcontrolador, en lugar de desarrollar protocolos desde cero.
- **Presupuesto:** La exigencia de un bajo coste limita las opciones de hardware a componentes estándar de electrónica de consumo, excluyendo soluciones como la modulación cruzada de frecuencias o sensores de mayor precisión, como los que utilizan tecnología láser.
- **Accesibilidad y replicabilidad:** La solución debe ser reproducible y auditable en un entorno educativo estándar, lo que implica el uso de tecnologías de código abierto y hardware de fácil adquisición.

4.4.2 Factores Técnicos

Las condiciones físicas y lógicas en las que funcionará la red de radares establecen las siguientes limitaciones:

- **Límites físicos del medio acústico:** La velocidad del sonido representa una restricción física ineludible. El sistema debe calcular ventanas de guarda temporales fundamentadas en esta constante para asegurar que el eco de un disparo se haya disipado antes de que un nodo adyacente incompatible emita el suyo.
 - **Latencia de red:** A pesar de que las redes WiFi cuentan con un ancho de banda adecuado, la latencia en la transmisión de paquetes pequeños puede resultar considerable. Esto obliga a evitar protocolos de texto pesados en favor de un protocolo binario ligero sobre TCP, con el fin de minimizar el *overhead* de comunicación.
 - **Modelo de control centralizado:** La autoridad para decidir sobre el acceso al medio recae exclusivamente en el servidor árbitro. Los nodos periféricos carecen de capacidad
-

de activación autónoma, lo que asegura un control total y determinista sobre el problema del *crosstalk*, tal como se justifica en la sección 4.2.

- **Entorno de pruebas controlado:** El sistema se valida en un entorno de laboratorio con condiciones acústicas óptimas, lo que elimina la necesidad de técnicas avanzadas de filtrado de señal y permite enfocar el diseño en la lógica de coordinación.
-

CAPÍTULO

5

Diseño

Esta sección describe el diseño y la ingeniería del sistema, considerando las limitaciones y requisitos que se han definido previamente. Se explicarán las decisiones tomadas, incluyendo la elección de la plataforma base, el desarrollo de la arquitectura distribuida, el diseño electrónico, el control algorítmico y el protocolo de red.

5.1 Diseño Hardware

5.1.1 Selección y Justificación de las Decisiones de Diseño Hardware

El diseño de un sistema distribuido económico exige un equilibrio riguroso entre el presupuesto, la capacidad de procesamiento y la fiabilidad del hardware. En esta sección, se describen y justifican los componentes elegidos para formar el nodo periférico, analizando las alternativas que fueron descartadas.

Selección de la Unidad de Procesamiento

Este proyecto requiere un procesador que pueda controlar un motor paso a paso (RF-02), medir los tiempos de vuelo a través de interrupciones de hardware (RF-03) y mantener una comunicación TCP/IP de baja latencia mediante Wi-Fi (RF-04). Esta carga concurrente obliga a dividir la gestión de la red de las tareas físicas para prevenir bloqueos en el ciclo principal.

De acuerdo con los criterios de capacidad multitarea, conectividad nativa y bajo coste unitario (RNF-09), se analizaron tres opciones: microcontroladores estándar (Arduino Uno), ordenadores de placa reducida (Raspberry Pi) y sistemas en chip o SoC (ESP32).

Opción A: Ordenadores de placa reducida (Raspberry Pi)

Se evaluó la viabilidad de emplear una Raspberry Pi Zero W. A pesar de que ofrece conectividad nativa y suficiente potencia, fue desestimada por dos razones fundamentales: su alto coste unitario (lo que infringe el RNF-09) y su considerable consumo energético, lo que complica significativamente la operación continua con baterías (RNF-01). Además, la gestión de los pines GPIO mediante un sistema operativo Linux introduce una latencia no determinista, lo que afecta negativamente la precisión en la medición del sensor ultrasónico.

Opción B: Microcontroladores estándar (Arduino Uno)

Es la plataforma más utilizada para la lectura de sensores y el control de motores. No obstante, fue desestimada debido a su principal desventaja: la falta de conectividad de red nativa. Para satisfacer el requisito RF-04, sería imprescindible incorporar módulos externos (como el ESP8266), lo que incrementa el coste del nodo y complica la arquitectura. Adicionalmente, su único núcleo a 16 MHz presenta retos para mantener un socket TCP abierto mientras gestiona las interrupciones mecánicas sin experimentar bloqueos.

Opción C: SoC ESP32 (Seleccionada)

Representa el equilibrio ideal para los requisitos del proyecto. Su arquitectura de 32 bits con doble núcleo (Tensilica Xtensa Dual-Core) a 240 MHz permite la ejecución de un sistema operativo en tiempo real (FreeRTOS) [23]. Esto soluciona el problema de la concurrencia: un núcleo se dedica exclusivamente a la pila TCP/IP Wi-Fi (RF-04), mientras que el otro gestiona de manera determinista el controlador del motor y los temporizadores de ultrasonido (RF-02 y RF-03). Todo esto se realiza manteniendo un coste por unidad extremadamente bajo y un consumo optimizado (RNF-01 y RNF-09).

Esta elección permite:

- Ejecutar un sistema operativo en tiempo real (FreeRTOS) nativo, dedicando un núcleo exclusivamente a la pila de red Wi-Fi y otro al control determinista de los periféricos físicos.
- Contar con la capacidad de procesamiento concurrente necesaria para medir el tiempo de vuelo con una precisión de microsegundos sin interrumpir la transmisión TCP.
- Disminuir considerablemente el consumo energético y mantener el presupuesto dentro de un rango que posibilite la escalabilidad geométrica de la red.

Con la selección del SoC ESP32 se satisfacen de manera directa los requisitos no funcionales relacionados con el uso de hardware comercial de fácil adquisición (RNF-03), el mantenimiento de una latencia de red mínima (RNF-05) y un estricto bajo coste unitario (RNF-09). Además, su alta eficiencia energética permite un funcionamiento continuo optimizado (RNF-01), un objetivo que habría sido inalcanzable al elegir un ordenador de placa reducida debido a su elevado y constante consumo eléctrico.

La siguiente tabla resume las métricas técnicas que justifican la decisión de selección de la plataforma de procesamiento:

Característica	Arduino Uno	Raspberry Pi Zero W	ESP32
Tipo	Microcontrolador	SBC (Ordenador)	SoC (Micro)
Procesador	16 MHz (8-bit, 1 núcleo)	1 GHz (32-bit, 1 núcleo)	240 MHz (32-bit, 2 núcleos)
Conectividad	Ninguna nativa	Wi-Fi 4 / Bluetooth	Wi-Fi 4 / Bluetooth
Sistema Operativo	Ninguno	Linux (Raspbian)	FreeRTOS (Nativo)
Consumo de corriente	~45 mA (activo)	100–250 mA (activo con Wi-Fi)	10 μ A (deep sleep) – 240 mA (Wi-Fi)
Coste relativo	Medio	Alto	Muy bajo

Tabla 4: Comparativa técnica de plataformas de control.

Es fundamental señalar que el consumo más elevado de la Raspberry Pi Zero W está directamente relacionado con su diseño como ordenador monoplaca, el cual opera un sistema operativo completo basado en Linux y demanda una mayor capacidad de procesamiento. En contraste, el ESP32, al ser un microcontrolador, facilita la gestión de modos de bajo consumo (*deep sleep*) en el orden de los microamperios, conservando picos de consumo comparables a los de la Raspberry únicamente durante las ráfagas de transmisión Wi-Fi.

Selección del Sensor

En las etapas iniciales del diseño, se analizaron diversas tecnologías para la detección de obstáculos y la medición de distancias. Se contempló la utilización de sensores infrarrojos (IR) y medidores láser (LiDAR). No obstante, los sensores IR presentan problemas de fiabilidad en exteriores debido a la luz ambiental, y la tecnología LiDAR excede considerablemente el presupuesto, incumpliendo el requisito no funcional de escalabilidad económica RNF-09.

Por esta razón, se optó por el sensor ultrasónico (tipo HC-SR04) [24]. Esta elección permite obtener datos espaciales confiables a un coste muy bajo, satisfaciendo la necesidad de calcular el tiempo de vuelo (RF-03) y mapear el entorno (RF-01). A pesar de que el cono de dispersión del ultrasonido provoca colisiones acústicas en redes distribuidas, este fenómeno físico es precisamente el que se aborda y resuelve a nivel de software mediante el algoritmo TDMA centralizado (RF-04).

Selección del Actuador Mecánico

Con el fin de satisfacer la necesidad de coordinar los barridos mecánicos (RF-01) y asegurar la correcta orientación del sensor ultrasónico en ángulos predefinidos (RF-02), el sistema necesita un actuador que proporcione precisión en la posición, rotación continua y estabilidad

geométrica. Se analizaron tres opciones tecnológicas de acuerdo con los criterios de control de posición, capacidad de giro y coste (RNF-09).

Opción A: Servomotores estándar (ej. SG90 o MG995)

Representan la alternativa más asequible y sencilla de gestionar a través de señales PWM (*Pulse Width Modulation*). No obstante, fueron desestimados debido a su restricción física de rotación (normalmente limitada a 180° o 270°), lo que imposibilita llevar a cabo barridos perimetrales continuos de 360° . Asimismo, exhiben ligeras vibraciones (*jitter*) al sostener una posición estática, lo que impacta de manera adversa en la lectura del cono ultrasónico.

Opción B: Motores de Corriente Continua (DC) con Encoder

Proporcionan una rotación continua y de alta velocidad. Fueron eliminados debido a que necesitan un control en lazo cerrado complejo PID (Proporcional, Integral, Derivativo) para lograr posiciones angulares precisas. La incorporación de encoders absolutos incrementa considerablemente el coste del conjunto, lo que va en contra del requisito de bajo coste unitario (RNF-09) y complica la electrónica del nodo.

Opción C: Motores Paso a Paso (Seleccionada)

Representan la solución ideal para el control de posición en lazo abierto. Permiten una rotación continua de 360° y un control angular preciso basado en el conteo de pasos generados por el microcontrolador. Se elige el motor paso a paso Usongshine NEMA 17 [25]. Este motor comercial (RNF-03) proporciona un paso angular físico de 1.8° (200 pasos por vuelta), ofreciendo la resolución necesaria para el escaneo sin requerir engranajes reductores. Posee un alto par de retención (*holding torque*), asegurando que el sensor permanezca estático durante la lectura del tiempo de vuelo sin desviaciones.

Para su control, se incorpora el controlador A4988 [26]. Esta decisión es fundamental por dos razones:

- **Control de micropasos (*Microstepping*):** Facilita la subdivisión del paso físico del motor, lo que suaviza el movimiento mecánico. Esto disminuye considerablemente las vibraciones acústicas que podrían provocar falsos positivos en el sensor ultrasónico.
- **Limitación de corriente activa:** Su potenciómetro integrado permite regular el voltaje de referencia para no sobrepasar la corriente nominal, optimizando el consumo energético del nodo y previniendo el sobrecalentamiento en funcionamiento continuo (RNF-01).

La siguiente tabla resume las métricas técnicas que justifican la elección del motor:

Característica	Servomotor (SG90)	Motor DC + Encoder	Paso a Paso (NEMA 17)
Rango de giro	Limitado (180°)	Continuo (360°)	Continuo (360°)
Precisión angular	Media (<i>jitter</i>)	Alta (requiere PID)	Alta (1.8°/paso)
Control de posición	Lazo abierto (PWM)	Lazo cerrado	Lazo abierto (pasos)
Coste	Muy bajo	Alto	Bajo

Tabla 5: Comparativa técnica de motores.

Selección del Lenguaje de Programación del Firmware

Para programar el ESP32, se consideraron lenguajes de alto nivel como MicroPython en comparación con lenguajes compilados como C/C++. Se optó por C++ utilizando el entorno de desarrollo nativo (ESP-IDF / Arduino Core).

A pesar de que MicroPython facilita un prototipado ágil, el cálculo del tiempo de vuelo (ToF) de las señales ultrasónicas exige la manipulación de registros de hardware y la atención a interrupciones en intervalos de microsegundos. El intérprete de Python introduce una latencia inaceptable para estas operaciones de bajo nivel. La elección de C++ asegura la velocidad de ejecución requerida y permite estructurar el código de forma modular (RNF-02), garantizando el envío inmediato de telemetría a través de TCP sin retraso (RNF-05).

Selección del Sistema de Alimentación

Cada nodo presenta dos perfiles de consumo: la electrónica de control (ESP32 y sensor ultrasónico) que requiere 5V estables, y la etapa de potencia (driver y motor) que necesita al menos 12V para garantizar el par y el control de micropaso.

Se descartó tanto la alimentación individual por nodo como una red centralizada a baja tensión. Distribuir 40W a 5V supondría más de 8A en el cableado común, con pérdidas resistivas proporcionales a la longitud del cable. En su lugar se optó por un bus centralizado a 24V alimentado desde una única fuente de 24V y 2,5A (RNF-09), lo que reduce la corriente del bus a menos de 2A para la misma potencia.

Cada nodo incorpora localmente un módulo LM2596 [27] que regula los 24V del bus a los 5V de la electrónica de señal, aislando la etapa de potencia del microcontrolador frente a los picos de arranque del motor.

5.1.2 Especificaciones Hardware

A continuación se describen todos los componentes electrónicos y mecánicos que integran el nodo radar periférico, con el fin de ofrecer una visión completa del hardware empleado en el sistema.

Plataforma de procesamiento local ESP32-WROOM-32D. La unidad de procesamiento elegida para cada nodo es el módulo ESP32-WROOM-32D. Este dispositivo integra el chip ESP32-D0WD, que cuenta con un microprocesador Xtensa LX6 de doble núcleo a 32 bits que funciona a una frecuencia de hasta 240 MHz, junto con 520 KB de SRAM interna y 4 MB de memoria Flash SPI externa. Su arquitectura de doble núcleo es esencial para dividir las tareas concurrentes de control mecánico y comunicaciones de red mediante FreeRTOS. Además, ofrece conectividad Wi-Fi 802.11 b/g/n, Bluetooth v4.2 y una antena PCB integrada, lo que facilita mantener una conexión TCP/IP de baja latencia con el servidor central sin requerir módulos de red adicionales.

Sensor acústico HC-SR04. Para llevar a cabo el mapeo del entorno espacial se utiliza el módulo HC-SR04. Este sensor utiliza un pin digital de salida para emitir el pulso de excitación y un pin de entrada para medir el ancho del pulso de retorno, lo que permite calcular el tiempo de vuelo y, a partir de este, la distancia a los obstáculos presentes en el campo de visión del nodo.

Actuador mecánico Motor paso a paso NEMA 17. El control de la orientación del cabezal sensor está a cargo de un motor paso a paso NEMA 17. A diferencia de los servomotores tradicionales, este tipo de actuador proporciona rotación continua de 360° y un posicionamiento angular preciso basado en el conteo de pasos, sin requerir un sensor de posición continuo. El motor tiene 200 pasos por vuelta y opera con una resolución de micropaso de 1/16, lo que se traduce en 3200 pasos por vuelta completa.

Controlador de potencia Driver A4988. El controlador A4988 funciona como intermediario entre el microcontrolador y el motor paso a paso. Su función primordial es manejar el elevado consumo de corriente de las bobinas del motor mediante señales digitales de dirección y paso que provienen de los pines GPIO del ESP32. Asimismo, implementa el control de micropasos, lo que suaviza el movimiento mecánico. El controlador recibe la tensión de 24V directamente del bus de alimentación, garantizando el par de retención necesario para mantener la posición angular incluso cuando el motor no está en movimiento.

Sensor de referencia angular Reed switch. Cada nodo cuenta con un interruptor de láminas magnéticas montado en la estructura fija del cabezal, alineado con la posición correspondiente al ángulo cero del sistema de referencia local del nodo. En el elemento rotatorio se coloca un imán permanente de pequeñas dimensiones. Cuando el motor orienta el cabezal hacia la posición de cero grados, el imán activa el reed switch, cuya señal es leída por el ESP32 a través de un pin configurado con resistencia de pull-up interna. Este mecanismo proporciona una referencia de posición absoluta que el firmware utiliza durante el proceso de inicialización angular, asegurando que los errores de posición derivados de la pérdida de pasos entre reinicios no se acumulen indefinidamente.

Regulador de tensión Módulo DC-DC LM2596. Cada nodo incluye un regulador reductor LM2596 que transforma la tensión del bus principal a los 5V estables requeridos por el ESP32 y el sensor ultrasónico. Este componente aísla la electrónica de control de los picos de corriente y el ruido eléctrico generados por el motor durante los arranques, protegiendo la integridad de la alimentación del sistema de procesamiento.

Sistema de alimentación centralizado. Se utiliza una fuente de alimentación comercial central de 24V y 60W que distribuye la energía a través de un bus común hacia todos los nodos. El controlador A4988 de cada nodo recibe la tensión de potencia directamente de esta línea, mientras que el regulador LM2596 la convierte a 5V para la electrónica de señal.

En la Figura 1 se puede observar cómo se conectan todos los componentes físicos mencionados del nodo radar al microcontrolador ESP32, utilizando la interfaz de pines GPIO y el controlador A4988 como etapa de potencia.

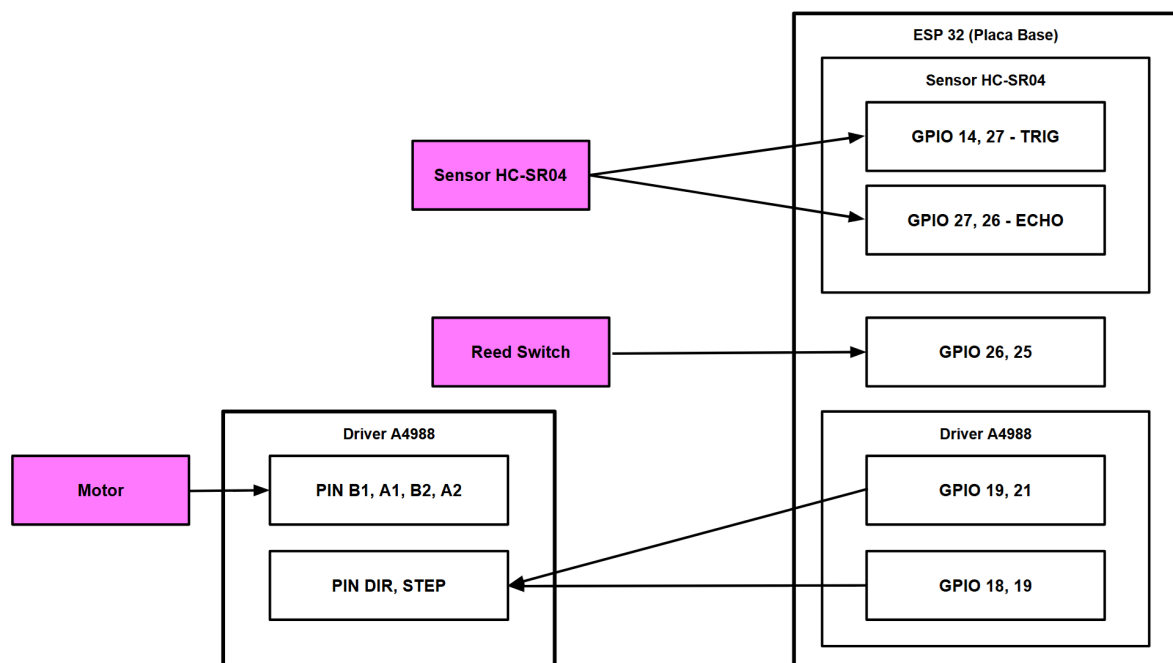
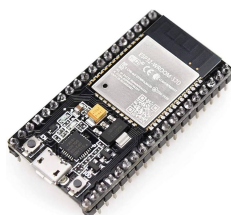


Figura 1: Esquema de conexiones del nodo radar.

A continuación se detallan las especificaciones técnicas de los principales componentes electrónicos:



ESP32-WROOM-32D [28]

Núcleo de procesamiento local del nodo (RF-02, RF-03, RF-04).

Specs: Dual-core Xtensa LX6 a 240 MHz, Wi-Fi 802.11 b/g/n, Bluetooth v4.2, 520 KB SRAM, 4 MB Flash.

Función: Ejecuta FreeRTOS, controla el motor, gestiona el sensor ultrasónico y mantiene la comunicación TCP/IP de baja latencia con el servidor central.



Sensor Ultrasónico HC-SR04 [24]

Transductor acústico (RF-01, RF-03).

Specs: Rango 2 cm – 400 cm, resolución 3 mm, frecuencia de emisión 40 kHz.

Función: Emite y capta pulsos de sonido para calcular el Tiempo de Vuelo (ToF) y medir distancias espaciales.

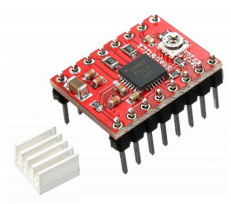


Motor Paso a Paso NEMA 17 [25]

Actuador mecánico (RF-02).

Specs: Ángulo de paso 1.8° (200 pasos/vuelta), resolución con micropaso 1/16 → 3200 pasos/vuelta, alto par de retención.

Función: Ejecuta el barrido perimetral de 360°, orientando el cabezal del sensor con precisión micrométrica.



Driver A4988 [26]

Controlador de potencia del motor (RF-02).

Specs: Soporta hasta 35V / 2A, soporta microstepping hasta 1/16.

Función: Traduce las señales digitales (Step/Dir) del ESP32 en energía para las bobinas del motor.



Regulador Step-Down LM2596 [27]

Convertidor de voltaje DC-DC (RNF-01).

Specs: Entrada hasta 40V, salida estabilizada a 5V (3A máx.).

Función: Reduce y aísla la tensión del bus principal de 24V para alimentar la lógica del ESP32 y el sensor.



Fuente de Alimentación 24V

Fuente de energía centralizada (RNF-09).

Specs: Salida de 24V DC, 60W (2,5A).

Función: Suministra potencia estable al bus común, alimentando directamente la etapa de potencia de los motores.



Reed Switch

Interruptor magnético (RF-02).

Función: Actúa como sensor de fin de carrera para establecer el punto de origen (cero angular) en la calibración inicial del motor.

Tabla 6: Componentes hardware de cada nodo.

5.1.3 Diagrama de conexiones

La Figura 2 presenta el diagrama de conexiones del nodo sensorial, que integra los periféricos con el microcontrolador ESP32 y establece la infraestructura de potencia.

La gestión de la energía utiliza un convertidor reductor LM2596 que aísla eléctricamente el dominio de potencia del motor paso a paso del dominio lógico del microcontrolador. Esta separación previene que el ruido electromagnético y los picos de corriente generados por el motor causen reinicios del hardware en el ESP32 (RNF-01).

El control del motor se lleva a cabo a través del driver A4988. Sus pines de configuración (MS1, MS2 y MS3) están conectados a la línea de 3.3V para fijar una resolución cinemática de 1/16 de paso. Las señales de control STEP y DIR incluyen resistencias pull-down de 10 k Ω , garantizando un estado lógico definido durante la secuencia de arranque.

El sensor ultrasónico HC-SR04 se alimenta a 5V y conecta su pin ECHO directamente al GPIO del ESP32 para la lectura del retorno del pulso. La calibración mecánica se gestiona mediante el reed switch, que proporciona una señal de interrupción estable durante la rutina de homing.

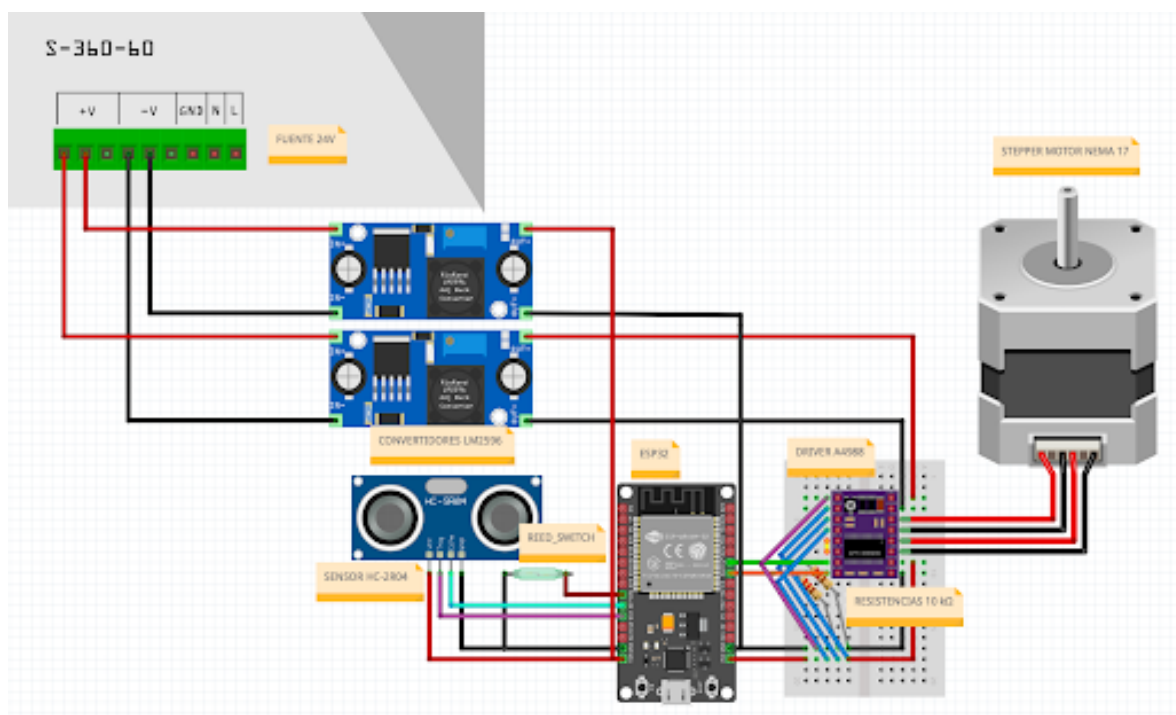


Figura 2: Diagrama de conexiones del nodo radar.

5.1.4 Elementos Estructurales de Soporte en impresión 3D

El diseño mecánico de las piezas estructurales del nodo radar ha sido creado desde cero a través del modelado paramétrico en 3D. Esta elección permite una personalización completa del ensamblaje, asegurando la fijación precisa de los componentes electrónicos y mecánicos, además de reducir significativamente los costos de producción para la escalabilidad de la red (RNF-09). El modelado se ha llevado a cabo utilizando el software Tinkercad.

El diseño tridimensional del nodo ha sido modularizado para simplificar su impresión, ensamblaje y mantenimiento, lo que permite la sustitución de piezas individuales sin necesidad de reconstruir toda la estructura. Se divide funcionalmente en:

- **Chasis (Placas Superior e Inferior):** Base estructural rígida que define el nodo y ancla el resto de los módulos.
- **Soportes de Control (ESP32 y Mini-Protoboard):** Aseguran de manera firme el microcontrolador y la electrónica de potencia.
- **Soportes Electromecánicos (Motor y Reed Switch):** Fijan el motor paso a paso contra la torsión y posicionan el sensor magnético para la calibración inicial (homing).
- **Cabezal Rotatorio (Soporte Sensor HC-SR04):** Se conecta al eje del motor y sostiene el sensor ultrasónico a presión, evitando vibraciones que puedan distorsionar la medición.
- **Elementos Auxiliares:** Piezas menores destinadas a la gestión del cableado interno y al refuerzo entre placas.

El diseño estructural del nodo periférico ha sido concebido de manera modular para facilitar su impresión en 3D y su posterior ensamblaje. En la Figura 3 se muestra el esquema del diseño tridimensional, donde se detallan las diferentes piezas y soportes individuales necesarios para albergar la electrónica y la mecánica del sistema. Una vez que todos estos componentes están integrados, el resultado es un bloque funcional y compacto. Como se ilustra en la Figura 4, las vistas de planta y alzado del ensamblaje final presentan las dimensiones generales expresadas en milímetros. Esta representación define el volumen espacial total que ocupa el dispositivo en su entorno operativo, garantizando las tolerancias físicas requeridas entre el microcontrolador ESP32, el motor paso a paso y el sensor ultrasónico.

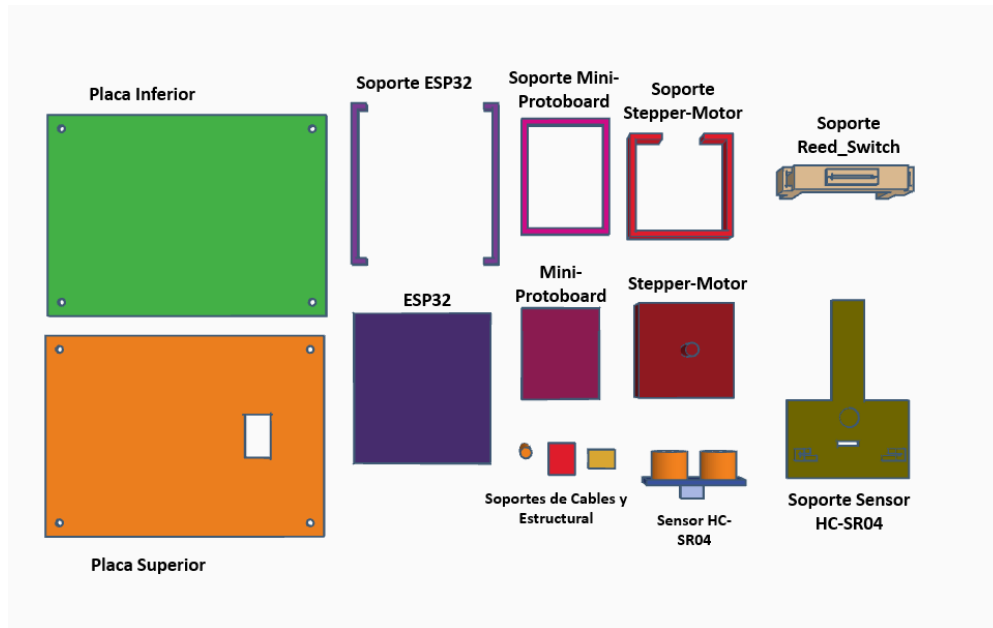


Figura 3: Esquema del diseño tridimensional.

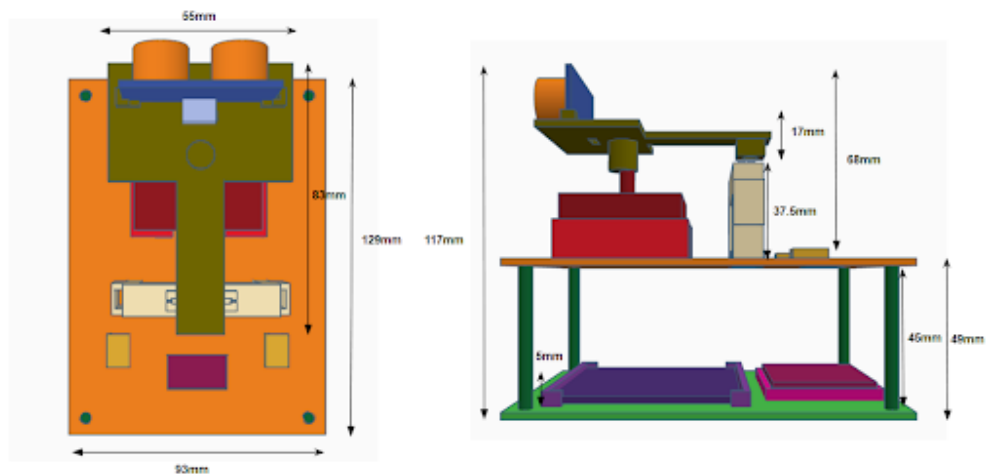


Figura 4: Vistas de planta y alzado del ensamblaje final del nodo periférico con cotas generales (mm).

Evolución Iterativa del Diseño

Para lograr el modelo definitivo, la estructura ha sido sometida a un proceso empírico de mejora continua, abordando diversos desafíos mecánicos en cada etapa. Como se muestra en la Figura 5, el desarrollo atravesó tres diseños preliminares antes de alcanzar el modelo final:

- **Iteración 1 (Figura 5-A):** Enfocada en el diseño general, sin considerar soportes

ni compartimentos del hardware. Consistía únicamente en dos placas superpuestas sin ajuste milimétrico ni dimensiones específicas.

- **Iteración 2 (Figura 5-B):** Rediseño estructural para integrar la electrónica. Se amplió el chasis inferior creando compartimentos para el ESP32 y la protoboard del driver A4988, teniendo en cuenta el espacio necesario para el ruteo del cableado. Se modeló y añadió un soporte lateral específico para el reed switch, esencial para llevar a cabo la rutina de homing mecánico, y se incorporaron orificios en la placa superior para la organización de cables.
- **Iteración 3 (Figura 5-C):** Rediseño total del soporte del sensor HC-SR04 para optimizar el uso del espacio y ajustarlo para la conexión del imán con el reed switch. También se rediseñó el lugar de sujeción del reed switch, con una apertura más amplia hacia los lados y un agujero en el centro para permitir el paso de cables sin interferir con la mecánica del motor.

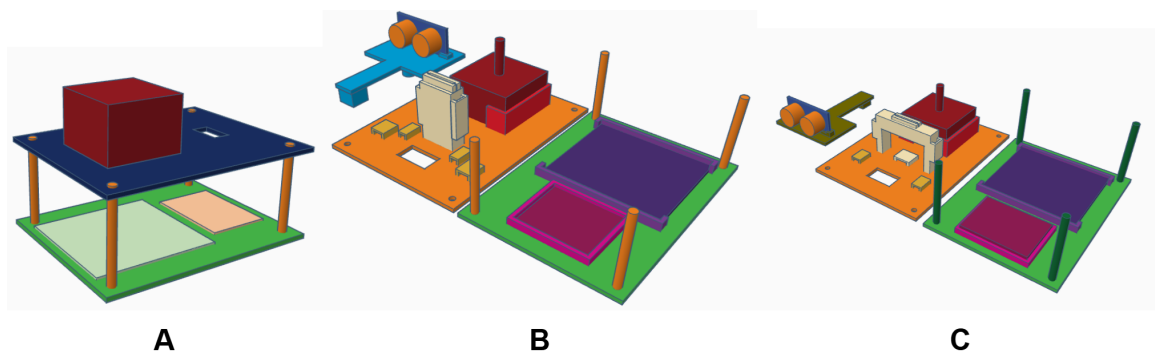


Figura 5: Evolución de los prototipos preliminares del nodo: A) Iteración 1, B) Iteración 2 y C) Iteración 3.

Finalmente, en la Versión Final (Figura 6), se realizó la optimización de los ajustes a presión (*press-fit*) con el objetivo de eliminar cualquier holgura o vibración durante el desplazamiento de todos los compartimentos del hardware, así como el ajuste final de los espacios para el cableado general.

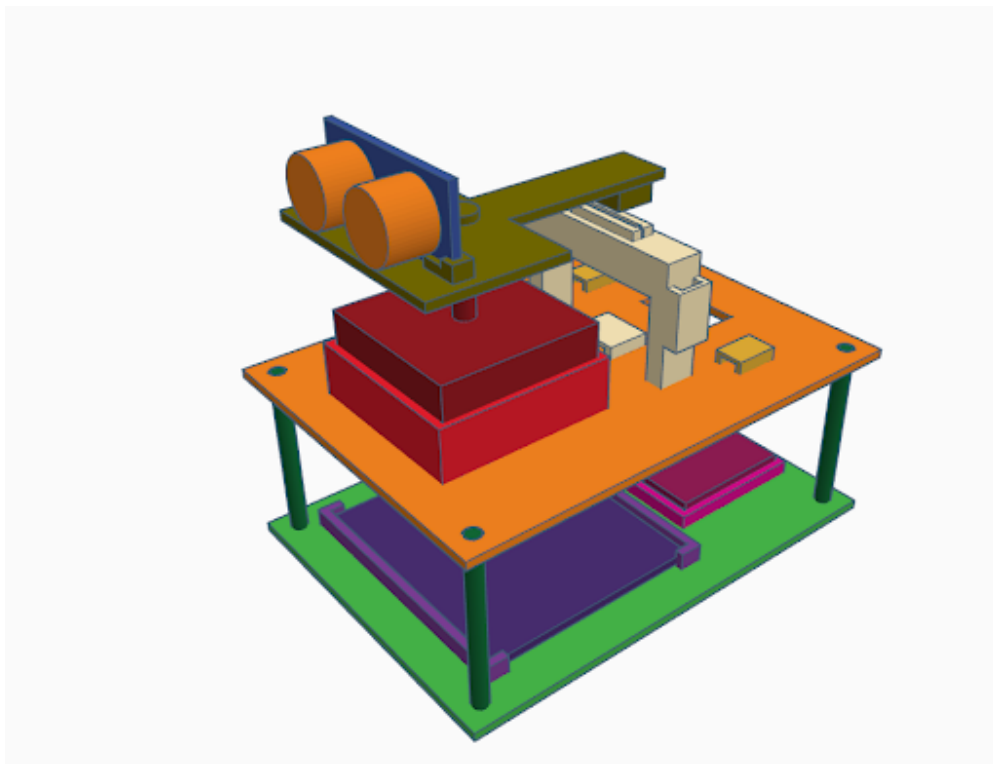


Figura 6: Diseño de la Versión Final del nodo.

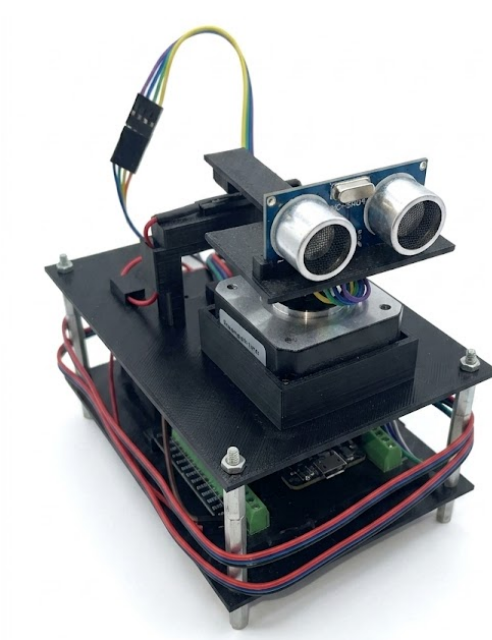


Figura 7: Nodo radar completamente ensamblado.

5.2 Diseño Software

El diseño del software del sistema incluye el servidor central Python y el protocolo de comunicación que lo vincula con los nodos ESP32. Esta sección aborda el sistema desde una perspectiva general hasta llegar a detalles específicos: primero se presenta la arquitectura global, luego la topología de red, a continuación el modelo de datos que establece qué información circula por el sistema, el modelo procedimental que explica cómo se lleva a cabo cada ciclo de medición, y finalmente los modos de operación que definen el comportamiento del sistema en tiempo real.

5.2.1 Visión General de la Arquitectura

El sistema se estructura en cinco capas funcionales que delimitan las responsabilidades de cada uno de sus componentes. En la base se encuentra el hardware físico de los nodos; encima de este, el firmware que lo gestiona; luego, la capa de comunicaciones; posteriormente, el servidor central que toma las decisiones; y en la parte superior, la interfaz gráfica que presenta los resultados. Esta jerarquía asegura que cada capa dependa únicamente de la inmediatamente inferior, lo que facilita el mantenimiento y permite la sustitución de componentes sin impactar en el resto. La Figura 8 ilustra esta estructura.

- **Capa 1: Capa Física y de Hardware:** Constituye la base electromecánica, compuesta por los elementos físicos de cada nodo: el SoC ESP32-WROOM-32D, el sensor ultrasónico HC-SR04, el motor paso a paso NEMA 17 junto con su driver A4988 y el sensor reed para la calibración mecánica. La utilización de hardware comercial de bajo costo asegura la escalabilidad de la red (RNF-03, RNF-09).
 - **Capa 2: Capa de Abstracción de Hardware (HAL):** Representa la capa de abstracción de hardware (HAL), que opera en el Core 1 del microcontrolador y aísla la lógica de control de los aspectos eléctricos de los periféricos, gestionando de manera determinista las interrupciones del sensor y los pulsos del motor (RF-02, RF-03).
 - **Capa 3: Capa de Comunicaciones y Enlace:** Se encarga de las comunicaciones entre los nodos y el servidor: implementa la pila TCP/IP, el protocolo binario y la supertrama TDMA que coordina el acceso al medio acústico sin interferencias (RF-01, RF-04).
 - **Capa 4: Capa de Orquestación y Procesamiento Lógico:** Es el servidor central desarrollado en Python, que centraliza la inteligencia del sistema: procesa la telemetría a través de trilateración, gestiona el cambio entre modos de operación y resuelve los conflictos en las áreas de solapamiento (RF-06, RF-08).
 - **Capa 5: Capa de Presentación e Interfaz de Usuario (HMI):** Es la interfaz gráfica, que presenta el entorno mapeado en un lienzo 2D y ofrece al operador paneles de control y consolas de depuración para gestionar la red (RF-07).
-

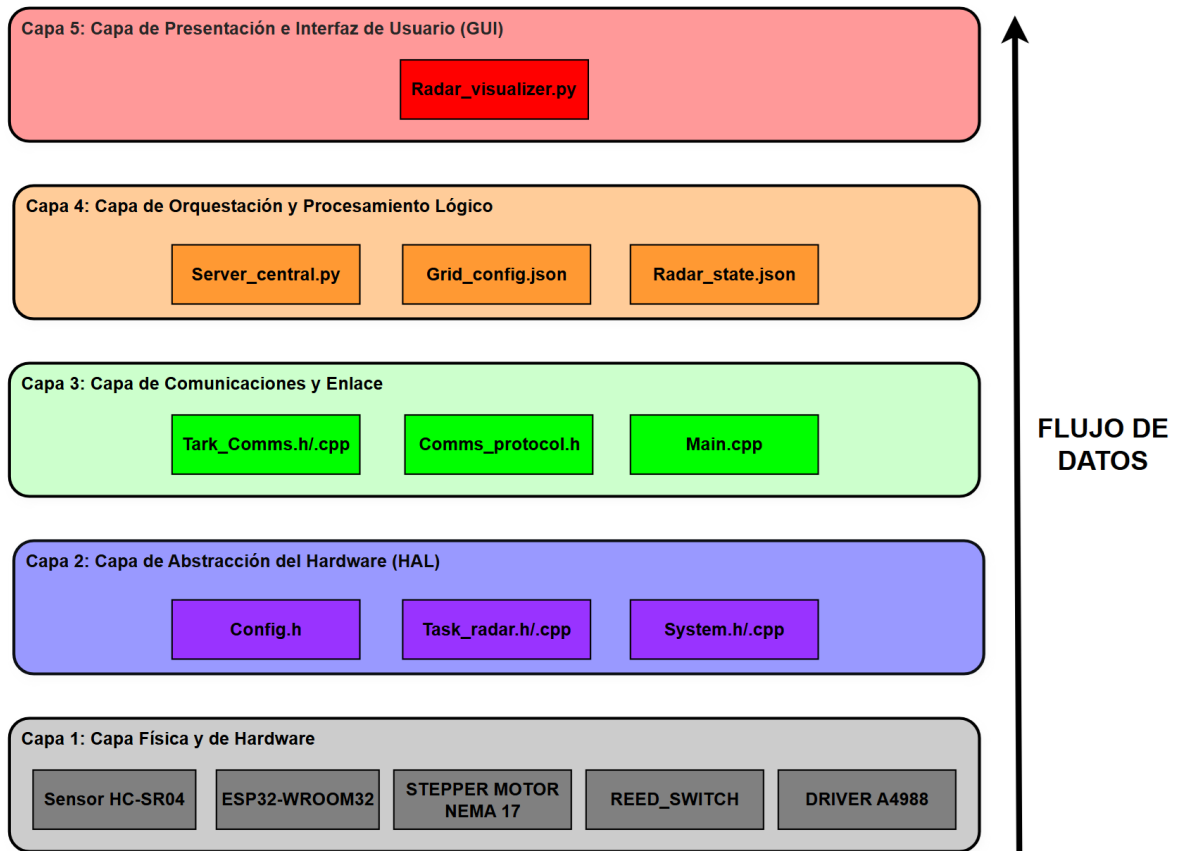


Figura 8: Diagrama de la estructura por capas del sistema.

El servidor central constituye el elemento más intrincado del software. La Figura 9 ilustra su ciclo de vida integral, desde el inicio hasta el bucle continuo de la supertrama. Al comenzar, el servidor registra su dirección en la red a través de mDNS y abre el puerto TCP 8080 para recibir conexiones de los nodos; simultáneamente, inicia un hilo de orquestación y un hilo manejador por cada nodo que se conecte. Una vez que al menos un nodo está conectado, el servidor se adentra en el bucle de supertrama que se repite de manera indefinida: emite el pulso de sincronización, recoge las propuestas de ángulo, calcula las asignaciones de slot, espera a que los nodos realicen las mediciones, recoge los resultados, actualiza el estado del sistema y envía los datos al visualizador. Cada una de las etapas de este bucle se detalla minuciosamente en el punto 5.2.4.

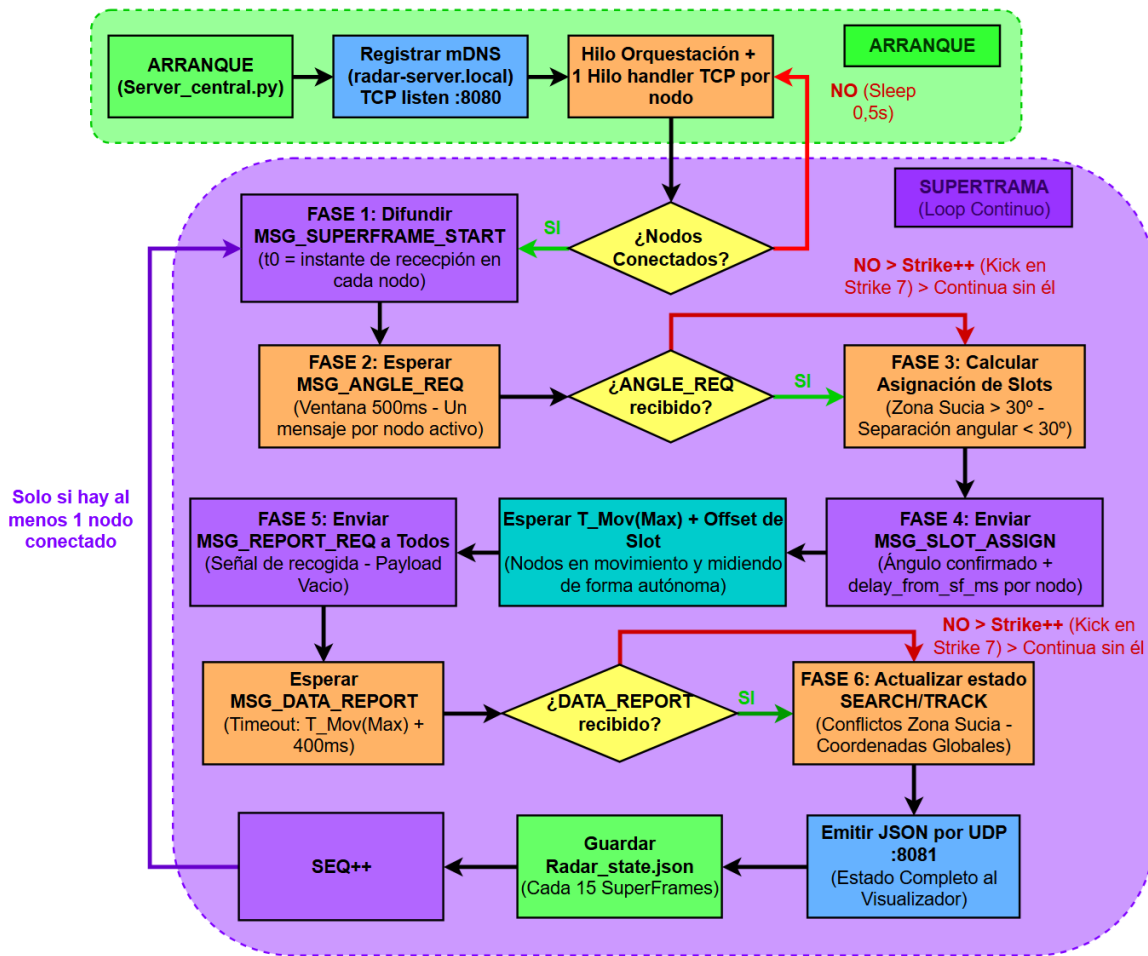


Figura 9: Diagrama de flujo del bucle de orquestación del servidor central.

5.2.2 Topología de Red y Distribución Física

Para que los tres nodos ESP32 y el servidor central puedan coordinarse en tiempo real, el sistema requiere una infraestructura de red que satisfaga tres condiciones: que los mensajes sean recibidos en orden y sin pérdidas, que la latencia sea lo suficientemente baja para no afectar la sincronización temporal del protocolo TDMA, y que los nodos tengan la capacidad de localizar al servidor de manera automática sin necesidad de configuración manual. Esta sección detalla cómo se abordan estas tres condiciones mediante la topología de red adoptada, el mecanismo de descubrimiento automático del servidor y la disposición física de los nodos en el área de vigilancia.

Modelo Cliente-Servidor en Estrella

La red del sistema adopta una topología en estrella con un modelo cliente-servidor, donde el servidor actúa como el único nodo maestro y los tres ESP32 funcionan como clientes periféricos. Como se ilustra en la Figura 10, ningún nodo se comunica directamente con otro; toda la coordinación se realiza a través del servidor. Esta elección se fundamenta en que el algoritmo de orquestación necesita conocer el estado de todos los nodos de manera simultánea antes de tomar cualquier decisión: asignar slots, detectar conflictos y calcular posiciones son operaciones que requieren una visión global que solo posee el servidor. Una topología distribuida o en malla implicaría un protocolo de consenso entre nodos, lo que añadiría latencia de sincronización y complejidad de implementación que son incompatibles con los requisitos temporales del sistema.

La comunicación emplea TCP con conexiones persistentes [29], en lugar de establecer una conexión por cada mensaje. Esto implica que, una vez que el nodo se conecta al servidor, el socket se mantiene abierto durante toda la sesión operativa. Esta elección conlleva tres consecuencias directas: el servidor puede detectar la caída de un nodo de inmediato al cerrarse el socket, sin requerir paquetes de keepalive explícitos; se elimina el costo de establecimiento de conexión (SYN/SYN-ACK/ACK) en cada ciclo, lo que en una red WiFi doméstica puede superar los 10 ms; y el estado de la conexión queda completamente delegado en la pila TCP del sistema operativo, lo que simplifica el firmware del nodo.

Para disminuir la latencia dentro de la conexión persistente, el sistema desactiva el algoritmo de Nagle [29] en todos los sockets TCP y el modo de ahorro de energía del módulo WiFi del ESP32. El algoritmo de Nagle acumula pequeños datos en un búfer antes de enviarlos para mejorar la eficiencia de la red, lo cual es adecuado para transferencias de archivos, pero resulta contraproducente en este caso: con mensajes que oscilan entre 0 y 13 bytes de carga útil y un ciclo de supertrama que depende de que cada mensaje llegue en el momento preciso, un retraso artificial de hasta 200 ms podría desincronizar las mediciones. El modo de ahorro de energía del WiFi introduce retardos similares al poner el módulo en modo de bajo consumo entre transmisiones.

Modelo Cliente-Servidor en Estrella

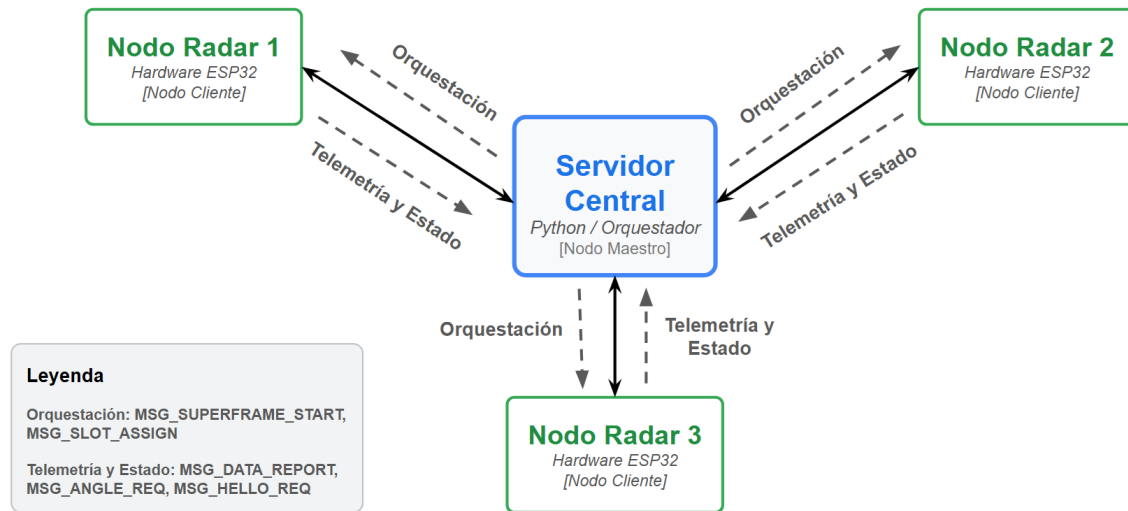


Figura 10: Representación lógica del modelo cliente-servidor en estrella.

Descubrimiento Automático del Servidor

Para que los nodos puedan localizar el servidor sin la necesidad de establecer direcciones IP estáticas, el sistema recurre a mDNS (*Multicast DNS*) [30]. Al iniciar, el servidor registra su dirección bajo el nombre de host `radar-server.local` utilizando la biblioteca `zeroconf`, que publica un registro de servicio del tipo `radar._tcp.local` con todas las interfaces de red activas del servidor. Este mecanismo permite implementar el sistema en cualquier red local sin necesidad de modificar el firmware ni el servidor, dado que la resolución de nombres se realiza de manera automática.

La Figura 11 representa el proceso integral de descubrimiento que lleva a cabo el nodo al establecer conexión con WiFi. En primer lugar, intenta resolver `radar-server.local` a través de mDNS; si no obtiene respuesta en dos intentos consecutivos lo cual puede suceder en redes donde el tráfico multicast está restringido, el nodo opta por utilizar la IP del gateway como alternativa. Este modo de respaldo es efectivo porque, en el entorno de despliegue habitual, el servidor se encuentra en la misma máquina que actúa como punto de acceso, lo que implica que el gateway y el servidor comparten la misma dirección IP. Una vez que la dirección es resuelta, se almacena en caché para que las reconexiones futuras puedan utilizarla directamente sin necesidad de realizar una nueva resolución mDNS.

El firmware también permite la configuración de múltiples redes WiFi de manera simultánea. Al iniciar, el nodo realiza un escaneo activo de las SSIDs disponibles y se conecta a la primera que coincida con alguna de las redes configuradas, siguiendo el orden de preferencia establecido

en **Config.h**. Esta funcionalidad facilita el despliegue del mismo firmware en laboratorio, en el hogar o en el campo sin necesidad de recompilación, lo que acelera el ciclo de desarrollo y pruebas.

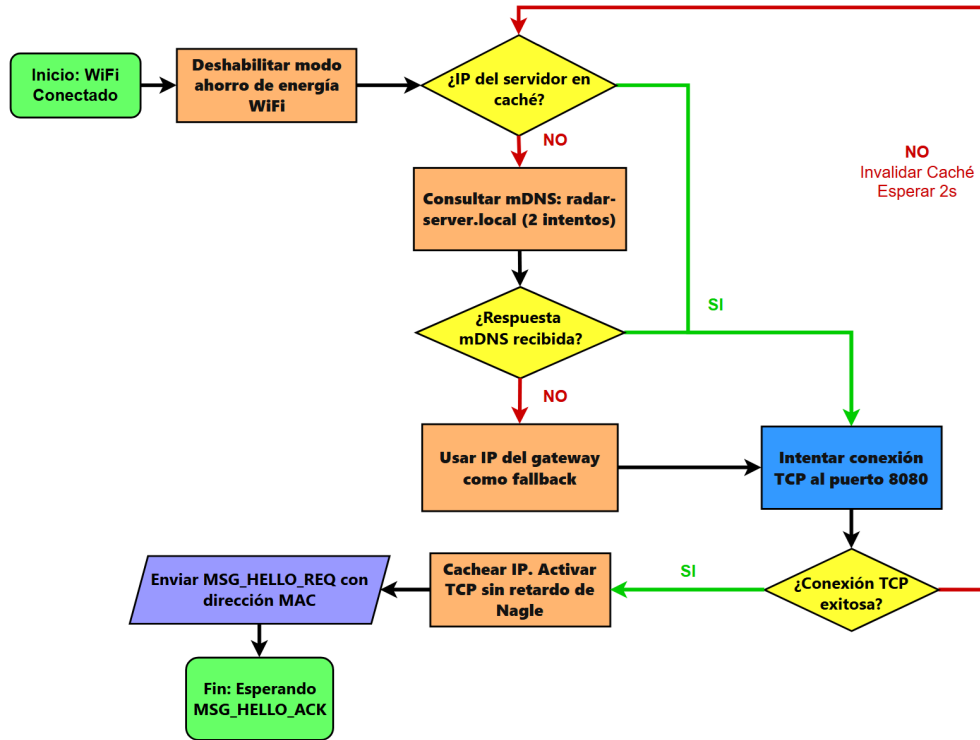


Figura 11: Diagrama de flujo del descubrimiento automático del servidor.

Disposición Física y Zonas de Cobertura

La disposición física de los nodos en el espacio no es aleatoria: influye directamente en la precisión del sistema y en las condiciones bajo las cuales se presenta la interferencia acústica. Los tres nodos se encuentran ubicados en el perímetro del área de vigilancia formando un triángulo, y cada uno tiene asignado un ángulo de orientación θ configurado en `grid_config.json` que dirige su eje frontal hacia el interior del área. La Figura 12 representa esta configuración y los efectos geométricos que genera.

Cada sensor opera dentro de un rango angular de $\pm 45^\circ$ en relación a su eje frontal, lo que proporciona a cada nodo un campo de visión efectivo de 90° . Dado que los nodos están orientados hacia el mismo espacio interior, los campos de visión de los pares adyacentes N1-N2 y N2-N3 se superponen parcialmente en los extremos de ese rango. Esta superposición da lugar a lo que el sistema denomina *zona sucia*: la región angular donde un nodo, al apuntar hacia el extremo de su campo de visión, puede estar dirigiéndose en la misma dirección que el nodo vecino. El umbral que define la zona sucia está establecido en $\pm 30^\circ$ de ángulo local; más allá de este valor, el nodo se considera en zona sucia y susceptible de causar interferencia

con su vecino.

Este solapamiento geométrico tiene dos consecuencias opuestas. Por un lado, es la causa del problema de interferencia acústica que el protocolo TDMA soluciona mediante la asignación de intervalos temporales. Por otro lado, representa una ventaja para la estimación de posición: cuando dos nodos detectan simultáneamente el mismo objetivo en zona sucia, el servidor cuenta con dos mediciones independientes del mismo punto y puede combinarlas para obtener una estimación más sólida de las coordenadas globales, tal como se detalla en el punto 5.2.5.

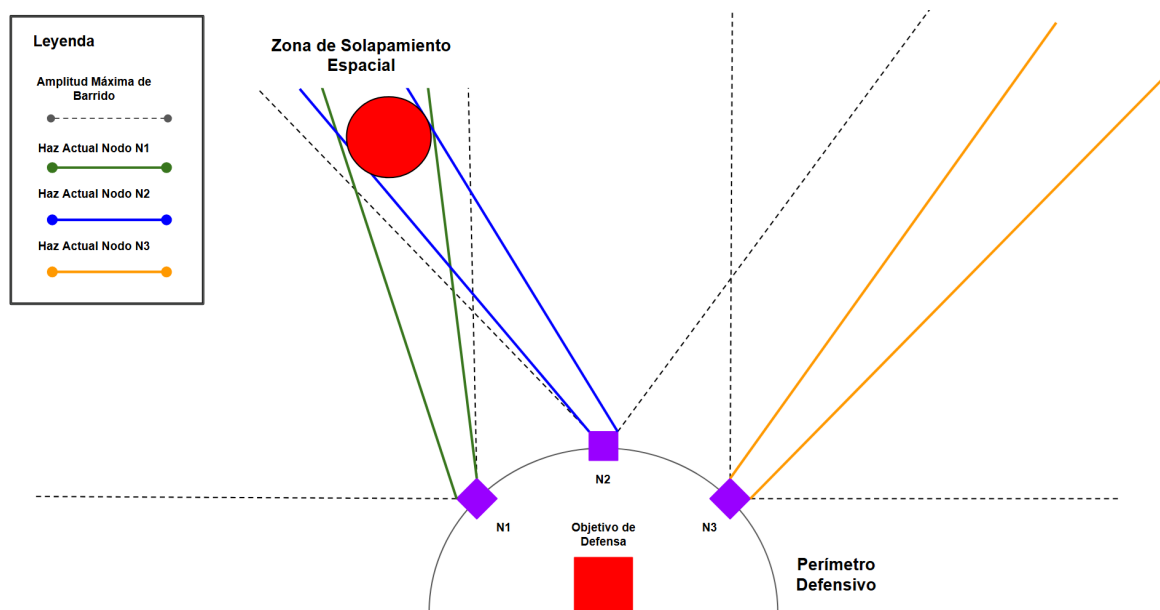


Figura 12: Abstracción geométrica de la distribución perimetral, orientaciones acústicas y áreas de intersección (*crosstalk*).

5.2.3 Modelo de Datos: Protocolo de Comunicación TCP

El modelo de datos del sistema se estructura en dos niveles. En el nivel de persistencia, `grid_config.json` guarda la configuración física de cada nodo (MAC, posición, ángulo θ), mientras que `radar_state.json` conserva el estado SEARCH/TRACK entre sesiones. En el nivel de comunicación, los siete mensajes del protocolo TCP binario forman las estructuras de datos que se transmiten en tiempo real entre el servidor y los nodos, siendo este el enfoque principal de esta sección.

La interoperabilidad entre los nodos periféricos y el servidor central se logra mediante un protocolo de comunicación de nivel de aplicación diseñado específicamente sobre la pila TCP/IP. La elección de TCP como capa de transporte responde a tres requisitos arquitectónicos fundamentales.

Primero, el protocolo de orquestación exige que los mensajes lleguen al nodo en el orden

exacto en que fueron enviados por el servidor. Un paquete de asignación de ranura temporal que llegue antes del pulso de sincronización podría resultar en un cálculo incorrecto del instante de medición, comprometiendo la coherencia temporal de toda la red de sensores. TCP garantiza este orden a través de su mecanismo de numeración de secuencia, sin requerir lógica adicional en el firmware.

En segundo lugar, TCP proporciona retransmisión automática en caso de pérdida de paquetes, eliminando la necesidad de implementar recuperación de errores a nivel de aplicación. Esto es especialmente relevante en el contexto del ESP32, donde los recursos de cómputo y memoria son limitados, y cada ciclo de CPU dedicado a la gestión de errores compite directamente con el control del motor y la adquisición del sensor.

En tercer lugar, la conexión persistente de TCP permite al servidor identificar de inmediato la caída de un nodo sin necesidad de paquetes de keepalive explícitos, dado que el cierre del socket activa automáticamente la lógica de recuperación. La penalización en latencia que TCP introduce en comparación con UDP es aceptable, ya que el protocolo opera a la escala de superframe, con ciclos que varían entre 300 y 1 400 milisegundos.

Diseño de la Trama Binaria

Se descartan los formatos de intercambio de texto (JSON/XML) debido a su alto coste de serialización, al consumo intensivo de ciclos de CPU y a la gestión dinámica de memoria, factores críticos en un entorno de tiempo real bajo FreeRTOS. El formato binario adoptado facilita el mapeo del búfer de red directamente sobre estructuras de datos nativas, lo que conlleva un coste computacional mínimo y constante.

La trama se compone de una cabecera fija de tres bytes y una carga útil variable. El primer byte determina el código de operación (OpCode). Los dos bytes siguientes, codificados en little-endian, especifican la longitud exacta del payload, permitiendo el procesamiento del flujo de datos sin delimitadores de trama. Se omite una suma de verificación a nivel de aplicación, confiando la integridad al checksum nativo del protocolo TCP. La Tabla 7 detalla la estructura de estos tres campos:

Bytes	Campo	Tipo	Descripción
0	Código de operación	uint8_t (1 byte)	Identifica el tipo de mensaje
1-2	Longitud del payload	uint16_t little-endian (2 bytes)	Número de bytes de la carga útil
3...N	Payload	Variable	Datos del mensaje en formato little-endian

Tabla 7: Estructura de la cabecera de trama (*Packet Header*).

Diccionario de Comandos

El protocolo define siete tipos de mensaje que cubren el ciclo de vida completo de un nodo: registro inicial, sincronización de superframe, negociación de ángulo, asignación de ranura temporal, solicitud de medición y reporte de telemetría. La comunicación es bidireccional y asimétrica: el servidor actúa como maestro de la orquestación y los nodos responden dentro de las ventanas temporales establecidas, tal como recoge la Tabla 8:

OpCode	Dirección	Nombre	Payload	Descripción
0xA0	Nodo → Servidor	MSG_HELLO_REQ	6 bytes	Registro inicial. El nodo envía su dirección MAC.
0xA1	Servidor → Nodo	MSG_HELLO_ACK	5 bytes	ID numérico asignado y ángulo guardado de la sesión anterior (RF-02).
0xB0	Servidor → Nodo	MSG_SUPERFRAME_START	8 bytes	Pulso de sincronización global. El nodo captura el instante de referencia temporal t_0 .
0xB1	Servidor → Nodo	MSG_SLOT_ASSIGN	10 bytes	Ángulo objetivo y retardo temporal de medición relativo a t_0 (RF-04).
0xB2	Servidor → Nodo	MSG_REPORT_REQ	0 bytes	Señal de solicitud de medición. Payload vacío.
0xC0	Nodo → Servidor	MSG_ANGLE_REQ	9 bytes	El nodo propone el ángulo del siguiente ciclo de barrido.
0xD0	Nodo → Servidor	MSG_DATA_REPORT	13 bytes	Telemetría: ángulo de medición, distancia detectada y marca temporal (RF-01, RF-06, RF-07).

Tabla 8: Diccionario de mensajes del protocolo.

Especificación Detallada de Cada Mensaje

Registro e Identificación: MSG_HELLO_REQ y MSG_HELLO_ACK. Al establecer la conexión TCP, el nodo envía inmediatamente su dirección MAC en seis bytes consecutivos, uno por octeto, siguiendo el orden natural de la dirección de hardware. El servidor la compara con el archivo de configuración de nodos, que asocia cada dirección MAC a un identificador numérico, una ubicación física en el plano de vigilancia y un ángulo de orientación. Si la dirección no se encuentra en el archivo, el nodo se registra automáticamente con valores predeterminados, lo que permite su calibración posterior sin interrumpir el funcionamiento del sistema.

La respuesta MSG_HELLO_ACK contiene dos campos: el identificador numérico asignado al nodo en el primer byte, y el ángulo en el que quedó el motor al finalizar la sesión anterior, codificado como un número en coma flotante de precisión simple en los cuatro bytes siguientes. Este

segundo campo facilita la reanudación de la operación sin necesidad de realizar un ciclo de homing completo, ya que el servidor conserva la última posición registrada de cada nodo.

Byte	Campo	Tipo	Descripción
0	Identificador de nodo	uint8_t	ID numérico asignado por el servidor (1, 2 o 3)
1–4	Ángulo guardado	float little-endian	Última posición angular registrada en sesión anterior

Tabla 9: Mapa de bytes de MSG_HELLO_ACK.

Pulso de Sincronización Global: MSG_SUPERFRAME_START. Este mensaje es el más importante del protocolo desde una perspectiva temporal. Su carga útil contiene el número de secuencia del superframe como un entero sin signo de cuatro bytes, seguido del tiempo del servidor en milisegundos, también representado en cuatro bytes. Sin embargo, ninguno de estos valores afecta los cálculos de sincronización del nodo: la referencia temporal t_0 se refiere al momento físico de recepción del mensaje, que se registra en el contador interno del microcontrolador, y no a un valor transmitido. Este diseño elimina la dependencia del sesgo de reloj entre el servidor y el nodo, garantizando que todos los cálculos de temporización sean relativos a un instante local medido con la precisión del oscilador del ESP32.

Propuesta de Ángulo: MSG_ANGLE_REQ. Justo después de registrar t_0 , el nodo determina el siguiente ángulo de su patrón de barrido y lo envía al servidor. La carga útil incluye tres campos: el identificador del nodo en un byte, el ángulo actual del motor en coma flotante de cuatro bytes, y el ángulo propuesto para el siguiente ciclo en otros cuatro bytes. Este mensaje es fundamental para el mecanismo de negociación: el servidor conoce la posición actual y la propuesta de todos los nodos antes de realizar las asignaciones, lo que le permite estimar los tiempos de movimiento y detectar posibles conflictos entre sensores adyacentes.

Asignación de Ranura Temporal: MSG_SLOT_ASSIGN. Este mensaje es la respuesta del servidor tras recibir las propuestas de todos los nodos activos. Contiene el ángulo confirmado para el ciclo —el cual puede diferir del propuesto si el servidor lo ajusta por un conflicto de zona sucia— y el campo de retardo temporal, que indica cuántos milisegundos después de t_0 debe llevarse a cabo la medición. El nodo calcula el instante absoluto de medición sumando este retardo al valor de t_0 registrado al inicio del ciclo.

La relevancia de fijar t_0 en MSG_SUPERFRAME_START en lugar de en MSG_SLOT_ASSIGN se debe a que el jitter en la entrega WiFi impacta ambos mensajes de manera significativamente distinta. El pulso de sincronización se distribuye a todos los nodos de manera inmediata con un jitter mínimo de aproximadamente 5 ms. En contraste, el mensaje de asignación se envía tras un ciclo que involucra cálculos de movimiento, estimaciones de tiempo y resolución de conflictos, acumulando un retardo variable que oscila entre 50 y 200 ms. Si se anclara en su recepción, ese jitter se trasladaría directamente al momento de medición. La fórmula del retardo es:

$$\text{delay_from_sf_ms} = t_{\text{elapsed}} + T_{\text{mov,max}} + n_{\text{slot}} \cdot T_{\text{slot}} \quad (5.1)$$

donde t_{elapsed} representa el tiempo transcurrido desde el inicio del superframe hasta el envío de la asignación, $T_{\text{mov,max}}$ es el tiempo del movimiento más prolongado del ciclo con un margen de 100 ms, T_{slot} es la duración de cada ranura (50 ms) y $n_{\text{slot}} \in \{0, 1\}$ es el índice de la ranura asignada. La Tabla 10 recoge el mapa de bytes completo de **MSG_SLOT_ASSIGN**.

Byte	Campo	Tipo	Descripción
0	Identificador de nodo	uint8_t	Nodo destinatario de la asignación
1	Ranura asignada	uint8_t	Índice de ranura temporal: 0 o 1
2-5	Retardo desde t_0	uint32_t little-endian	Milisegundos desde t_0 hasta el instante de medición
6-9	Ángulo confirmado	float little-endian	Ángulo objetivo validado por el servidor en grados

Tabla 10: Mapa de bytes de **MSG_SLOT_ASSIGN**.

Solicitud y Reporte de Medición: MSG_REPORT_REQ y MSG_DATA_REPORT. Una vez completados todos los slots del ciclo, el servidor envía **MSG_REPORT_REQ**, un mensaje con payload vacío que actúa como señal de activación para que cada nodo transmita el resultado de su cola interna. En condiciones normales la medición ya está disponible al recibirla, ya que el motor ha terminado su movimiento y el sensor fue activado en el momento previsto. El firmware establece un tiempo de espera de 300 ms: si no hay resultado disponible, el ciclo se descarta sin interrumpir el siguiente pulso de sincronización.

La respuesta del nodo, **MSG_DATA_REPORT**, es el mensaje con mayor carga útil del protocolo, con trece bytes. El campo de distancia tiene tres valores semánticos: positivo indica detección válida en centímetros, cero indica ausencia de eco, y negativo indica medición inválida por objeto situado por debajo del umbral de operación confiable del HC-SR04. La Tabla 11 detalla el mapa de bytes completo:

Byte	Campo	Tipo	Descripción
0	Identificador de nodo	uint8_t	Nodo emisor de la telemetría
1-4	Ángulo de medición	float little-endian	Ángulo en grados en el que se realizó la medición (-45° a $+45^\circ$)
5-8	Distancia	float little-endian	Distancia en cm; 0.0: sin obstáculo; -1.0 : medición inválida
9-12	Marca temporal	uint32_t little-endian	Contador de milisegundos en el instante de la medición

Tabla 11: Mapa de bytes de **MSG_DATA_REPORT**.

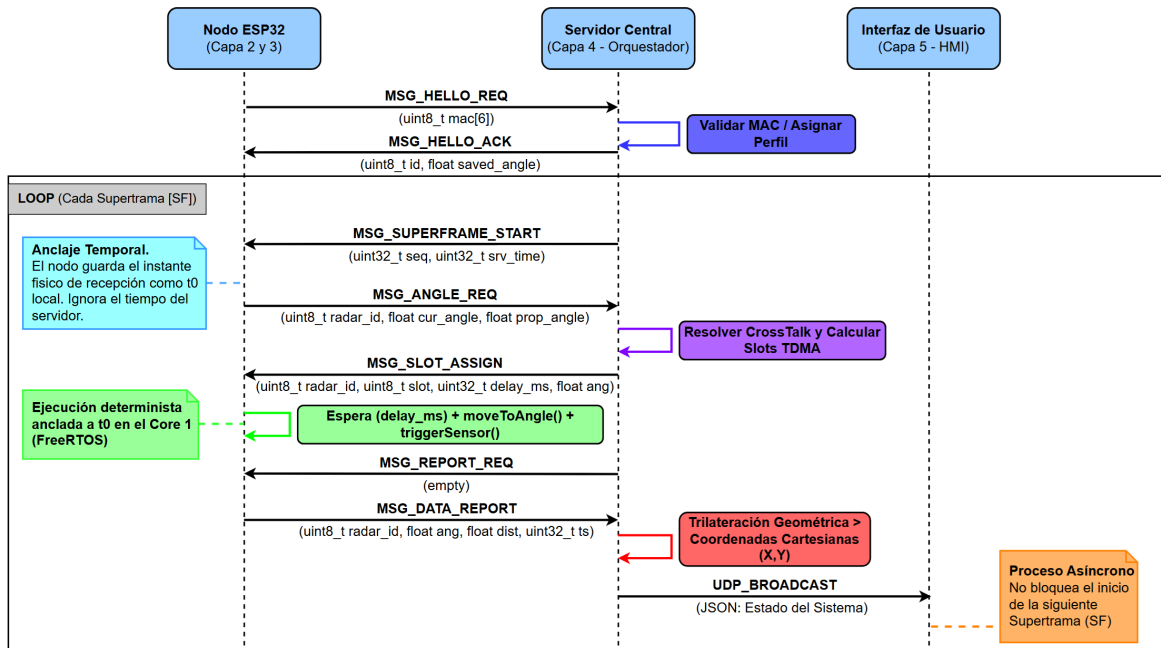


Figura 13: Diagrama de Secuencia UML — Protocolo de comunicación TDMA entre el nodo ESP32 y el servidor central durante una sesión operativa completa.

Servidor central: clase ArbitroServer

El servidor central se implementa íntegramente en el archivo `server_central.py` como una única clase Python, `ArbitroServer`, que concentra toda la inteligencia del árbitro. Aunque la clase constituye una unidad de código, sus responsabilidades se articulan en cuatro agrupaciones funcionales que el diagrama UML de la Figura 14 refleja de forma explícita: el **estado central compartido**, el **orquestador de supertramas**, el **localizador cooperativo** y el **gestor de persistencia**. Los subapartados siguientes documentan cada agrupación con su tabla de variables y métodos correspondiente.

El núcleo de la clase agrupa los atributos de instancia accesibles de forma simultánea desde el hilo de aceptación de conexiones y los hilos individuales de cada nodo: los registros de clientes activos, los búferes de recepción TCP, los diccionarios de estado operativo por nodo, los contadores de tolerancia y las variables de progreso del ciclo. El acceso concurrente a este estado compartido se serializa mediante `lock`, un `threading.Lock` que actúa como árbitro de escritura entre los múltiples hilos. La Tabla 12 recoge sus variables e instancia y sus métodos de gestión del ciclo de vida.

Módulo ArbitroServer — Estado central (server_central.py)			
Clase principal del servidor. Mantiene el estado global compartido entre el hilo de aceptación de conexiones y los hilos individuales de cada nodo. Gestiona las conexiones TCP entrantes, despacha los opcodes del protocolo, administra los contadores de tolerancia y emite el estado completo del sistema al visualizador vía UDP al final de cada supertrama.			
Tabla de variables			
Nombre	Descripción	Tipo de dato	
clients	Mapeo de socket activo al identificador numérico de nodo.	dict[socket, int]	
client_sockets	Acceso inverso: de identificador de nodo a su socket, utilizado para el envío dirigido de mensajes.	dict[int, socket]	
client_buffers	Búfer de recepción TCP por conexión, necesario para parsear tramas de longitud variable sobre un flujo de bytes continuo.	dict[socket, bytes]	
track_states	Estado operativo completo por nodo: modo SEARCH/-TRACK, contadores misses y edge_flips, dirección de barrido y ángulo actual.	dict[int, dict]	
latest_reports	Último reporte de telemetría recibido por nodo en el ciclo en curso.	dict[int, dict]	
reports_received	Conjunto de identificadores de nodo que ya han entregado su reporte en el ciclo activo.	set	
track_lock	Superframe hasta el cual el nodo ganador de un conflicto de zona sucia mantiene inmunidad de TRACK.	dict[int, int]	
strikes	Contador de superframes consecutivos sin respuesta válida por nodo. Al alcanzar el umbral, el servidor cierra su conexión.	dict[int, int]	
grace_until_sf	Superframe hasta el cual un nodo recién conectado no acumula strikes.	dict[int, int]	
requests	Registro de si cada nodo activo ha enviado su propuesta de ángulo en la fase actual.	dict[int, bool]	
seq	Número de secuencia del superframe en curso, incrementado al final de cada ciclo.	int	
current_phase	Fase activa del ciclo de supertrama (0-5), utilizada para sincronizar el orquestador.	int	
running	Bandera de control del bucle principal. En False el servidor concluye ordenadamente.	bool	
lock	Mutex que serializa el acceso concurrente al estado compartido desde los hilos de cliente y el hilo de orquestación.	threading.Lock	
grid	Configuración física de los nodos (posición, orientación θ , MAC). Cargado desde grid_config.json al inicio.	dict	
udp_sock	Socket UDP para emitir el estado completo del sistema al visualizador en el puerto 8081 al final de cada superframe.	socket.socket	
Tabla de métodos			
Nombre	Descripción	Dato(s) de entrada	Dato(s) de salida
__init__()	Inicializa todas las estructuras de estado, carga la configuración de red y el estado de la sesión anterior, y abre el socket UDP.	Ninguno	Ninguno
get_state()	Devuelve el diccionario de estado de un nodo, inicializando sus campos con valores predeterminados si aún no existe registro para ese nodo.	r_id: int	dict
handle_client()	Hilo manejador de la conexión TCP de un nodo: parsea las tramas entrantes y despacha las acciones correspondientes a cada opcode del protocolo.	conn, addr	Ninguno
remove_client()	Elimina todos los registros asociados a un nodo desconectado y cierra su socket.	conn, r_id	Ninguno
start()	Punto de entrada del servidor: registra el servicio mDNS, abre el puerto TCP 8080, lanza el hilo de orquestación y entra en el bucle de aceptación de conexiones.	Ninguno	Ninguno

Tabla 12: Descripción del estado central de ArbitroServer (server_central.py).

Orquestador de Supertramas

El orquestador de supertramas encapsula la lógica temporal del sistema. Las constantes definidas a nivel de módulo parametrizan los tres ejes del algoritmo TDMA: el umbral de zona sucia que determina cuándo dos nodos son incompatibles acústicamente, los límites de tolerancia que controlan las transiciones de modo TRACK, y los parámetros cinemáticos del motor que permiten calcular los retardos de asignación de ranura. El método `orchestration_loop()` ejecuta de forma continua las seis fases de cada ciclo de medición descritas en el apartado 5.2.4. La Tabla 13 documenta sus constantes y métodos.

Módulo OrquestadorSuperframe (server_central.py)			
Encapsula la lógica temporal del ciclo TDMA. Las constantes de módulo parametrizan la detección de conflictos acústicos, el control del modo TRACK y la cinemática del motor. <code>orchestration_loop()</code> ejecuta de forma continua las seis fases de cada supertrama, apoyándose en <code>calculate_movement_time()</code> para calcular los retardos de asignación de ranura.			
Constantes de módulo			
Nombre	Descripción	Valor	
SWEEP_ANGLE_TOTAL	Arco de barrido total del sensor en grados.	90.0°	
A_MAX / A_MIN	Límites simétricos del arco de barrido local del nodo.	+45.0° / -45.0°	
DIRTY_MARGIN	Umbral de zona sucia. Ángulos locales con valor absoluto superior a este umbral implican riesgo de interferencia con el nodo adyacente.	30.0°	
MAX_MISSES	Número máximo de ecos consecutivos no recibidos en modo TRACK antes de regresar a SEARCH.	3	
STEP_ANGLE	Incremento angular por ciclo de supertrama en modo SEARCH, en grados.	2.5°	
TRACK_LOCK_DURATION	Supertramas durante las cuales el nodo ganador de un conflicto de zona sucia mantiene inmunidad de TRACK.	6	
TRILAT_MAX_DIST_RATIO	Ratio máximo entre las dos distancias de un par en trilateración. Si se supera, la estimación cooperativa se descarta.	2.0	
Tabla de métodos			
Nombre	Descripción	Dato(s) de entrada	Dato(s) de salida
<code>orchestration_loop()</code>	Bucle principal de la supertrama TDMA: ejecuta secuencialmente las seis fases de cada ciclo de medición y coordina todos los nodos activos.	Ninguno	Ninguno
<code>calculate_movement_time()</code>	Estima el tiempo de desplazamiento del motor entre dos ángulos en milisegundos, a partir de los parámetros cinemáticos del sistema (<code>MOTOR_STEPS_REV</code> , <code>MICROSTEPPING</code> , <code>STEP_DELAY_MS</code>).	<code>current_angle</code> , <code>target_angle</code> : float	int

Tabla 13: Descripción del orquestador de supertramas (`server_central.py`).

Localizador Cooperativo

El localizador cooperativo agrupa los umbrales de criticidad y los métodos de procesamiento geométrico que permiten al servidor pasar del dominio polar local al dominio cartesiano global. Los cuatro umbrales de distancia delimitan las zonas DEFCON que determinan el nivel de alerta de cada detección y son recargados desde `visualizer_settings.json` al inicio de cada sesión, lo que permite ajustarlos sin recompilar el servidor. Los métodos de cálculo implementan las dos variantes del procesamiento espacial: la transformación directa de una única medición y la trilateración cooperativa cuando dos nodos observan simultáneamente el mismo objetivo. La Tabla 14 detalla sus variables y métodos.

Módulo LocalizadorCooperativo(server_central.py)			
Agrupa los umbrales de criticidad y los métodos de procesamiento geométrico. Los umbrales delimitan las zonas DEFCON que determinan el nivel de alerta de cada detección. Los métodos transforman las mediciones polares de los nodos en coordenadas cartesianas globales y ejecutan la estimación cooperativa de posición mediante trilateración.			
Tabla de variables			
Nombre	Descripción	Tipo de dato	
defcon1_limit	Umbral de distancia en cm de criticidad máxima (zona DEFCON 1). Detecciones por debajo de este valor activan el barrido concentrado.	float	
defcon2_limit	Umbral de distancia en cm de criticidad media (zona DEFCON 2).	float	
defcon3_limit	Umbral de distancia en cm de criticidad baja (zona DEFCON 3).	float	
valla_limit	Umbral de distancia en cm del perímetro exterior de la zona vigilada. Detecciones por encima de este valor se descartan como ruido.	float	
Tabla de métodos			
Nombre	Descripción	Dato(s) de entrada	Dato(s) de salida
calculate_global_coords()	Convierte la medición polar local de un nodo (ángulo y distancia) a coordenadas cartesianas del plano global de vigilancia, aplicando la orientación θ del nodo almacenada en <code>grid</code> .	radar_id, local_angle, distance: float	tuple(x,y) None
calcular_trilateracion()	Estima la posición de un obstáculo compartido mediante la intersección geométrica de dos circunferencias centradas en los nodos del par activo. Si el ratio de distancias supera <code>TRILAT_MAX_DIST_RATIO</code> , descarta la estimación.	id1, d1, id2, d2	tuple(x,y) None

Tabla 14: Descripción del localizador cooperativo (server_central.py).

Gestor de Persistencia

El gestor de persistencia centraliza la interfaz de lectura y escritura de los archivos JSON de sesión. Su función es garantizar que tanto la configuración topológica de la red como el estado operativo de cada nodo sobrevivan entre reinicios del servidor: cuando se vuelve a arrancar, el sistema puede retomar el barrido exactamente donde lo dejó, sin necesidad de repetir el proceso de homing ni de reconfigurar los nodos manualmente. Los métodos de guardado siguen dos estrategias: el guardado explícito, invocado al detener el servidor y acompañado de un mensaje de confirmación por consola, y el guardado silencioso periódico, que se ejecuta cada 15 superframes para reducir el riesgo de pérdida de estado ante caídas inesperadas. La Tabla 15 documenta sus variables y métodos.

Módulo GestorPersistencia (server_central.py)			
Centraliza la interfaz de lectura y escritura de los archivos JSON de sesión. Garantiza que la configuración topológica de la red y el estado operativo de cada nodo sobrevivan entre reinicios del servidor, posibilitando la reanudación del barrido sin necesidad de un homing físico completo.			
Tabla de variables			
Nombre	Descripción	Tipo de dato	
CONFIG_FILE	Ruta al archivo de configuración topológica de nodos (<code>grid_config.json</code>).	str	
STATE_FILE	Ruta al archivo de persistencia del estado operativo de cada nodo (<code>radar_state.json</code>).	str	
VIZ_SETTINGS	Ruta al archivo de configuración de umbrales del visualizador (<code>visualizer_settings.json</code>).	str	
Tabla de métodos			
Nombre	Descripción	Dato(s) de entrada	Dato(s) de salida
load_grid()	Lee <code>grid_config.json</code> y devuelve el diccionario de configuración física de los nodos. Si el archivo no existe, lo crea con valores predeterminados.	Ninguno	dict
save_grid()	Persiste el diccionario <code>grid</code> en <code>grid_config.json</code> .	Ninguno	Ninguno
load_state()	Carga <code>radar_state.json</code> tras confirmación interactiva. Devuelve el diccionario de estados o un diccionario vacío si el usuario rechaza la sesión anterior.	Ninguno	dict
save_state()	Escribe <code>track_states</code> en <code>radar_state.json</code> con mensaje de confirmación por consola. Invocado al detener el servidor.	Ninguno	Ninguno
_save_state_silent()	Guardado periódico silencioso del estado, ejecutado cada 15 superframes sin interrumpir la salida de consola.	Ninguno	Ninguno

Tabla 15: Descripción del gestor de persistencia (`server_central.py`).

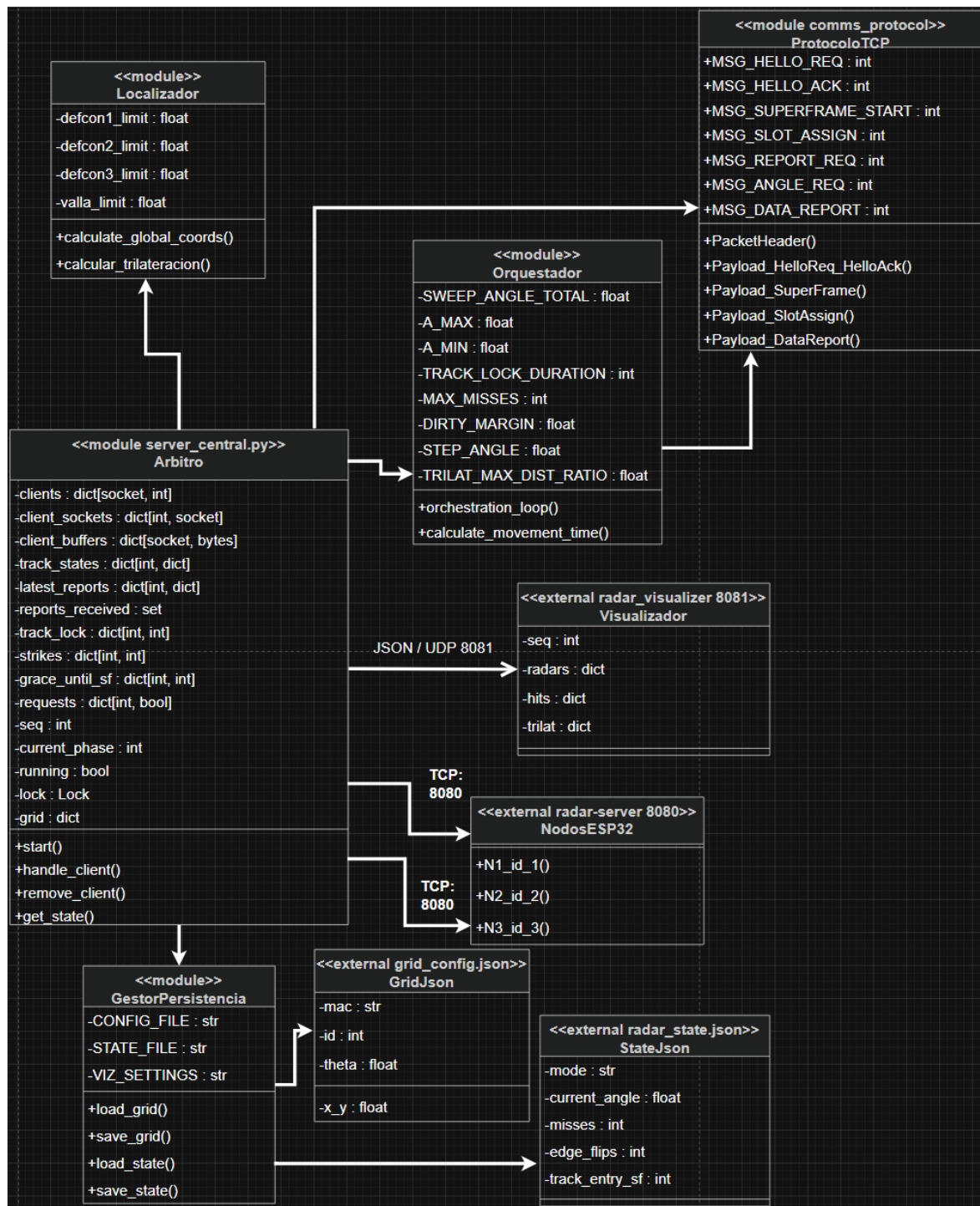


Figura 14: Diagrama de módulos UML del servidor central (server_central.py).

5.2.4 Modelo Procedimental: La Supertrama TDMA

El componente principal del servidor es el bucle de supertrama TDMA, presentado en la Figura 9 del apartado 5.2.1. Cada iteración de este bucle constituye una supertrama: un ciclo completo en el que el servidor organiza a todos los nodos para que realicen sus mediciones de manera ordenada. La supertrama aborda el problema de la interferencia acústica: si dos nodos adyacentes emiten un pulso ultrasónico simultáneamente, el eco que recibe cada uno puede ser el del otro, contaminando la medición. La solución es temporal: se distribuyen las mediciones en el tiempo, asignando a cada nodo una ventana exclusiva dentro del ciclo.

La supertrama se divide en seis fases secuenciales:

- **Fase 1 — Difusión del pulso de sincronización:** El servidor envía `MSG_SUPERFRAME_START` a todos los nodos conectados. Cada nodo registra el momento de recepción como t_0 . Este instante sirve como referencia temporal del ciclo: todos los cálculos de temporización posteriores son relativos a él, no a ningún valor transmitido en el payload. Este diseño elimina la dependencia del sesgo de reloj entre el servidor y los nodos.
- **Fase 2 — Recogida de propuestas de ángulo:** Tras recibir `MSG_SUPERFRAME_START`, cada nodo calcula su próximo ángulo de barrido y lo envía al servidor mediante `MSG_ANGLE_REQ`. El servidor espera hasta 500 ms. Si un nodo no responde, acumula un strike; al séptimo strike consecutivo, el servidor cierra su conexión TCP. Los nodos recién conectados cuentan con ocho superframes de gracia durante los cuales no se les penaliza.
- **Fase 3 — Cálculo de asignaciones de slot:** Con las propuestas de todos los nodos activos, el servidor ejecuta el algoritmo de asignación. Por defecto, todos los nodos reciben el slot 0; este se cambia a 1 si se identifica un conflicto de crosstalk. Existen dos condiciones que generan conflicto: que un par adyacente ($N1-N2$ o $N2-N3$) tenga ambos nodos dirigiéndose hacia la zona sucia (ángulo local superior a $\pm 30^\circ$), o que dos nodos no extremos tengan sus ángulos globales separados por menos de 30° .
- **Fase 4 — Envío de asignaciones y espera:** El servidor envía `MSG_SLOT_ASSIGN` a cada nodo con el ángulo confirmado y el retardo desde t_0 , calculado según la Ecuación 5.1. Una vez enviadas las asignaciones, el servidor espera pasivamente mientras los nodos se desplazan y miden de forma autónoma.
- **Fase 5 — Recogida de resultados:** El servidor envía `MSG_REPORT_REQ` a todos los nodos activos del ciclo, indicando que deben entregar su resultado. Los nodos responden con `MSG_DATA_REPORT`. El servidor espera hasta recibir todos los reportes o hasta que se agote el tiempo de espera; los nodos que no responden acumulan strikes de la misma manera que en la Fase 2.
- **Fase 6 — Procesamiento y emisión:** Con todos los reportes recibidos, el servidor actualiza el estado `SEARCH/TRACK` de cada nodo, resuelve los conflictos de zona sucia, calcula las coordenadas globales de los objetos detectados y emite el estado completo al visualizador a través de UDP en el puerto 8081. El estado se guarda en `radar_state.json` cada 15 superframes. A continuación, el contador de secuencia se incrementa y se inicia la siguiente supertrama.

La secuencia temporal completa de los mensajes intercambiados en estas seis fases se puede seguir en detalle en la Figura 13 del punto 5.2.3.

5.2.5 Modos de Operación y Localización Cooperativa

El comportamiento dinámico del sistema en tiempo real se articula en torno a dos mecanismos complementarios que el servidor gestiona a través de la supertrama TDMA. El primero son los modos de operación individuales de cada nodo: cada dispositivo se encuentra en todo momento en modo SEARCH, ejecutando un escaneo continuo de su sector, o en modo TRACK, concentrando el haz sobre un objetivo confirmado. El segundo es la localización cooperativa: cuando dos nodos pertenecientes a un par de CONFLICT_PAIRS detectan simultáneamente el mismo objetivo en la zona de solapamiento, el servidor no se limita a arbitrar cuál de los dos mantiene el seguimiento, sino que combina las dos mediciones polares independientes para obtener una estimación de la posición global del obstáculo con mayor solidez geométrica [17], tal como ilustra la Figura 15. La Figura 16 presenta el diagrama de estados completo del nodo con las condiciones de transición entre modos y la activación del arbitraje cooperativo.

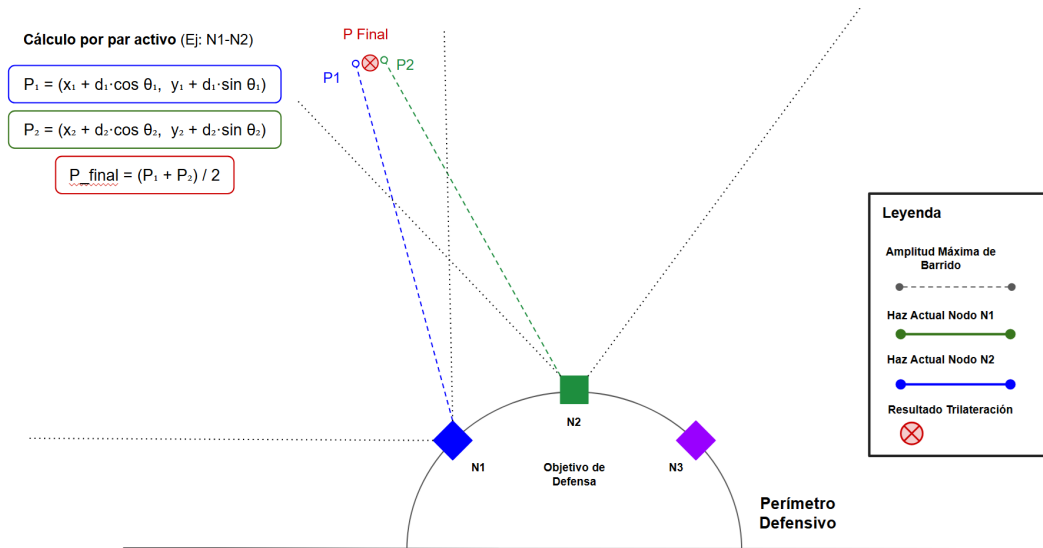


Figura 15: Localización cooperativa por trilateración: estimación de posición global combinando las mediciones polares de dos nodos adyacentes en zona de solapamiento.

Modo SEARCH

En este modo el nodo barre el sector asignado, emitiendo pulsos ultrasónicos en cada posición angular y esperando el eco correspondiente. Si no se recibe respuesta dentro de la ventana de tiempo de vuelo, el barrido continúa sin registrar detección. Cuando un eco retorna dentro del umbral de cobertura en una posición válida, el nodo considera la detección confirmada, registra la supertrama de entrada en `track_entry_sf` y transita a TRACK. Mientras no se

produzca esa detección, el nodo permanece en SEARCH de forma indefinida.

Modo TRACK

Una vez en seguimiento, el nodo opera entre tres subestados según la calidad de la señal: en **MIDIENDO**, el eco se recibe dentro del umbral DEFCON 1 y los contadores de fallo son nulos; en **BORDE**, la medición supera DEFCON 1 indicando que el objetivo se aproxima al límite de cobertura, por lo que se incrementa **edge_flips** y se invierte el sentido del barrido; en **MISS**, no se recibe eco y se incrementa **misses**. Cuando **edge_flips** alcanza **MAX_EDGE_FLIPS** = 5 o **misses** alcanza **MAX_MISSES** = 3, el nodo regresa a SEARCH.

Si dos nodos de **CONFLICT_PAIRS** detectan simultáneamente el mismo objetivo en la zona sucia, el servidor arbitra cuál permanece en TRACK y otorga al ganador una inmunidad de **TRACK_LOCK_DURATION** = 6 supertramas. En caso de detección estable en ambos, se activa la localización cooperativa [17]:

$$P_{\text{final}} = \frac{P_1 + P_2}{2} \quad (5.2)$$

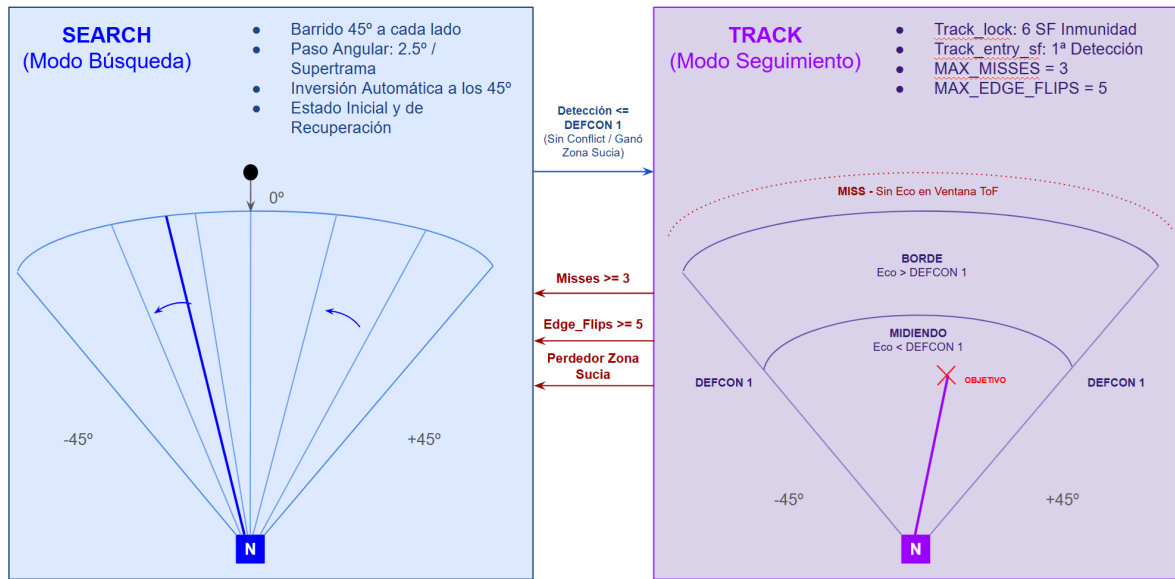


Figura 16: Modos de operación del nodo sensor: SEARCH y TRACK con sus subestados y condiciones de transición.

5.3 Diseño del Firmware

El firmware de los nodos periféricos ha sido desarrollado en C++ utilizando el entorno de abstracción Arduino, con FreeRTOS como sistema operativo de tiempo real. Este apartado se dedica exclusivamente al diseño del nodo sensor: cómo inicia, cómo gestiona su ciclo de vida, qué arquitectura interna lo respalda y cómo se relaciona con el hardware físico.

El comportamiento del nodo se organiza en torno a una Máquina de Estados Finita que controla su ciclo de vida completo, desde el inicio hasta la operación normal. Las secciones siguientes desglosan, de afuera hacia adentro, cómo se define ese comportamiento, qué arquitectura lo respalda, qué estructuras de datos utiliza y qué operaciones físicas lleva a cabo.

5.3.1 Máquina de Estados Finita del nodo

El funcionamiento del nodo sensor se fundamenta en una máquina de estados finita (FSM) que consta de cuatro estados, los cuales regulan todo su ciclo de vida, desde el inicio hasta la operación normal. Cada estado tiene responsabilidades específicas, y el sistema solo avanza al siguiente estado una vez que se han satisfecho todas las condiciones requeridas. La Figura 17 ilustra el diagrama de flujo completo de esta FSM, incluyendo todas las transiciones para la recuperación ante fallos.

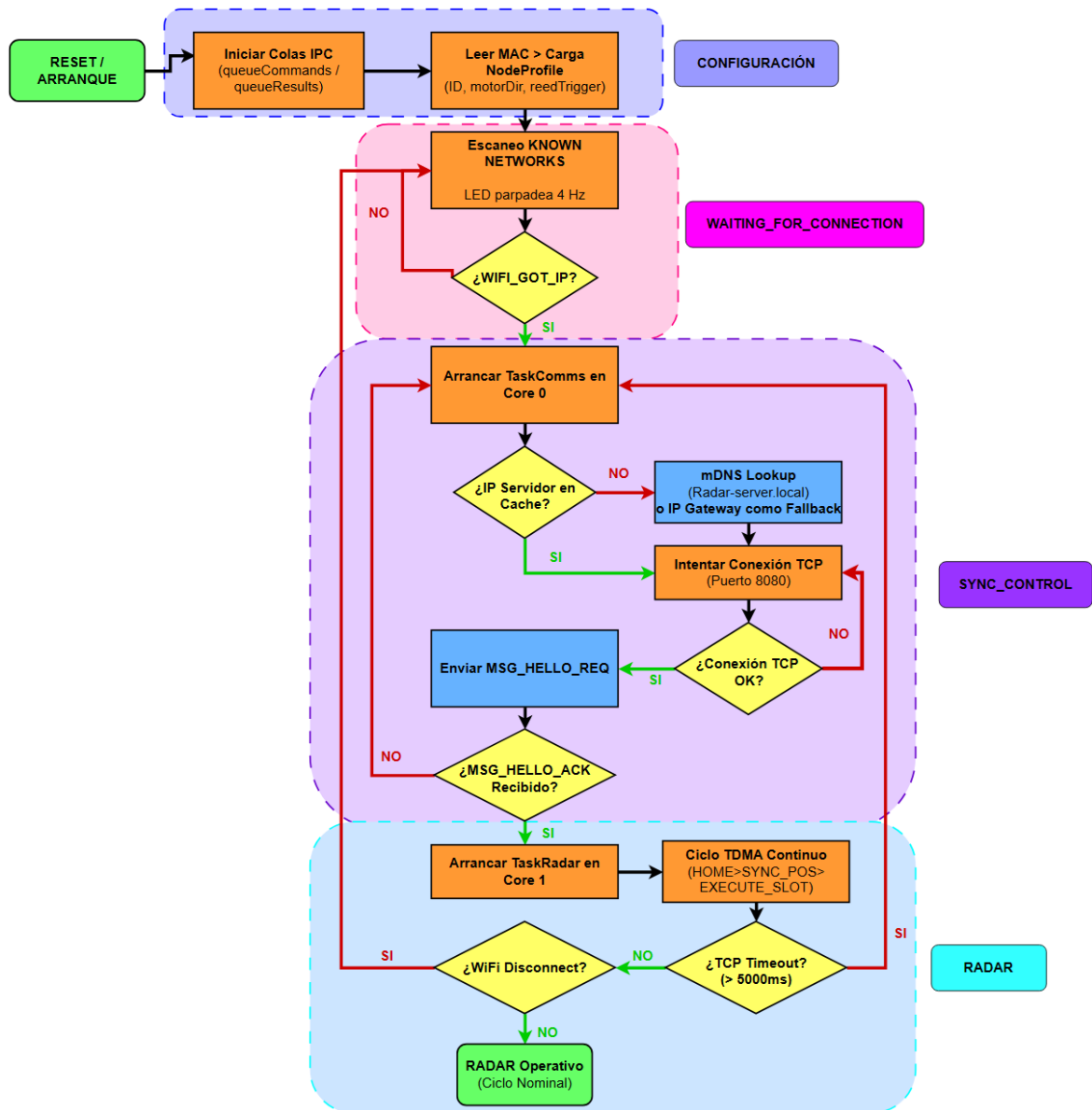


Figura 17: Diagrama de flujo de la FSM del nodo sensor ESP32.

Estado CONFIGURACION

Inmediatamente después de encender o reiniciar el microcontrolador, el nodo lleva a cabo las tareas de inicialización que no requieren conexión a la red. Primero, establece las dos colas de comunicación entre tareas: `queueCommands`, a través de la cual la tarea de comunicaciones envía órdenes al hardware, y `queueResults`, por la que el hardware devuelve mediciones y confirmaciones. Luego, lee su dirección MAC y la utiliza para localizar su entrada en la tabla `NodeProfile`, donde se encuentran sus parámetros hardware específicos: el identificador de

nodo, la inversión de dirección del motor y el nivel de disparo del sensor reed. El identificador de nodo es particularmente crucial, ya que es lo que el nodo enviará al servidor para registrarse durante `SYNC_CONTROL`. Al finalizar esta inicialización, el nodo transita directamente a `WAITING_FOR_CONNECTION`.

Estado `WAITING_FOR_CONNECTION`

Durante este estado, el nodo permanece a la espera de que el WiFi obtenga una dirección IP. Mientras tanto, el LED parpadea a 4 Hz para señalar que el sistema está encendido pero sin conexión a la red. Internamente, la tarea principal escanea la lista de `KNOWN_NETWORKS` y activa el módulo WiFi. El nodo no avanza hasta que recibe el evento `WIFI_GOT_IP` del *stack* de red, ya que sin una dirección IP no hay manera de comunicarse con el servidor. Si no se recibe, el ciclo se repite indefinidamente. Separar la obtención de la dirección IP de la conexión con el servidor es razonable, ya que permite que el nodo funcione sin inconvenientes en situaciones donde el router tarda en asignar una dirección o el servidor aún no está operativo.

Estado `SYNC_CONTROL`

Una vez que se establece la conexión WiFi, el nodo inicia la tarea `TaskComms` en el núcleo 0 y comienza a conectarse al servidor. El primer paso es determinar la dirección IP del servidor: si existe información en caché, se utiliza directamente; de lo contrario, se realiza una consulta mDNS utilizando el nombre `radar-server.local` y, si no hay respuesta, se recurre a la puerta de enlace como alternativa. Con la dirección IP disponible, se intenta establecer una conexión TCP en el puerto 8080, reintentando hasta que se logre con éxito.

Una vez que la conexión TCP se ha establecido, el nodo envía el mensaje `MSG_HELLO_REQ` al servidor. Esto inicia el proceso de identificación: el servidor registra al nodo, le asigna su posición en la supertrama TDMA y responde con `MSG_HELLO_ACK`. La recepción de este acuse es la última condición necesaria para poder operar. Si no se recibe, el nodo cierra la conexión y reinicia desde el inicio de `TaskComms`, lo que permite recuperarse de situaciones en las que la conexión TCP fue exitosa pero el servidor no pudo completar el *handshake*.

Estado `RADAR`

Al recibir el `MSG_HELLO_ACK`, el nodo inicia `TaskRadar` en el núcleo 1 y comienza a operar de manera continua. `TaskRadar` repite indefinidamente el ciclo TDMA que le ha sido asignado por el servidor: `HOME` para asegurar la referencia angular, `SYNC_POS` para ajustar la posición al ángulo indicado y `EXECUTE_SLOT` para realizar la medición ToF en el momento preciso de la supertrama. El ciclo continúa mientras la conexión permanezca activa.

5.3.2 Arquitectura de tareas

El ESP32 cuenta con dos núcleos de procesamiento Xtensa LX6 que el firmware utiliza a través de un esquema de multiprocesamiento asimétrico (AMP) administrado por FreeRTOS [23]. Este diseño se justifica porque las dos funciones principales del nodo presentan requisitos temporales muy diferentes: la comunicación TCP con el servidor puede tolerar variaciones de latencia propias de la red, mientras que el control del hardware requiere tiempos de respuesta deterministas para cumplir con los intervalos de la supertrama TDMA. Integrar ambas funciones en una sola tarea provocaría *jitter* en el motor y el sensor, lo que deterioraría la precisión de las mediciones. La Figura 18 ilustra cómo se distribuyen las tareas entre los núcleos y cómo se comunican entre sí; la Tabla 16 indica la prioridad y el núcleo en el que están alojadas.

Tarea	Núcleo	Prioridad	Pila
TaskComms	Core 0	2	4096 B
TaskRadar	Core 1	3	4096 B

Tabla 16: Configuración de tareas FreeRTOS del nodo sensor.

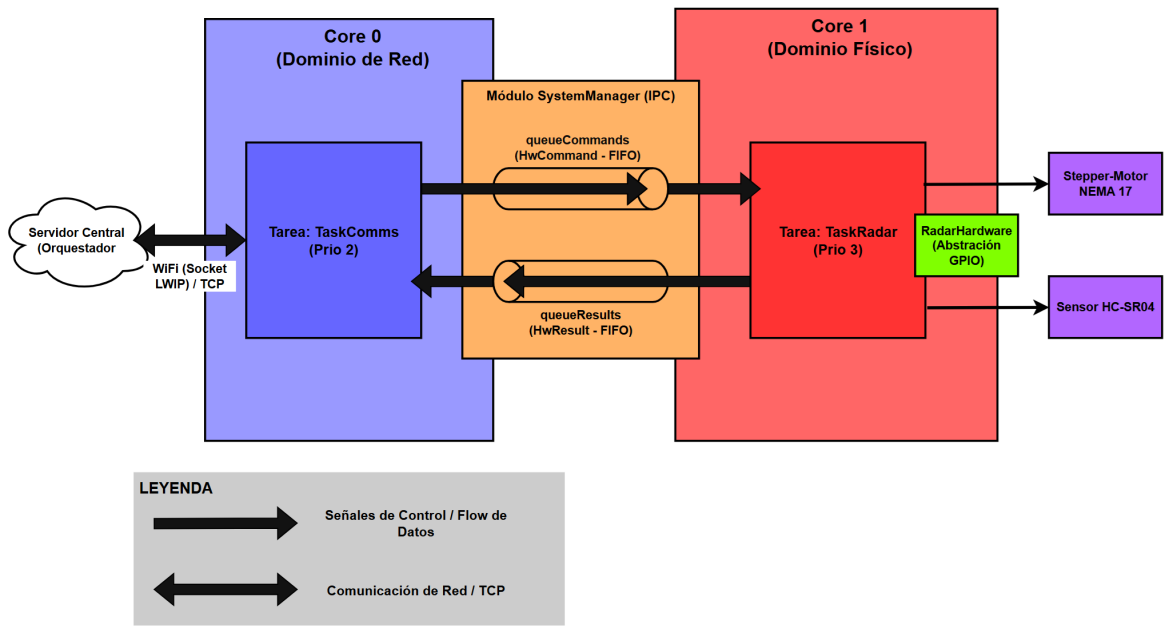


Figura 18: Arquitectura AMP del nodo sensor: distribución de tareas por núcleo y mecanismo de comunicación IPC.

TaskComms opera en el núcleo 0 y se encarga de mantener la conexión TCP, recibir los comandos del servidor, traducirlos en órdenes para el hardware y devolver los resultados. TaskRadar

se ejecuta en el núcleo 1 con mayor prioridad y es la única responsable del control del motor paso a paso y del sensor de distancia, llevando a cabo el ciclo TDMA con precisión temporal. La diferencia de prioridad asegura que FreeRTOS nunca interrumpa la ejecución del hardware debido a una operación de red.

La comunicación entre las dos tareas se realiza exclusivamente a través de las dos colas IPC del módulo **SystemManager**, que se encuentra en el centro de la Figura 18. **queueCommands** transporta mensajes de **TaskComms** hacia **TaskRadar** y funciona como canal de mando. **queueResults** opera en sentido inverso, de **TaskRadar** hacia **TaskComms**, y actúa como canal de telemetría. En todas las lecturas de estas colas se utiliza **portMAX_DELAY**, lo que provoca que cada tarea se bloquee sin consumir CPU hasta que recibe el dato que necesita.

5.3.3 Modelo de Datos

El modelo de datos del firmware establece las estructuras que se intercambian entre las dos tareas y los tipos enumerados que regulan la lógica de la FSM. El diseño se basa en el principio de mínimo acoplamiento: cada estructura contiene únicamente la información necesaria para la tarea receptora, evitando campos adicionales que puedan inflar las colas o generar dependencias innecesarias.

El estado global de la FSM se almacena en el enumerado **SystemState**, que puede adoptar uno de los cuatro valores que corresponden a los estados descritos en la sección 5.3.1: **CONFIGURACION**, **WAITING_FOR_CONNECTION**, **SYNC_CONTROL** y **RADAR**. Este valor es gestionado por **SystemManager**, un objeto *singleton* de acceso global que centraliza el estado compartido del nodo y previene que las tareas accedan directamente a variables globales sin supervisión.

Los comandos que **TaskComms** envía a **TaskRadar** están encapsulados en la estructura **HwCommand**, que consta de tres campos. El campo **type** es de tipo **HwCmdType** e indica la operación a realizar; los valores posibles son **HOME**, que solicita un ciclo de homing para establecer la referencia angular; **SYNC_POS**, que desplaza el motor al ángulo indicado sin realizar mediciones; **MOVE**, que lleva a cabo un desplazamiento relativo; y **EXECUTE_SLOT**, que combina el movimiento al ángulo con la medición ToF. El campo **param** es un número flotante que representa el ángulo objetivo en grados. El campo **execution_time_ms** es un entero sin signo de 32 bits que señala el momento exacto, en milisegundos desde el arranque, en que debe ejecutarse el comando, lo que permite a **TaskRadar** mantener la sincronización de la supertrama.

Los resultados que **TaskRadar** proporciona a **TaskComms** se organizan en la estructura **HwResult**, la cual consta de tres campos: **type**, de tipo **HwResType**, que señala el tipo de resultado; **value**, que representa la distancia medida en centímetros; y **angle**, que indica el ángulo en el que se realizó la medición. La separación entre el ángulo del comando y el ángulo del resultado permite al servidor identificar si el nodo no se posicionó correctamente.

Las dos colas IPC son responsables de transferir la información entre las tareas. **queueCommands** transporta mensajes de **TaskComms** a **TaskRadar** y actúa como canal de control. **queueResults** opera en la dirección opuesta, de **TaskRadar** a **TaskComms**, y funciona como canal de teleme-

tría. Ambas colas tienen un tamaño fijo y se crean durante la fase de CONFIGURACION, antes de que cualquier tarea las utilice.

La configuración hardware de cada nodo físico se almacena en la tabla `NodeProfile`, indexada por la dirección MAC. Cada entrada contiene el identificador numérico del nodo, un valor booleano que indica si es necesario invertir la dirección del motor para compensar la forma en que está instalada la correa, y el nivel lógico de activación del sensor reed. Esta tabla se consulta únicamente una vez durante la CONFIGURACION y sus valores permanecen inalterados a lo largo del ciclo de vida del nodo.

Módulo `comms_protocol.h`

Define exclusivamente los tipos de datos del protocolo TCP: las siete constantes *OpCode* y los ocho *structs* con alineamiento forzado a un byte mediante `#pragma pack(push, 1)`. La directiva de empaquetado garantiza que el *cast* directo de un búfer de red sobre la estructura sea seguro y tenga coste computacional nulo. No contiene lógica ejecutable; actúa como contrato de interoperabilidad entre el firmware y el servidor: cualquier cambio en estas definiciones debe reflejarse simultáneamente en ambos extremos. La Tabla 17 detalla sus definiciones.

Módulo <code>comms_protocol.h</code>		
Define las constantes <i>OpCode</i> del protocolo binario y las estructuras <i>packed</i> de cada mensaje. Actúa como contrato de interoperabilidad entre el firmware del nodo y el servidor central; cualquier cambio en estas definiciones debe reflejarse simultáneamente en ambos extremos.		
Tabla de variables		
Nombre	Descripción	Tipo de dato
<code>MSG_HELLO_REQ</code> ... <code>MSG_DATA_REPORT</code>	Siete constantes <i>OpCode</i> (0xA0–0xD0) que identifican cada tipo de mensaje del protocolo.	<code>#define (uint8_t)</code>
<code>PacketHeader</code>	Cabecera de tres bytes presente en todo mensaje: <code>type</code> (<i>OpCode</i>) y <code>length</code> (longitud del payload en little-endian).	<code>struct (3 B)</code>
<code>Payload_HelloReq</code>	Payload de registro inicial: dirección MAC del nodo en seis octetos consecutivos.	<code>struct (6 B)</code>
<code>Payload_HelloAck</code>	Payload de confirmación de registro: identificador numérico asignado y ángulo guardado de la sesión anterior.	<code>struct (5 B)</code>
<code>Payload_SuperFrame</code>	Payload del pulso de sincronización global: número de secuencia y tiempo del servidor en milisegundos.	<code>struct (8 B)</code>
<code>Payload_AngleReq</code>	Payload de propuesta de ángulo: identificador de nodo, ángulo actual y ángulo propuesto para el siguiente ciclo.	<code>struct (9 B)</code>
<code>Payload_SlotAssign</code>	Payload de asignación de ranura temporal: identificador de nodo, índice de ranura, retardo desde t_0 y ángulo confirmado.	<code>struct (10 B)</code>
<code>Payload_DataReport</code>	Payload de telemetría: identificador de nodo, ángulo de medición, distancia en centímetros y marca temporal.	<code>struct (13 B)</code>

Tabla 17: Descripción del módulo `comms_protocol.h`.

Módulo `config.h`

Centraliza todas las constantes de configuración hardware y red del firmware. Su propósito es agrupar los parámetros ajustables en un único punto de edición, de modo que modificar el

cableado o el comportamiento mecánico no requiera alterar la lógica de las tareas. La Tabla 18 documenta sus constantes.

Módulo <code>config.h</code>		
Repositorio de constantes del nodo. Agrupa pines GPIO, parámetros cinemáticos del motor y parámetros de red en un único archivo, facilitando la adaptación del firmware a variantes hardware sin modificar la lógica de las tareas.		
Tabla de variables		
Nombre	Descripción	Tipo de dato
PIN_MOTOR_STEP / PIN_MOTOR_DIR	GPIOs de control del driver A4988: pin de pulso de paso (GPIO 18) y pin de dirección (GPIO 19).	int (define)
PIN_REED_SWITCH	GPIO del reed switch magnético, empleado como sensor de fin de carrera durante la rutina de homing (GPIO 26).	int (define)
PIN_TRIG / PIN_ECHO	GPIOs del sensor ultrasónico HC-SR04: disparo del pulso (GPIO 14) y recepción del eco (GPIO 27).	int (define)
STEPS_PER_REV	Pasos físicos por revolución del motor NEMA 17: 200 pasos, equivalente a un paso angular físico de 1,8°.r.	int (define)
MICROSTEPPING	Factor de micropasos configurado en el A4988 mediante los pines MS1-MS3 (valor: 16, resolución de 1/16 de paso).	int (define)
HOMING_STEP_US	Período del pulso de paso durante el homing en microsegundos (700 µs), más lento que en operación normal para no saltarse el trigger del reed switch.	int (define)
RADAR_MIN_ANGLE / RADAR_MAX_ANGLE	Límites del arco de barrido en grados: -45° y +45° respectivamente.	float (define)
RADAR_STEP_ANGLE	Incremento angular por ciclo de supertrama: 5° por defecto.	float (define)
CORE_NET / CORE_PHYS	Afinidad de núcleo para cada tarea FreeRTOS: Core 0 para TaskComms y Core 1 para TaskRadar.	int (define)
SERVER_HOSTNAME	Nombre mDNS del servidor ("radar-server"), empleado en la resolución automática de su dirección IP.	string (define)
SERVER_PORT	Puerto TCP de escucha del servidor (8080).	int (define)

Tabla 18: Descripción del módulo `config.h`.

Módulo `SystemManager (system.h)`

`SystemManager` es el repositorio global de estado del nodo, implementado como *singleton* de Meyers para garantizar una única instancia compartida entre `TaskComms` y `TaskRadar`. Las colas FreeRTOS `queueCommands` y `queueResults` constituyen el único canal de comunicación entre las dos tareas, evitando el uso de variables globales accesibles desde ambos contextos. Los atributos `motorDirInvert` y `reedTriggerLevel` almacenan el perfil hardware del nodo, asignado durante la inicialización a partir de la tabla `NodeProfile`. La Tabla 19 documenta sus atributos y métodos.

Módulo SystemManager — system.h			
Singleton que centraliza el estado global del nodo ESP32. Mantiene la máquina de estados de la FSM, las colas de comunicación inter-tarea FreeRTOS y los parámetros de perfil hardware. Toda interacción entre TaskComms y TaskRadar se realiza a través de esta clase, garantizando un punto único de acceso al estado compartido.			
Tabla de variables			
Nombre	Descripción	Tipo de dato	
currentState	Estado activo de la FSM del nodo: CONFIGURACION, WAITING_FOR_CONNECTION, SYNC_CONTROL o RADAR.	SystemState	
queueCommands	Cola FreeRTOS de TaskComms hacia TaskRadar. Transporta estructuras HwCommand con el tipo de operación hardware y sus parámetros.	QueueHandle_t	
queueResults	Cola FreeRTOS de TaskRadar hacia TaskComms. Transporta estructuras HwResult con el resultado de la operación hardware ejecutada.	QueueHandle_t	
radarId	Identificador numérico del nodo asignado por el servidor durante el handshake (valor 1, 2 o 3).	uint8_t	
t0_last_superframe	Marca temporal en milisegundos del instante de recepción del último MSG_SUPERFRAME_START. Referencia para el cálculo del instante absoluto de medición.	uint32_t	
current_angle_logic	Ángulo lógico actual del motor en grados, sincronizado con el servidor en cada ciclo de supertrama.	float	
sweep_direction_up	Dirección activa del barrido: true indica barrido hacia ángulos crecientes, false hacia decrecientes. Se invierte al alcanzar los límites del arco.	bool	
motorActive	Activo desde la recepción de HELLO_ACK hasta que el homing finaliza. Determina si TaskComms debe ejecutar la rutina de homing al detectar una desconexión.	bool	
justHomed	Indica que el motor acaba de completar el homing. Determina si HELLO_ACK ordena un movimiento físico al ángulo guardado o solo una sincronización lógica sin mover el motor.	bool	
motorDirInvert	Perfil hardware: true si el bobinado del motor requiere invertir la señal DIR para girar en el sentido correcto.	bool	
reedTriggerLevel	Perfil hardware: nivel lógico (HIGH/LOW) en que el reed switch activa la interrupción de homing, según sea de tipo NC o NO.	int	
Tabla de métodos			
Nombre	Descripción	Dato(s) de entrada	Dato(s) de salida
instance()	Devuelve la única instancia del singleton (patrón de Meyers). Se crea en el primer acceso y se destruye al final del programa.	Ninguno	SystemManager&
init()	Crea las colas FreeRTOS queueCommands y queueResults e inicializa los parámetros de perfil hardware del nodo.	Ninguno	Ninguno
changeState(newState)	Realiza la transición de la FSM: actualiza currentState y emite un mensaje de diagnóstico por el puerto Serial.	SystemState	Ninguno

Tabla 19: Descripción del módulo `SystemManager` (`system.h`).

Módulo `TaskComms` (`task_comms.cpp/.h`)

`TaskComms` es la tarea FreeRTOS asignada al Core 0 del ESP32. Gestiona la conexión TCP con el servidor, parsea el flujo de bytes entrante en tramas de protocolo, despacha las acciones correspondientes a cada opcode y publica los comandos hardware en `queueCommands`. Se comunica con `TaskRadar` exclusivamente a través de las colas de `SystemManager`. La Tabla 20 describe sus elementos.

Módulo TaskComms — task_comms.cpp			
Tarea FreeRTOS del Core 0. Implementa la máquina de estados de la sesión TCP: descubrimiento del servidor por mDNS, registro mediante <i>handshake</i> HELLO, parseo continuo del flujo de protocolo binario y gestión de la reconexión ante caídas. Actúa como puente entre las órdenes del servidor y la cola de comandos hardware de TaskRadar.			
Tabla de variables			
Nombre	Descripción	Tipo de dato	
client	Socket TCP activo con el servidor. Su estado de conexión determina si se ejecuta la ruta de reconexión en el siguiente ciclo del bucle principal.	WiFiClient	
buffer[128]	Búfer de recepción del payload TCP. Los bytes leídos del socket se copian aquí antes de realizar el <i>cast</i> sobre la estructura de payload correspondiente al opcode recibido.	uint8_t[128]	
last_msg_time	Marca temporal del último mensaje recibido. Si transcurren 5 000 ms sin actividad, el nodo considera el servidor caído y cierra el socket para forzar la reconexión.	uint32_t	
cachedServerIP (static)	IP del servidor almacenada tras la primera conexión exitosa. Permite reconectar directamente sin repetir la resolución mDNS, acelerando la recuperación ante desconexiones transitorias.	IPAddress	
Tabla de métodos			
Nombre	Descripción	Dato(s) de entrada	Dato(s) de salida
TaskComms(pvParameters)	Bucle principal de la tarea. Comprueba el estado del socket, parsea cabeceras de trama, despacha los opcodes del protocolo y lanza el homing si detecta caída del servidor con el motor activo.	void*	Ninguno
connectToServer()	Resuelve el servidor mediante mDNS (<i>radar-server.local</i>); si falla, prueba la IP cacheada o el gateway por defecto. Devuelve true si la conexión TCP se establece con éxito.	Ninguno	bool
sendPacket(type, payload, length)	Construye la trama completa (cabecera de 3 B más payload) en un búfer de salida y la escribe de forma atómica en el socket TCP.	uint8_t, void*, uint16_t	Ninguno
calcularSiguienteAngulo()	Calcula el ángulo del siguiente paso de barrido según la dirección activa y los límites de <i>config.h</i> . Invierte la dirección al alcanzar los extremos del arco.	Ninguno	float

Tabla 20: Descripción del módulo TaskComms (*task_comms.cpp*).

Módulo RadarHardware / TaskRadar (*task_radar.cpp/.h*)

TaskRadar es la tarea FreeRTOS asignada al Core 1. Recibe estructuras *HwCommand* de la cola *queueCommands* y las delega en *RadarHardware*, que encapsula el control del motor paso a paso y la adquisición del sensor ultrasónico. La separación entre la tarea y la clase hardware garantiza que la lógica de control de los periféricos sea completamente independiente del protocolo de comunicaciones. La Tabla 21 describe su estructura.

Módulo RadarHardware / TaskRadar — task_radar.cpp/.h			
Capa de abstracción de hardware (HAL) del nodo. TaskRadar (Core 1) recibe comandos de la cola inter-tarea y los delega en la clase RadarHardware. Esta clase encapsula el control del motor paso a paso A4988/NEMA 17 y la adquisición del sensor ultrasónico HC-SR04, garantizando tiempos de respuesta deterministas.			
Tabla de variables			
Nombre	Descripción	Tipo de dato	
HwCmdType	Enumeración de tipos de comando hardware: HOME (homing con reed), SYNC_POS (sincronización lógica sin movimiento físico), MOVE (movimiento físico a ángulo) y EXECUTE_SLOT (ciclo completo de movimiento y medición).	enum	
HwCommand	Estructura de comando enviada a TaskRadar: type (tipo de operación), param (ángulo objetivo en grados) y execution_time_ms (instante absoluto de ejecución calculado por TaskComms).	struct	
HwResType	Enumeración de tipos de resultado: MOVED (movimiento completado), MEASURED (medición puntual), SLOT_DONE (ciclo completo ejecutado) y HOME_DONE (homing completado).	enum	
HwResult	Estructura de resultado depositada en queueResults: type (tipo de resultado), value (distancia medida en cm) y angle (ángulo en que se realizó la medición).	struct	
currentAngle (privado)	Ángulo actual del motor en grados, mantenido internamente por RadarHardware. Se actualiza en cada paso para permitir movimientos relativos precisos en lazo abierto.	float	
Tabla de métodos			
Nombre	Descripción	Dato(s) de entrada	Dato(s) de salida
TaskRadar(pvParameters)	Bucle principal de la tarea del Core 1. Bloquea en la cola de comandos, despacha cada HwCommand al método correspondiente de RadarHardware y deposita el HwResult en queueResults.	void*	Ninguno
init()	Configura los GPIOs de los periféricos: pines STEP/DIR del driver A4988 como salidas digitales y pines TRIG/ECHO del sensor HC-SR04.	Ninguno	Ninguno
moveToAngle(targetAngle)	Calcula el número de pasos necesarios desde currentAngle hasta targetAngle, emite los pulsos STEP con la dirección correcta y actualiza currentAngle.	float	Ninguno
goHome()	Ejecuta la rutina de homing: gira el motor lentamente hasta que el reed switch se activa, resetea currentAngle a 0° y aplica el desplazamiento de calibración HOMING_TRIGGER_OFFSET.	Ninguno	Ninguno
syncPosition(angle)	Actualiza currentAngle sin emitir pulsos al motor. Se usa cuando el servidor restituye la posición guardada al reconectar, evitando un homing físico innecesario.	float	Ninguno
getDistance()	Dispara el pulso TRIG del HC-SR04, mide el ancho del pulso ECHO y convierte el tiempo de vuelo a distancia en centímetros. Devuelve −1,0 si la lectura queda fuera del rango operativo del sensor.	Ninguno	float
step(direction)	Emite un único pulso de paso al driver A4988: establece DIR según el parámetro, genera el flanco STEP y espera el período mínimo establecido por el factor de microstepping.	bool	Ninguno

Tabla 21: Descripción del módulo RadarHardware/TaskRadar (task_radar.cpp/.h).

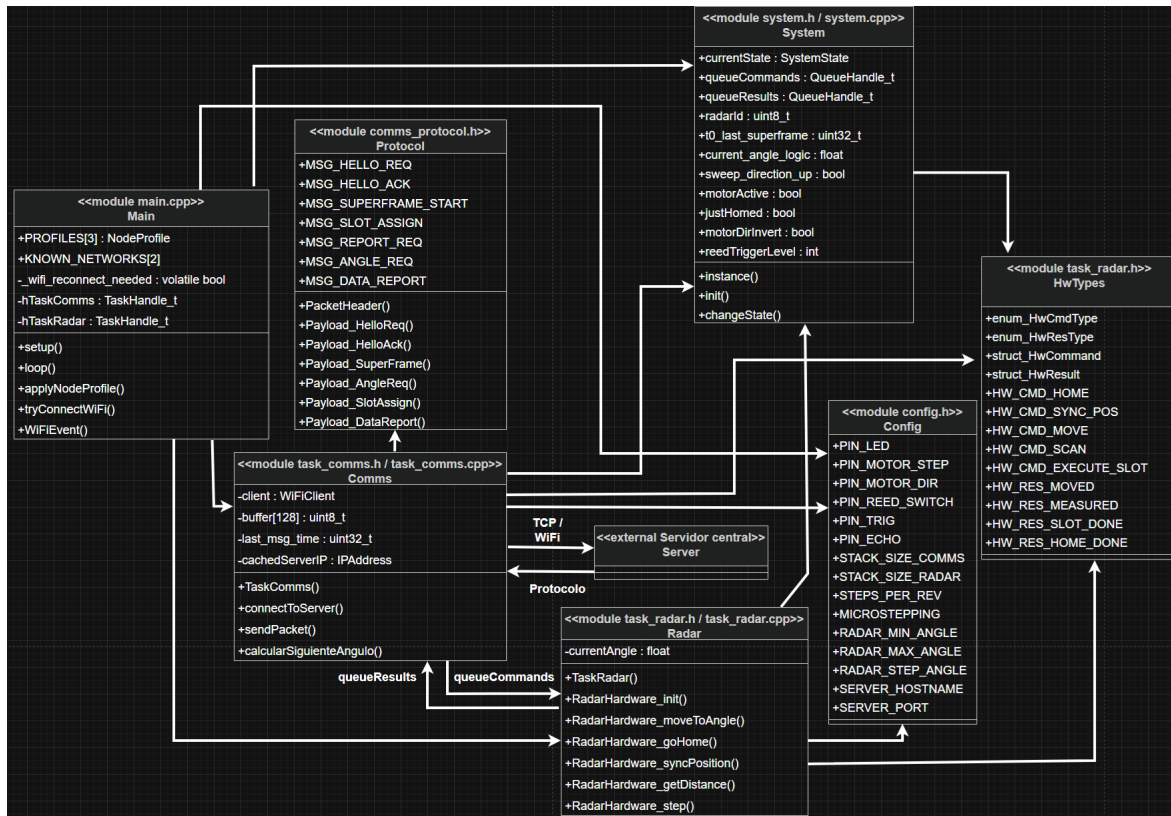


Figura 19: Diagrama de módulos UML del firmware del nodo ESP32.

5.3.4 Control de hardware

El comportamiento del nodo durante el estado RADAR puede describirse a través del diagrama de secuencia de la Figura 20, que ilustra el ciclo completo de una medición desde que el servidor envía el comando hasta que recibe el resultado. Este ciclo consta de ocho fases:

1. La recepción del comando MSG_SLOT_ASSIGN por parte de TaskComms.
2. El despacho del HwCommand hacia TaskRadar a través de queueCommands.
3. El posicionamiento del motor en el ángulo especificado.
4. La espera hasta el instante indicado por execution_time_ms para cumplir con el slot TDMA.
5. El disparo del pulso ultrasónico y la captura del eco.
6. El cálculo de la distancia a partir del tiempo de vuelo.
7. El envío del HwResult de regreso a TaskComms mediante queueResults.
8. El envío del resultado al servidor a través de MSG_DATA_REPORT.

En todas las lecturas de las colas se utiliza portMAX_DELAY, de modo que ninguna tarea con-

sume CPU mientras espera los datos. Los dos apartados siguientes detallan la implementación de los dos periféricos físicos que participan en las fases tres a seis de este ciclo:

Motor paso a paso

El motor es un modelo de tipo paso a paso con 200 pasos por vuelta en resolución completa. El firmware lo gestiona en un modo de 16 micropasos por paso, lo que resulta en 3200 micropasos por vuelta completa. La conversión de un ángulo en grados al número correspondiente de micropasos se realiza mediante la siguiente expresión:

$$p = \left\lfloor \frac{\theta}{360} \times 200 \times 16 \right\rfloor \quad (5.3)$$

lo que se traduce en un mínimo de $2,5^\circ$ por paso de barrido. Cada pulso al driver A4988 tiene una duración de 1 ms, lo que representa un equilibrio entre la velocidad y el par del motor. El rango operativo está restringido a $\pm 45^\circ$ en relación con la posición de referencia, lo cual es suficiente para abarcar el sector asignado al nodo dentro del barrido de la red.

Los cuatro tipos de comandos que `TaskRadar` puede recibir a través de `queueCommands` determinan las acciones del motor. `HOME` inicia el proceso de homing que se detalla más adelante. `SYNC_POS` desplaza el motor al ángulo del campo `param` sin realizar mediciones, utilizado por el servidor para sincronizar la posición del nodo al comienzo de cada supertrama. `MOVE` realiza un desplazamiento relativo desde la posición actual. `EXECUTE_SLOT` combina el movimiento hacia el ángulo indicado con la medición ToF en el momento exacto de `execution_time_ms`; los ángulos que el servidor introduce en este comando varían según el modo de operación activo del nodo, como se describe en la sección 5.3.5.

El proceso de homing establece la referencia angular del sistema. Al recibir un comando `HOME`, el motor se desplaza lentamente hasta que el sensor reed detecta el imán del eje, momento en el cual se detiene y se registra esa posición como origen. Este procedimiento se lleva a cabo siempre al iniciar el estado `RADAR` y también cuando el vigilante TCP detecta que se ha perdido la conexión, garantizando que el hardware siempre comienza desde una posición conocida antes de reanudar las mediciones.

Sensor de distancia

La medición de distancia se basa en el tiempo de vuelo del pulso ultrasónico [7]. Tras emitir el pulso de disparo, el firmware mide la duración del eco recibido y aplica la siguiente expresión para obtener la distancia en centímetros:

$$d = \frac{t_{eco}}{58,0} \quad (5.4)$$

donde t_{eco} se expresa en microsegundos. El valor 58,0 resulta de combinar la velocidad del sonido en el aire a temperatura ambiente con el factor de ida y vuelta del pulso. El tiempo máximo de espera del eco está fijado en 15 ms, lo que corresponde a una distancia máxima de medición de aproximadamente 258 cm; pasado ese tiempo sin eco, la medición se descarta y se notifica como fuera de rango. Adicionalmente, los ecos con duración inferior a 116 μ s se descartan como falsos positivos originados por reflexiones en la estructura mecánica del propio nodo. Ante una medición fallida, el firmware realiza hasta dos intentos antes de reportar el fallo al servidor, reduciendo la tasa de errores espontáneos debidos a condiciones acústicas momentáneamente adversas.

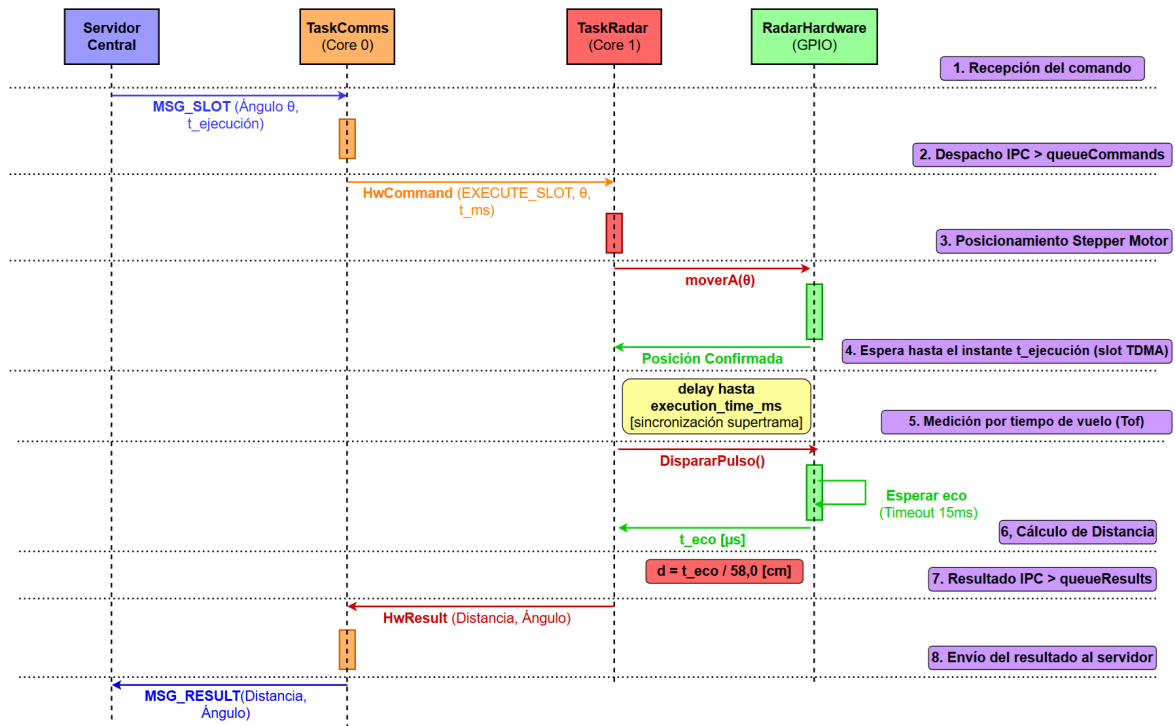


Figura 20: Diagrama de secuencia del ciclo EXECUTE_SLOT: desde la recepción del comando del servidor hasta la entrega del resultado.

5.4 Interfaz gráfica de usuario

5.4.1 Descripción general

La Figura 21 y la Figura 22 ilustran las dos perspectivas que proporciona la aplicación de visualización, conocida como *DEFCON Centro de Mando Táctico*. La aplicación está desarrollada en Python haciendo uso de la biblioteca Tkinter para la interfaz gráfica; recibe el estado del sistema desde el servidor central a través de un socket UDP en el puerto 8081

al final de cada supertrama, lo que le permite actualizar la representación sin interferir en el ciclo de medición. La ventana, con dimensiones fijas, se segmenta horizontalmente en dos áreas permanentes: un lienzo de representación radar que ocupa la parte izquierda y un panel de control lateral ubicado a la derecha. Esta división no cambia entre modos; lo único que se modifica al alternar de uno a otro es el contenido del panel.

La aplicación funciona en dos modos mutuamente excluyentes: el Modo Diseño (Figura 21), que está activo al iniciar y tiene como objetivo configurar el despliegue antes de comenzar la vigilancia, y el Modo Telemetría (Figura 22), que muestra la actividad del sistema en tiempo real una vez que la operación ha comenzado. Todas las configuraciones realizadas en el Modo Diseño se guardan en disco al confirmar el cambio al modo operativo, de tal manera que la sesión se recupera automáticamente en los arranques posteriores.

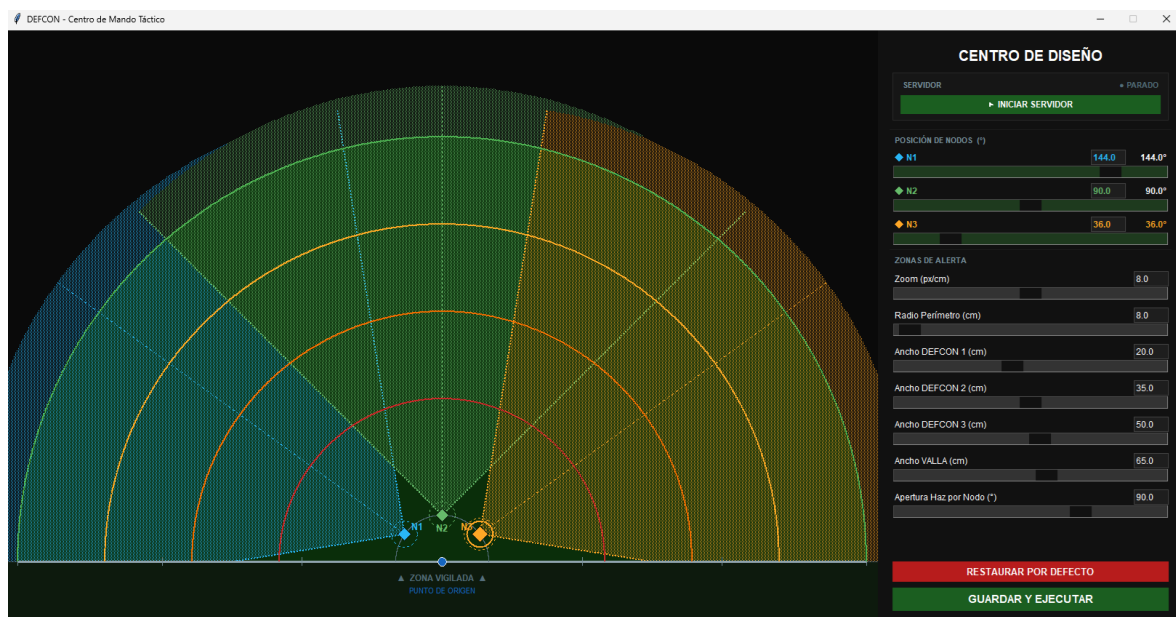


Figura 21: Modo Diseño de la interfaz gráfica de usuario.

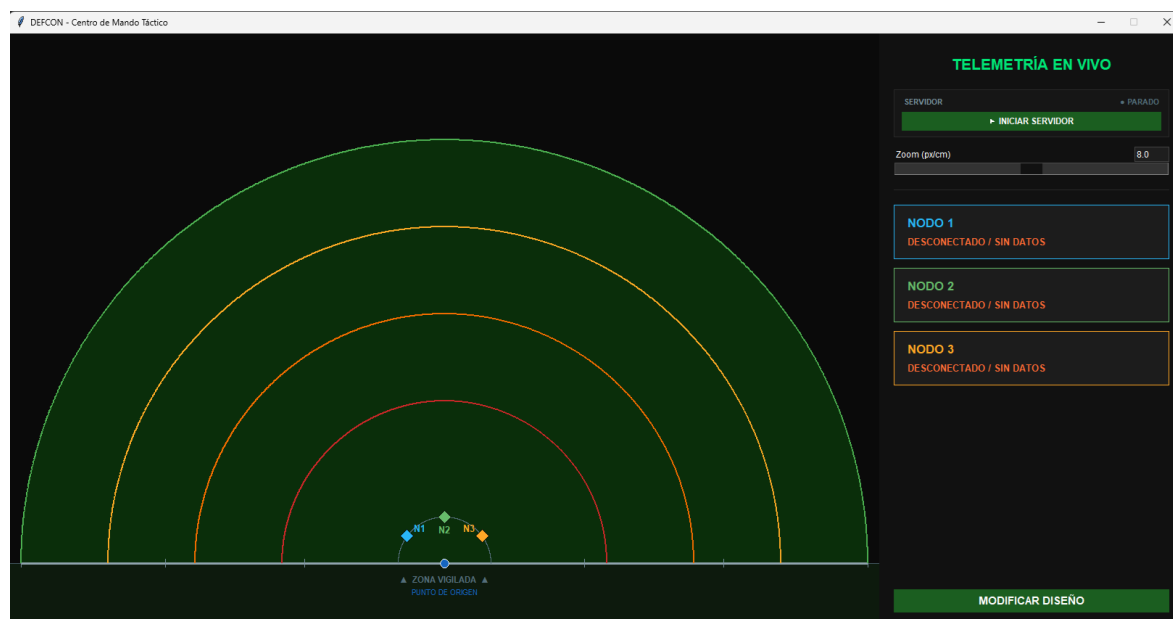


Figura 22: Modo Telemetría de la interfaz gráfica de usuario.

5.4.2 Campo de visualización radar

El lienzo presenta una representación polar semiesférica con un fondo negro y la zona monitoreada coloreada en un verde muy oscuro. La línea horizontal que atraviesa la parte inferior del semicírculo simboliza la barrera física del perímetro defensivo; el punto azul ubicado en su centro es el origen del sistema, desde el cual se proyectan radialmente todos los elementos. Sobre el fondo, se superponen de manera concéntrica cuatro arcos que delimitan las zonas de alerta, de exterior a interior: la VALLA en verde, la DEFCON 3 en amarillo, la DEFCON 2 en naranja y la DEFCON 1 en rojo. El radio de cada arco se determina sumando al radio del perímetro el ancho configurado para esa zona, escalado por el factor de zoom en px/cm; de esta manera, el operador puede ajustar la representación visual a la escala física real de la instalación.

Los tres nodos sensores se ilustran como rombos situados sobre un arco de perímetro interior: N1 en azul, N2 en verde y N3 en naranja. Sus posiciones angulares sobre dicho arco son configurables. La diferencia visual más notable entre los dos modos se observa precisamente en el lienzo: en el Modo Diseño (Figura 21) cada nodo proyecta un abanico de cobertura semitransparente en su tono identificativo, de tal forma que las áreas de solapamiento entre nodos quedan visualmente evidentes sobre la vista de las zonas de alerta; en el Modo Telemetría (Figura 22) los abanicos desaparecen, dejando únicamente los arcos de zona y los rombos de los nodos, para no obstaculizar la interpretación de las detecciones en tiempo real.

5.4.3 Panel de control: Modo Diseño

Tal como se observa en la Figura 23, la interfaz en Modo Diseño se segmenta en dos áreas funcionales: el canvas de visualización, que ocupa la parte izquierda y representa el espacio físico que se está monitorizando, y el panel lateral derecho, que agrupa todos los controles necesarios para la configuración de la sesión. En el canvas se visualizan los tres nodos radar identificados cromáticamente como azul (N1), verde (N2) y naranja (N3) junto con los arcos concéntricos que delimitan las cuatro zonas de alerta. Este modo no recibe ni procesa datos de los dispositivos físicos; su única función es definir la geometría de vigilancia antes de comenzar la sesión de monitorización. El panel lateral se organiza en tres bloques funcionales dispuestos verticalmente: el bloque superior contiene el selector de modo y el botón para iniciar el servidor; el bloque intermedio alberga los controles de posición y ángulo inicial de cada nodo; y el bloque inferior concentra los controles para la configuración de las zonas de alerta y el factor de zoom del canvas.

La Figura 23 muestra el comportamiento del canvas cuando el operador interactúa directamente con uno de los nodos. Al hacer clic en el icono de un nodo, la aplicación resalta su posición mediante un doble anillo concéntrico en el color identificativo del nodo, confirmando visualmente la selección activa. En este estado, el nodo puede ser desplazado libremente por el canvas arrastrando el puntero, de modo que sus coordenadas se actualizan en tiempo real tanto en el panel lateral como en la representación gráfica. Este mecanismo de posicionamiento directo sobre el lienzo constituye la principal vía para definir la geometría de despliegue, ya que permite al operador ajustar la separación y orientación de los nodos sin necesidad de introducir valores numéricos explícitos.

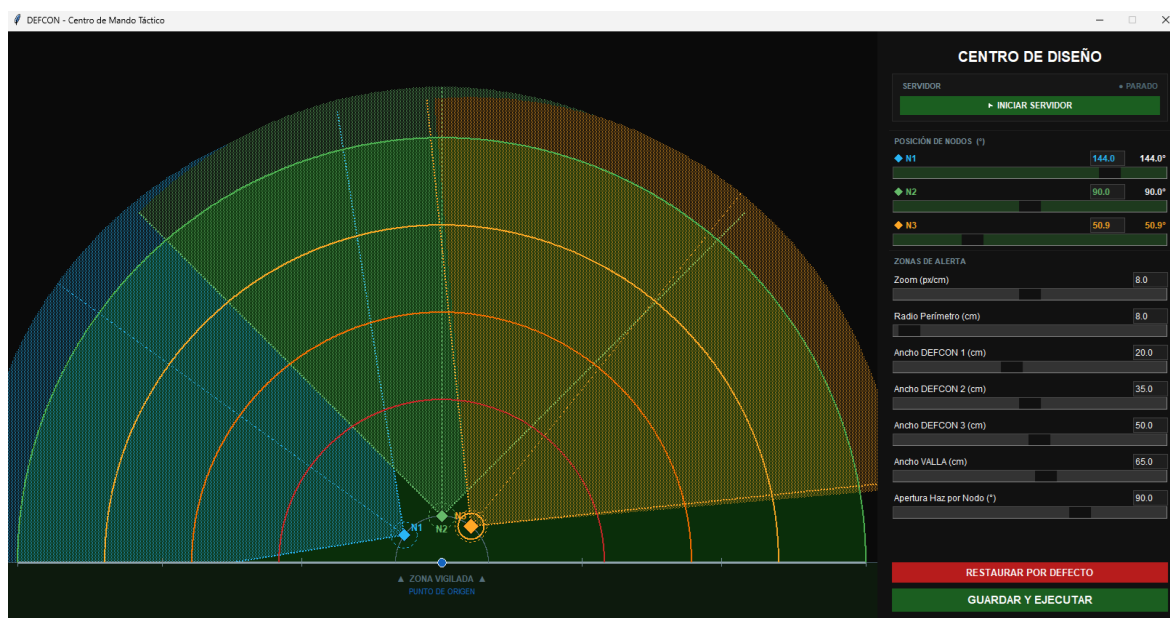


Figura 23: Selección interactiva de nodo en el canvas.

La Figura 24 ilustra el impacto del control de zoom en la representación del lienzo. El factor de escala, ajustable mediante el control deslizante correspondiente en el panel lateral, determina cuántos centímetros del espacio físico monitorizado se representan por cada píxel en el canvas. Con niveles de zoom altos, los arcos de alerta concéntricos se expanden, ocupando una gran parte del lienzo, lo que facilita una inspección detallada de la geometría en entornos de dimensiones reducidas. Con niveles bajos de zoom, el encuadre se amplía, permitiendo la supervisión simultánea de una mayor extensión del perímetro. La escala aplicada no influye en los umbrales de distancia configurados, sino que afecta únicamente a su representación visual; los valores numéricos de cada área permanecen constantes en el panel lateral, sin importar el nivel de zoom elegido.

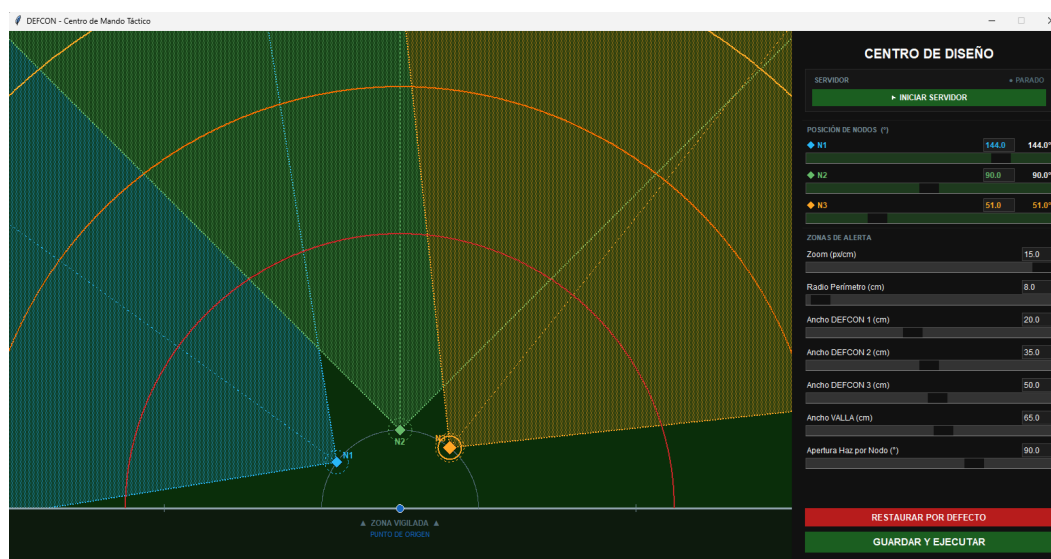


Figura 24: Efecto del zoom sobre la representación de zonas.

La Figura 25 ilustra un detalle del bloque de controles de las zonas de alerta en el panel lateral. Cada zona cuenta con un control deslizante horizontal que muestra el valor numérico actual en centímetros, lo que permite al operador ajustar los umbrales de manera intuitiva sin perder de vista los valores exactos establecidos. Las cuatro zonas siguen una jerarquía estricta de radio creciente: DEFCON 1, DEFCON 2, DEFCON 3 y VALLA, y los colores que las identifican en el panel son los mismos que se utilizan en los arcos del canvas y en las tarjetas de telemetría, asegurando una coherencia visual en toda la interfaz. Cualquier modificación en un umbral se refleja de manera instantánea en los arcos del canvas, permitiendo al operador evaluar visualmente el impacto de cada ajuste antes de confirmar la configuración e iniciar la sesión.

CENTRO DE DISEÑO

SERVIDOR

• PARADO

▶ INICIAR SERVIDOR

POSICIÓN DE NODOS (°)

◆ N1

144.0

144.0°

◆ N2

90.0

90.0°

◆ N3

51.0

51.0°

ZONAS DE ALERTA

Zoom (px/cm)

15.0

Radio Perímetro (cm)

8.0

Ancho DEFCON 1 (cm)

20.0

Ancho DEFCON 2 (cm)

35.0

Ancho DEFCON 3 (cm)

50.0

Ancho VALLA (cm)

65.0

Apertura Haz por Nodo (°)

90.0

RESTAURAR POR DEFECTO

GUARDAR Y EJECUTAR

Figura 25: Panel de configuración de parámetros de la interfaz.

5.4.4 Panel de control: Modo Telemetría

Como se observa en la Figura 22, la disposición espacial de la ventana en Modo Telemetría es equivalente a la del Modo Diseño, con el canvas a la izquierda y el panel lateral a la derecha; sin embargo, el contenido de ambas áreas varía drásticamente para acomodar la información en tiempo real. El canvas deja de ser un lienzo de configuración estático y comienza a recibir las detecciones reportadas por los nodos físicos, representándolas como puntos de color sobre la geometría de zonas previamente definida. El panel lateral elimina los controles de ajuste y, en su lugar, presenta tres tarjetas de estado, una por cada nodo, que concentran toda la información de telemetría relevante para la supervisión operativa.

La Figura 26 ilustra en detalle una tarjeta de nodo durante un estado de seguimiento activo, es decir, cuando el nodo ha detectado un objeto dentro de su área de cobertura y ha cambiado al modo TRACK. En este estado, la tarjeta muestra el ángulo de apuntamiento actual del radar y la distancia medida al último eco registrado, ambos actualizados en cada ciclo de refresco. El borde de la tarjeta adopta el color identificativo del nodo —azul para N1, verde para N2 y naranja para N3—, lo que permite al operador identificar rápidamente qué nodo está en seguimiento sin necesidad de leer el identificador textual. Cuando la distancia medida supera el umbral DEFCON 1, la tarjeta refleja el subestado de borde; si el nodo acumula ciclos consecutivos sin eco, el contador de pérdidas visible en la tarjeta aumenta hasta que se produce la transición de vuelta al modo búsqueda.



Figura 26: Tarjeta de nodo en estado de seguimiento activo.

La Figura 27 muestra el lienzo durante una sesión con detecciones activas. Cada eco que un nodo registra se representa como un punto de color en el lienzo, donde el tono indica la zona

de alerta en la que se realizó la detección: rojo para DEFCON 1, naranja para DEFCON 2, amarillo para DEFCON 3, azul claro para la zona VALLA y gris para ecos fuera del perímetro establecido. A lo largo de la sesión, los puntos se acumulan formando un rastro visual de la trayectoria del objeto detectado. Esta representación permite al operador evaluar de manera inmediata la gravedad de una intrusión y la evolución espacial del objeto sin necesidad de interpretar los valores numéricos de las tarjetas laterales.

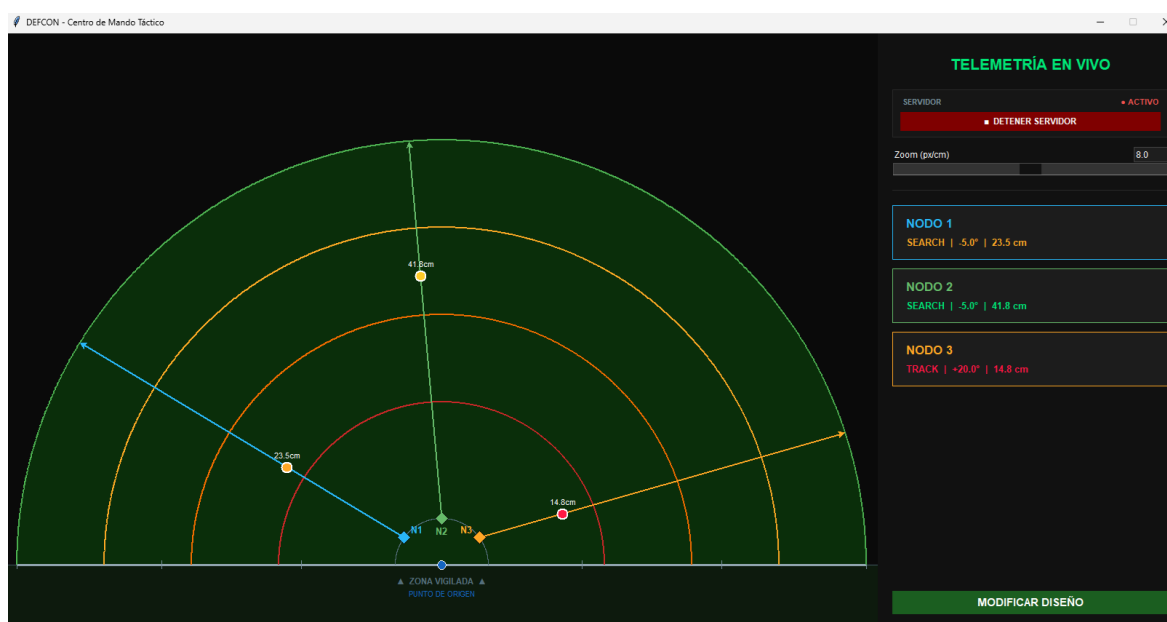


Figura 27: Canvas con detecciones activas por zona de alerta.

La Figura 28 captura el instante en que el sistema genera una estimación de posición a través de localización cooperativa. Cuando dos nodos del par activo —ya sea N1 y N2, o N2 y N3— están simultáneamente en modo TRACK, cada uno calcula la posición cartesiana del objeto basándose en su ángulo de apuntamiento y la distancia medida, y el servidor promedia ambas estimaciones para obtener una posición final. Esta posición se representa en el lienzo mediante un círculo amarillo centrado en las coordenadas calculadas, acompañado de una etiqueta textual que muestra los valores numéricos de abscisa y ordenada en centímetros con respecto al origen del sistema de referencia. El círculo permanece visible mientras el par activo mantiene el seguimiento y desaparece en el momento en que alguno de los nodos pierde el objetivo.

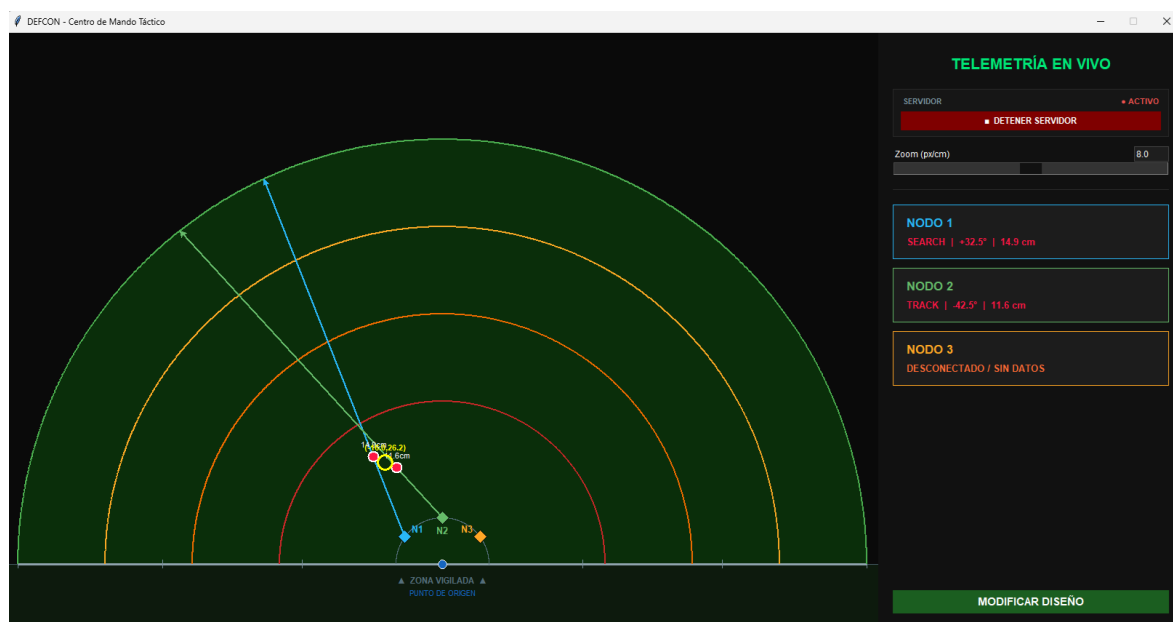


Figura 28: Estimación de posición por localización cooperativa: dos nodos en modo TRACK sobre el mismo objeto en zona de solapamiento.

5.4.5 Reanudación de sesión

La Figura 29 ilustra el diálogo de reanudación de sesión que la aplicación presenta al iniciarse, al detectar un archivo de estado previo en el sistema. Con el fin de mantener la configuración de despliegue entre sesiones sucesivas, la aplicación serializa de manera periódica las posiciones de los nodos, los ángulos de apuntamiento y los umbrales de alerta en un archivo de estado local. Si este archivo está presente en el momento del lanzamiento, la interfaz interrumpe el proceso de inicialización y ofrece al operador un diálogo modal con dos alternativas: reanudar la sesión anterior, recuperando todas las posiciones y parámetros almacenados, o comenzar una nueva sesión con los valores predeterminados. Esta elección es especialmente significativa en situaciones de vigilancia continua, donde el sistema puede haberse reiniciado de manera inesperada —por un corte de energía, por ejemplo— y el operador necesita restaurar la geometría de despliegue sin tener que reconfigurar manualmente cada nodo.

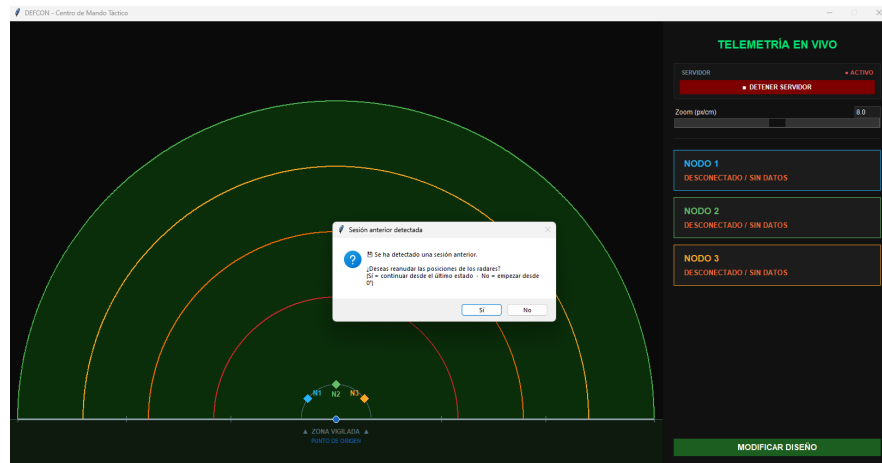


Figura 29: Cuadro de diálogo de reanudación de sesión anterior.

En este capítulo se presentan las pruebas realizadas sobre el sistema de vigilancia distribuido con el fin de verificar que el comportamiento real del prototipo se ajusta a los requisitos funcionales definidos durante la fase de diseño. Las pruebas han sido diseñadas de forma incremental: la primera evalúa el funcionamiento global del sistema bajo condiciones normales de operación, la segunda profundiza en los mecanismos de gestión de conflictos de ranura y localización cooperativa que entran en juego cuando dos nodos apuntan simultáneamente hacia la misma zona, y la tercera verifica la capacidad del sistema para recuperarse ante la desconexión y posterior reconexión de un nodo, restaurando su estado angular sin pérdida de cobertura. Cada prueba sigue una estructura uniforme que incluye la descripción de los componentes implicados, el escenario físico empleado, los requisitos que se pretenden validar, los resultados obtenidos y las conclusiones extraídas.

6.1 Prueba 1: Verificación del Ciclo de Supertrama y Detección de Objetos

La primera prueba tiene como finalidad comprobar el comportamiento global del sistema de vigilancia distribuido en condiciones normales de operación. En esta evaluación se verifica que los tres nodos radar se inician correctamente, establecen comunicación con el servidor central, participan en el protocolo de supertrama y detectan objetos, transmitiendo los resultados al panel de visualización. Esta prueba establece la base para las verificaciones específicas que se llevarán a cabo en las pruebas subsiguientes.

6.1.1 Descripción de los Componentes Utilizados

El servidor central, que se ejecuta en un ordenador personal, implementa el protocolo de supertrama y la lógica de asignación de recursos. Junto a este, opera el panel de visualización *DEFCON Centro de Mando Táctico*, desarrollado en Python con la biblioteca Tkinter, que recibe los datos del servidor a través de un canal UDP y los representa gráficamente en un plano bidimensional. El sistema hardware es el descrito en los capítulos anteriores, con su diseño en impresión 3D, su firmware y sus componentes electrónicos.

La disposición física de los nodos sigue una topología triangular fija. El nodo N1 se localiza en las coordenadas $(-25, 15)$ cm, el nodo N2 en $(0, 28)$ cm y el nodo N3 en $(25, 15)$ cm, con orientaciones base de 144° , 90° y 36° respectivamente en relación al eje horizontal del

sistema de referencia global.

6.1.2 Escenario de Prueba

La prueba se lleva a cabo en un entorno interior controlado, sobre una superficie plana libre de obstáculos no deseados. Los tres nodos se disponen en sus posiciones fijas de acuerdo con la topología triangular previamente descrita y se conectan a la red inalámbrica local que utiliza el servidor. Tanto el servidor como el panel de visualización operan en el mismo equipo anfitrión. La Figura 30 muestra la configuración del despliegue en el panel de visualización antes de iniciar la sesión, con los sectores de cobertura de cada nodo y las zonas de solapamiento entre pares adyacentes visibles sobre el plano.

6.1.3 Funcionamiento del Sistema

Al activarse, cada nodo establece de manera automática una conexión TCP con el servidor central y envía un mensaje de presentación que contiene su identificador de hardware. Al recibir este mensaje, el servidor registra al nodo, le asigna una posición y orientación basándose en su tabla de configuración y le responde con un mensaje de confirmación que puede incluir el último ángulo registrado si el nodo ya había estado en funcionamiento anteriormente.

Una vez que se completa la fase de inicialización, el servidor inicia el ciclo de supertrama de forma periódica. Cada supertrama comienza con un mensaje de inicio que se envía a todos los nodos conectados. Posteriormente, el servidor solicita el ángulo actual de cada nodo; con esta información, calcula la asignación de ranuras temporales para el periodo de reporte y transmite a cada nodo su ranura asignada. Cuando llega el momento correspondiente a cada ranura, el servidor envía una solicitud de reporte al nodo correspondiente, que responde con la distancia y el ángulo medidos en ese momento.

En el modo de búsqueda, cada nodo barre continuamente el sector angular que le corresponde, avanzando un paso fijo por supertrama dentro de los límites de su rango de cobertura. Cuando el sensor detecta un objeto a una distancia inferior al umbral de seguimiento, el nodo cambia al modo de seguimiento, en el que mantiene el sensor orientado hacia el objetivo detectado. El servidor procesa los reportes de todos los nodos activos y envía al visualizador, al final de cada supertrama, un paquete con las posiciones estimadas y los estados actuales, que el panel muestra en tiempo real.

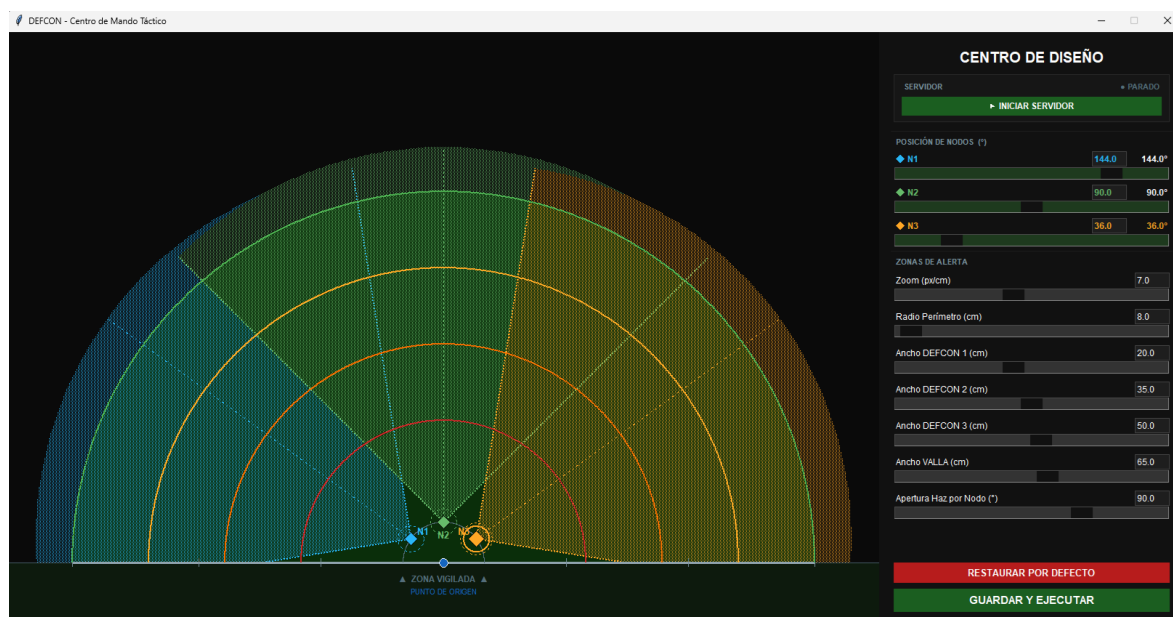


Figura 30: Configuración del despliegue en el Modo Diseño del visualizador: posición de los tres nodos sobre el plano, sectores de cobertura angular y zonas de solapamiento entre pares adyacentes.

Una vez que se ha confirmado que los tres nodos aparecen como conectados en el panel de visualización, se introduce un objeto sólido dentro del área de cobertura del sistema, a una distancia de entre 30 y 50 cm de uno de los nodos, asegurándose de que esté en una posición que no interfiera con las trayectorias de barrido de los otros dos. Se monitorea el comportamiento del sistema durante al menos diez supertramas para verificar la detección, la transición de estado y la representación gráfica.

6.1.4 Requisitos a Cumplir

La Tabla 22 recoge los requisitos funcionales cuya verificación es el objeto de esta prueba.

Código	Descripción
RF01	Los nodos deben establecer conexión automática con el servidor al encenderse, sin intervención manual.
RF03	Los nodos deben ejecutar un barrido angular continuo del sector asignado cuando no se detecta ningún objeto en rango.
RF04	El servidor debe coordinar la comunicación con los nodos mediante un ciclo de supertrama TDMA de periodicidad fija.
RF07	El sistema debe detectar la presencia de un objeto y hacer que el nodo correspondiente adopte el modo de seguimiento, orientando el sensor hacia el objetivo.
RF08	El panel de visualización debe mostrar en tiempo real la posición estimada de los objetos detectados y el estado de cada nodo.

Tabla 22: Requisitos funcionales evaluados en la Prueba 1.

6.1.5 Resultados Obtenidos

Inicialización y conexión (RF01): Los tres nodos completaron la secuencia de arranque y establecieron conexión con el servidor en un tiempo inferior a cinco segundos desde la alimentación eléctrica. El panel de visualización reflejó el estado conectado de cada nodo inmediatamente tras la recepción del mensaje de confirmación del servidor.

Protocolo de supertrama (RF04): El servidor inició el ciclo de supertramas de forma periódica y mantuvo la cadencia durante toda la prueba sin interrupciones. Los tres nodos respondieron a las solicitudes del servidor en el orden y los instantes temporales correspondientes a sus ranuras asignadas. No se registró ninguna colisión de mensajes ni pérdida de respuesta atribuible al protocolo de coordinación.

Barrido en modo de búsqueda (RF03): Antes de la introducción del objeto, los tres nodos realizaron el barrido angular de forma continua, avanzando un paso angular fijo por supertrama entre los límites establecidos. El visualizador mostró la posición angular actualizada de cada nodo en cada ciclo, permitiendo confirmar visualmente que el sector asignado se recorría de forma íntegra y sin saltos, tal como se observa en la Figura 31.

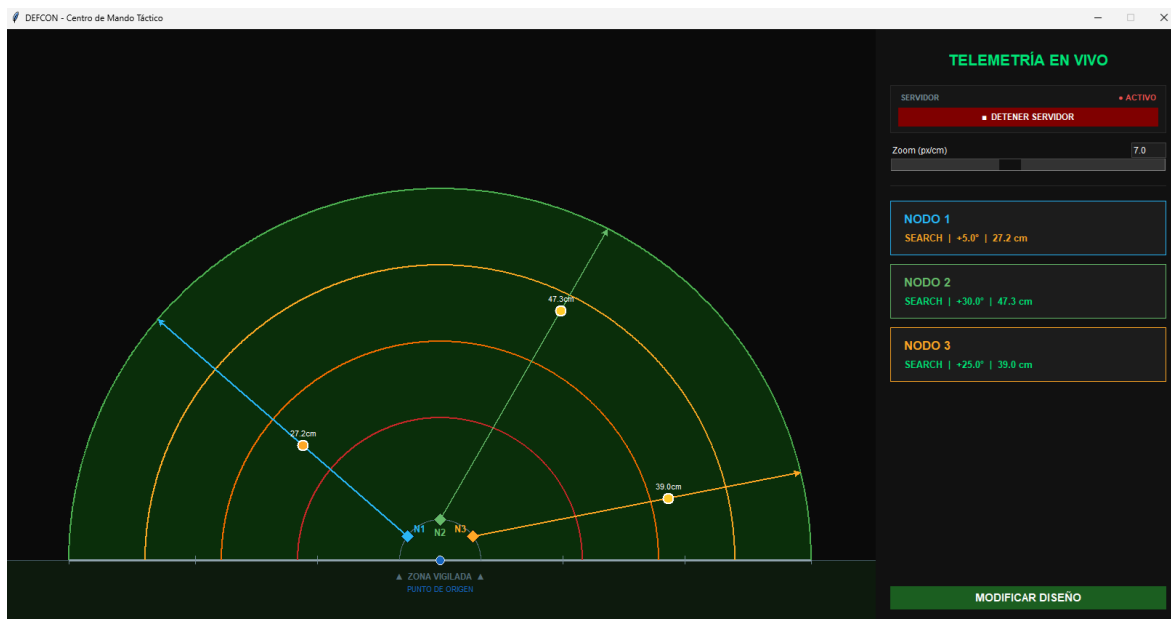


Figura 31: Panel de visualización en Modo Telemetría con los tres nodos activos en modo búsqueda, mostrando el ángulo de apuntamiento actual de cada nodo antes de la introducción del objeto.

Detección y modo de seguimiento (RF07): Al introducir el objeto en el área de cobertura, el nodo más próximo detectó su presencia dentro del umbral de seguimiento tras un máximo de cuatro supertramas, correspondientes a los pasos angulares necesarios para que el barrido alcanzara el ángulo del objetivo. A partir de ese momento el nodo mantuvo el sensor orientado hacia el objeto y continuó reportando distancias estables al servidor sin interrupciones.

Visualización en tiempo real (RF08): El panel de visualización actualizó la representación gráfica en cada supertrama, mostrando el modo de operación de cada nodo —búsqueda o seguimiento— y marcando la posición estimada del objeto sobre el plano. La latencia visual observada fue inferior a un periodo de supertrama, como ilustra la Figura 32.

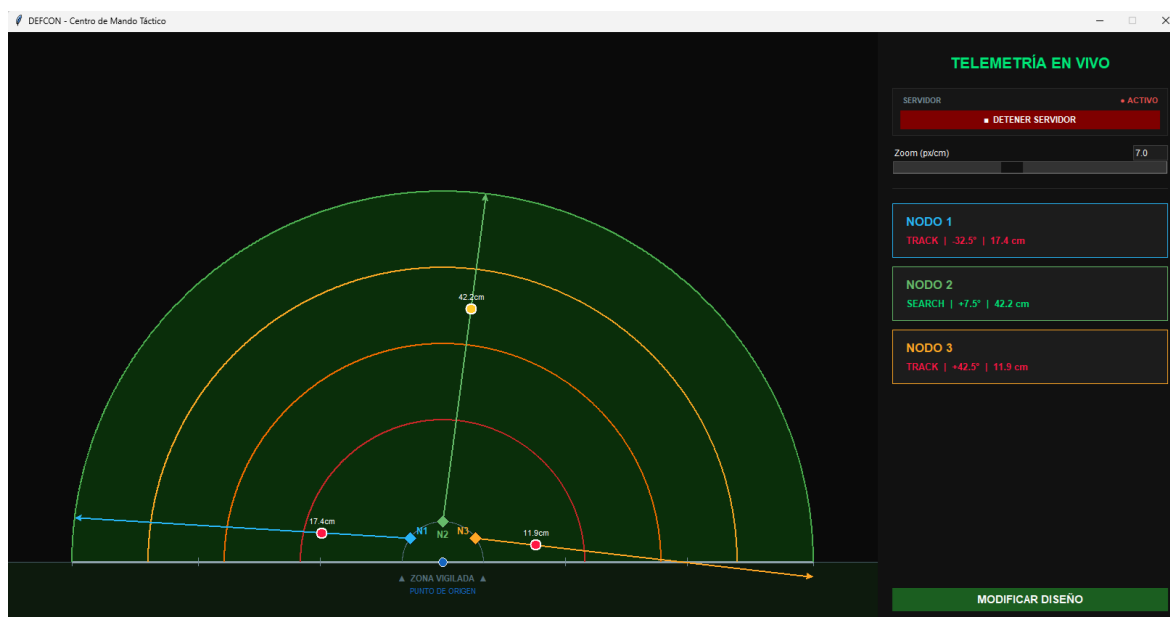


Figura 32: Panel de visualización en Modo Telemetría tras la detección: nodo en modo seguimiento con el marcador de posición del objeto sobre el plano y las lecturas de distancia y ángulo activas en la tarjeta lateral.

6.1.6 Evaluación de Resultados Obtenidos

La evaluación confirma que el sistema funciona de manera integrada y sin errores en condiciones normales de operación. Se cumplieron los cinco requisitos funcionales analizados: los nodos se conectan de manera autónoma al servidor, el protocolo de supertrama gestiona la comunicación sin colisiones, el barrido angular opera adecuadamente en ausencia de detección, la transición al modo de seguimiento ocurre tan pronto como el objeto ingresa al campo de visión del sensor, y el visualizador muestra el estado del sistema con una latencia mínima.

6.2 Prueba 2: Resolución de Conflictos de Ranura y Localización Cooperativa

La segunda prueba tiene como finalidad comprobar los mecanismos que el sistema activa cuando dos nodos del mismo par de conflicto apuntan simultáneamente hacia la zona de interferencia angular. En tal circunstancia, el servidor debe resolver el conflicto de ranura asignando momentos de reporte diferentes a cada nodo y, al mismo tiempo, calcular una estimación cooperativa de la posición del objeto a partir de las mediciones de ambos. Dado que la condición geométrica que permite la trilateración y la que provoca el conflicto de ranura son idénticas —ambos nodos del par deben tener el objeto dentro de su zona sucia—, esta prueba valida ambos mecanismos de manera conjunta.

6.2.1 Descripción de los Componentes Utilizados

Los componentes empleados son los mismos tres nodos radar descritos en la Prueba 1, junto con el servidor central y el panel de visualización *DEFCON Centro de Mando Táctico*. En esta prueba cobran especial relevancia la lógica de asignación de ranuras del servidor, que opera sobre la información de ángulo reportada por cada nodo en cada supertrama, y el módulo de trilateración, que estima la posición del objeto combinando las medidas de distancia y ángulo de los dos nodos del par.

El sistema define dos pares de nodos con riesgo de interferencia acústica: el formado por N1 y N2, y el formado por N2 y N3. La interferencia es posible únicamente cuando ambos nodos de un par dirigen su sensor hacia la zona sucia, entendida como los ángulos locales que superan en valor absoluto el límite de zona limpia establecido en el diseño. Fuera de esa zona, la propagación de las ondas ultrasónicas no produce solapamientos relevantes entre nodos del mismo par.

6.2.2 Funcionamiento del Sistema

Cuando al inicio de cada supertrama el servidor recibe los ángulos actuales de los nodos, verifica si algún par de nodos en conflicto se encuentra simultáneamente en zona sucia. Si esta condición se cumple y ambos nodos tienen activa una detección, el servidor determina al ganador del conflicto —el que entró antes en modo de seguimiento, registrado en el campo `track_entry_sf`— y le asigna la ranura prioritaria. Al nodo perdedor se le otorga la ranura secundaria, separada en el tiempo de la primera por una cantidad fija, y se le obliga a regresar al modo de búsqueda, aunque mantiene el registro de su primera detección para no perder su posición en futuros desempates. El nodo ganador recibe además un periodo de inmunidad durante el cual ningún nuevo conflicto puede desplazarlo.

Cabe señalar que, dado que N2 pertenece a dos pares de conflicto (N1-N2 y N2-N3), podría plantearse la situación de que existan objetos en ambas zonas sucias de forma simultánea.

En la práctica, esto no genera conflictos solapados: cada nodo dispone de un único sensor y solo puede apuntar en una dirección a la vez, por lo que un nodo en modo de seguimiento permanece orientado hacia el primer objeto detectado y no entra en conflicto por su otra zona sucia. El servidor solo evalúa el conflicto del par cuyo nodo esté apuntando activamente a la zona compartida.

Simultáneamente, siempre que ambos nodos del par se encuentren en zona sucia, el servidor calcula en cada supertrama una estimación cooperativa de la posición del objeto combinando las medidas de distancia y ángulo de ambos nodos, y transmite el resultado al visualizador diferenciado de las detecciones individuales.

6.2.3 Escenario de Prueba

La prueba se lleva a cabo en el mismo montaje físico que la Prueba 1. Para activar el par de conflicto N1-N2, se coloca un objeto en la región del plano que se encuentra dentro del sector de zona sucia de N1 y, al mismo tiempo, dentro del sector de zona sucia de N2. Dado que N1 tiene su orientación base a 144° y N2 a 90° , esta región se sitúa aproximadamente en la parte superior izquierda del triángulo formado por los nodos. La Figura 33 muestra las zonas sucias de ambos pares de conflicto sobre el Modo Diseño del visualizador.

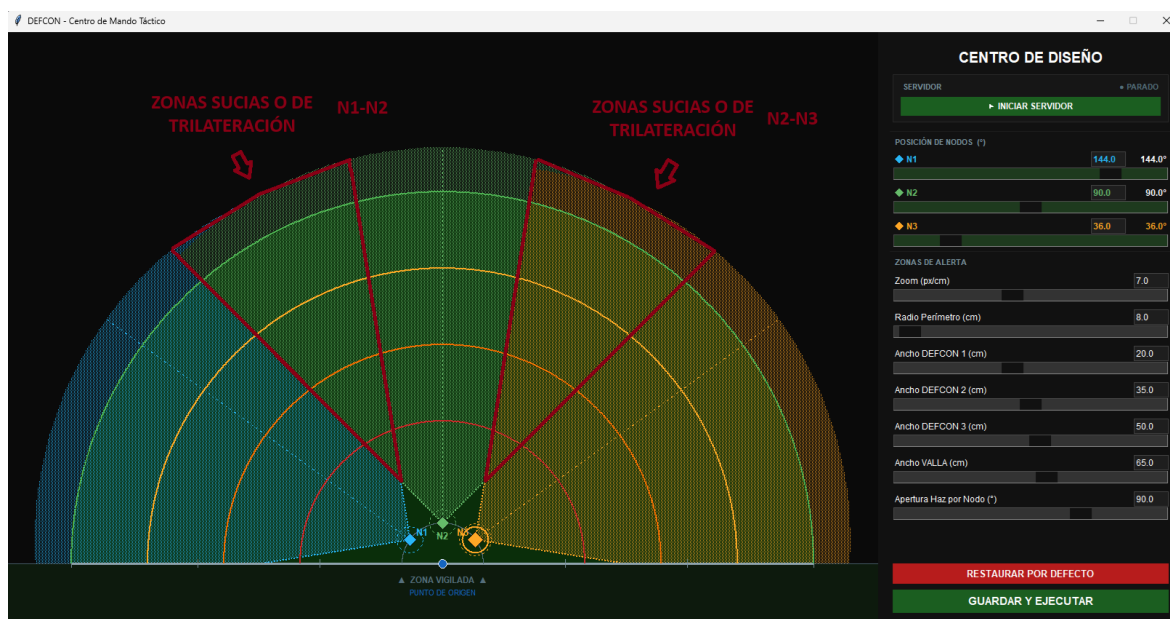


Figura 33: Modo Diseño del visualizador con las zonas sucias de los pares N1-N2 y N2-N3 señaladas.

El objeto se introduce con los tres nodos en modo de búsqueda y se mantiene en esa posición durante un mínimo de quince supertramas. Durante este periodo se observa la transición de ambos nodos al modo de seguimiento, la resolución del conflicto de ranura y la aparición del

marcador de posición cooperativa en el panel de visualización.

6.2.4 Requisitos a Cumplir

La Tabla 23 recoge los requisitos funcionales cuya verificación es el objeto de esta prueba.

Código	Descripción
RF04	El servidor debe detectar situaciones de conflicto entre nodos del mismo par y asignarles ranuras temporales distintas para evitar interferencias acústicas.
RF06	Cuando dos nodos del mismo par detectan un objeto en la zona de interferencia, el sistema debe calcular una estimación cooperativa de la posición del objeto mediante trilateración y mostrarla en el panel de visualización.

Tabla 23: Requisitos funcionales evaluados en la Prueba 2.

6.2.5 Resultados Obtenidos

Detección del conflicto, asignación de ranuras y localización cooperativa (RF04, RF06): La Figura 34 muestra el registro del servidor durante tres supertramas consecutivas (SF 62 a SF 64) con el par N1-N2 activo en zona sucia. Cada supertrama incluye tres bloques de información. En primer lugar, la línea con prefijo [TRILAT] indica las coordenadas cartesianas de la posición cooperativa calculada en ese ciclo, que se mantienen estables en torno a $(-16,3, 30,5)$ cm a lo largo de los tres superframes. En segundo lugar, la línea con prefijo [CONF] registra la decisión de arbitraje: en SF 62 el servidor determina que N2 es el ganador porque fue el primero en entrar en modo de seguimiento (en SF 51), y le otorga un periodo de inmunidad que se refleja en las supertramas siguientes como **N2 bloqueo hasta SF68** y **N2 bloqueo hasta SF69**. En tercer lugar, la tabla de asignación de ranuras muestra el estado de cada nodo: N1 en modo búsqueda con ranura 0 y N2 en modo seguimiento con ranura 1, con sus respectivos ángulos de apuntamiento, nivel DEFCON y distancia medida.

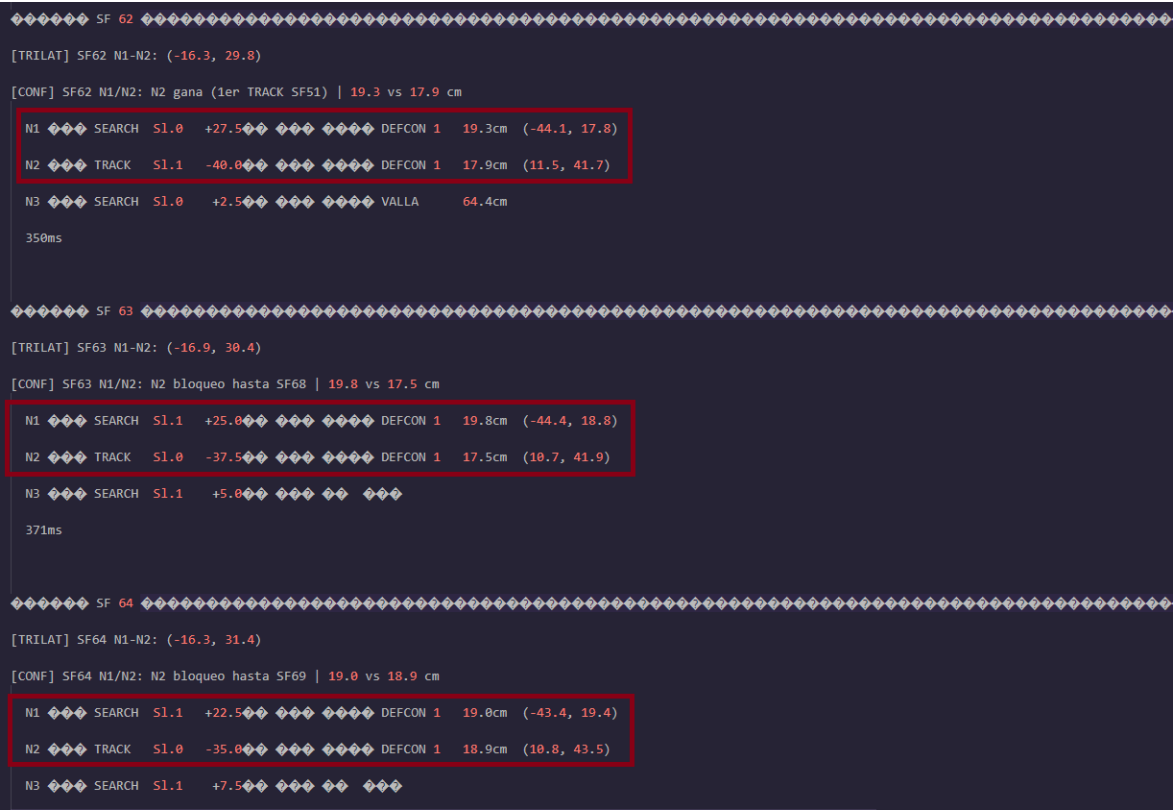


Figura 34: Registro del servidor durante las supertramas SF62 a SF64 con el par N1-N2 activo en zona sucia.

Representación visual de la localización cooperativa (RF06): La Figura 35 muestra el estado del panel de visualización durante la prueba. En el canvas se distinguen dos puntos rojos correspondientes a las detecciones individuales de N1 y N2, y un círculo amarillo con las coordenadas de la posición cooperativa estimada, coherente con los valores calculados en el log. Las tarjetas laterales confirman el estado de cada nodo: N1 en modo búsqueda y N2 en modo seguimiento, con ambos reportando distancias en zona DEFCON 1.

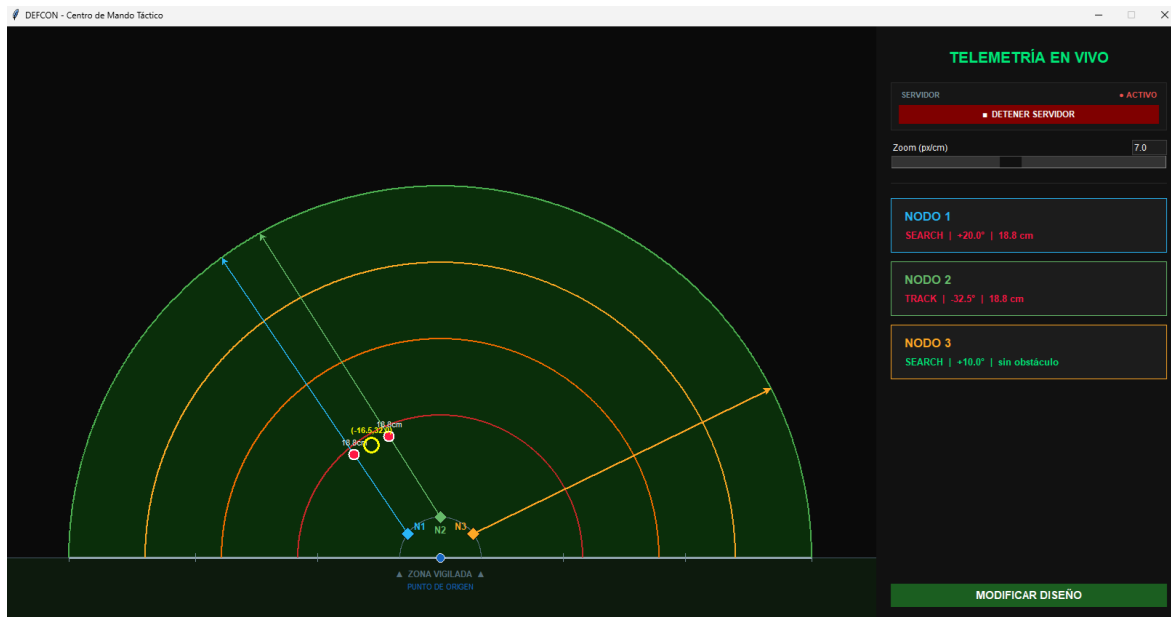


Figura 35: Panel de visualización en Modo Telemetría con localización cooperativa activa: detecciones individuales de N1 y N2 (puntos rojos) y posición combinada estimada (círculo amarillo).

6.2.6 Evaluación de Resultados Obtenidos

La prueba confirma que el sistema maneja adecuadamente las situaciones de interferencia acústica entre nodos adyacentes. La resolución del conflicto de ranura ocurre de manera automática en la supertrama que sigue inmediatamente a la detección simultánea, el ganador se determina por criterio objetivo de primera detección y recibe inmunidad durante el número de supertramas establecido, y la localización cooperativa genera coordenadas estables y coherentes con la posición real del objeto actualizadas en cada ciclo. Ambos requisitos evaluados son cumplidos.

6.3 Prueba 3: Detección de Desconexión y Recuperación de Estado Angular

La tercera prueba tiene como finalidad comprobar la solidez del sistema frente a la pérdida de conexión de uno de los nodos y su posterior reconexión. Se verifica que el servidor identifica la desconexión de manera autónoma tras acumular un número específico de respuestas perdidas de forma consecutiva, excluye al nodo del ciclo de supertrama activo y, cuando el nodo se reconecta, le restaura el último ángulo registrado para que pueda retomar la cobertura de su sector sin necesidad de reiniciar el barrido desde el principio.

6.3.1 Descripción de los Componentes Utilizados

Los componentes utilizados son los mismos tres nodos radar, el servidor central y el panel de visualización mencionados en pruebas anteriores. En esta prueba, el elemento central es el mecanismo de supervisión de conectividad del servidor, que mantiene un contador de fallos de respuesta por nodo y actúa sobre él cuando se supera el umbral establecido. Complementariamente, interviene el módulo de persistencia de estado, que almacena periódicamente en disco el ángulo de apuntamiento de cada nodo activo, y el protocolo de presentación, mediante el cual un nodo que se conecta al servidor solicita y recibe el último estado guardado antes de integrarse al ciclo de supertrama.

6.3.2 Funcionamiento del Sistema

Durante la operación normal, el servidor espera la respuesta de cada nodo en el momento correspondiente a su ranura dentro de la supertrama. Si un nodo no responde, el servidor incrementa su contador de fallos consecutivos. Cuando ese contador alcanza el umbral máximo de siete fallos permitidos, el servidor considera al nodo desconectado, lo elimina de la lista de participantes activos y deja de incluirlo en las supertramas siguientes. Este proceso se lleva a cabo de manera completamente automática y sin interrumpir el ciclo de los nodos restantes.

Cuando el nodo se reinicia y establece una nueva conexión TCP con el servidor, envía un mensaje de presentación. El servidor, al identificar el identificador del nodo, busca en su tabla de estado el último ángulo registrado y lo comunica en el mensaje de respuesta. El nodo adopta ese ángulo como su punto de partida y retoma el barrido desde esa posición, sin realizar un homing completo desde el extremo del sector. Durante un periodo de gracia posterior a la reconexión, el servidor no contabiliza como fallos las respuestas tardías del nodo recién incorporado.

Es importante destacar que los nodos no disponen de autonomía operativa si el servidor cae: la asignación de ranuras temporales y la coordinación del acceso al medio acústico dependen íntegramente del servidor central. Cuando un nodo detecta la pérdida de la conexión TCP —tras el umbral de inactividad de 5 000 ms sin recibir mensajes—, detiene sus mediciones,

ejecuta el procedimiento de *homing* para restablecer la referencia angular y entra en un bucle de reconexión hasta que el servidor vuelva a estar disponible. Esta prueba cubre únicamente la caída de un nodo individual; la resiliencia ante un reinicio del servidor queda garantizada por el mecanismo de persistencia de estado descrito anteriormente.

6.3.3 Escenario de Prueba

La prueba se lleva a cabo con los tres nodos funcionando de manera normal y sin objetos en el área de cobertura, con el fin de aislar el comportamiento de reconexión de cualquier interacción con la lógica de detección. La Figura 36 muestra el estado del sistema antes de la desconexión, con los tres nodos activos y N1 en modo seguimiento sobre un objeto en la zona DEFCON 1.

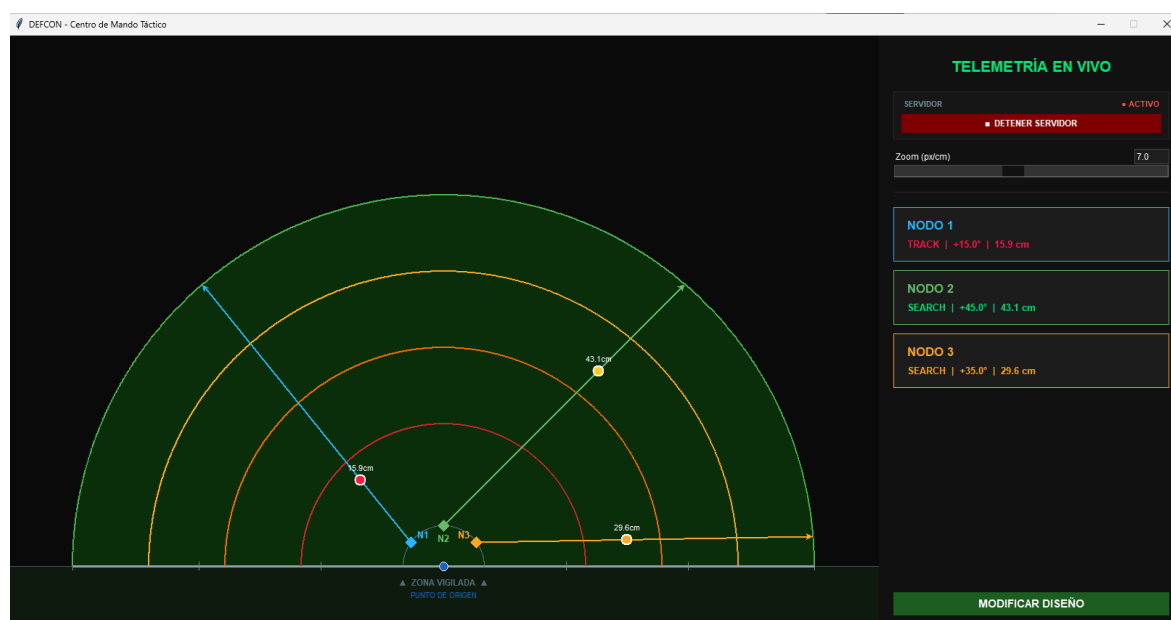


Figura 36: Panel de visualización con los tres nodos activos antes de la desconexión de N2.

Una vez alcanzadas al menos veinte supertramas estables, se interrumpe la alimentación del nodo N2, simulando una desconexión no planificada. Posteriormente se restablece la alimentación y se verifica la reincorporación automática al ciclo.

6.3.4 Requisitos a Cumplir

La Tabla 24 recoge los requisitos funcionales cuya verificación es el objeto de esta prueba.

Código	Descripción
RF02	Cuando un nodo se reconecta al servidor, el sistema debe restaurar su último ángulo de apuntamiento registrado para que retome la cobertura de su sector sin reiniciar el barrido desde el inicio.
RF05	El servidor debe detectar la pérdida de conexión de un nodo de forma autónoma tras un número determinado de respuestas perdidas consecutivas y eliminarlo del ciclo de supertrama activo.

Tabla 24: Requisitos funcionales evaluados en la Prueba 3.

6.3.5 Resultados Obtenidos

Detección de la desconexión (RF05): Tras interrumpir la alimentación de N2, el servidor comenzó a registrar los fallos de respuesta en cada supertrama. La Figura 37 muestra, de forma compuesta, dos zonas del registro del servidor: en la parte izquierda, las supertramas SF 150 y SF 151, donde se acumulan los strikes 6/7 y 7/7 respectivamente y se emite el mensaje [KICK #1] Nodo 2 atascado en inicio. Forzando cierre, seguido del indicador **sin reconectar**: N[2] que acompaña a cada supertrama durante la ausencia del nodo; en la parte derecha, las supertramas SF 257 a SF 260, que muestran el ciclo operando con N1 y N3 únicamente, y la línea de reconexión en SF 260: [SYS] Conectado MAC F0:24:F9:44:0A:20 -> ID 2 (Reanudando en -27.5°) [reconexión tras KICK #1].

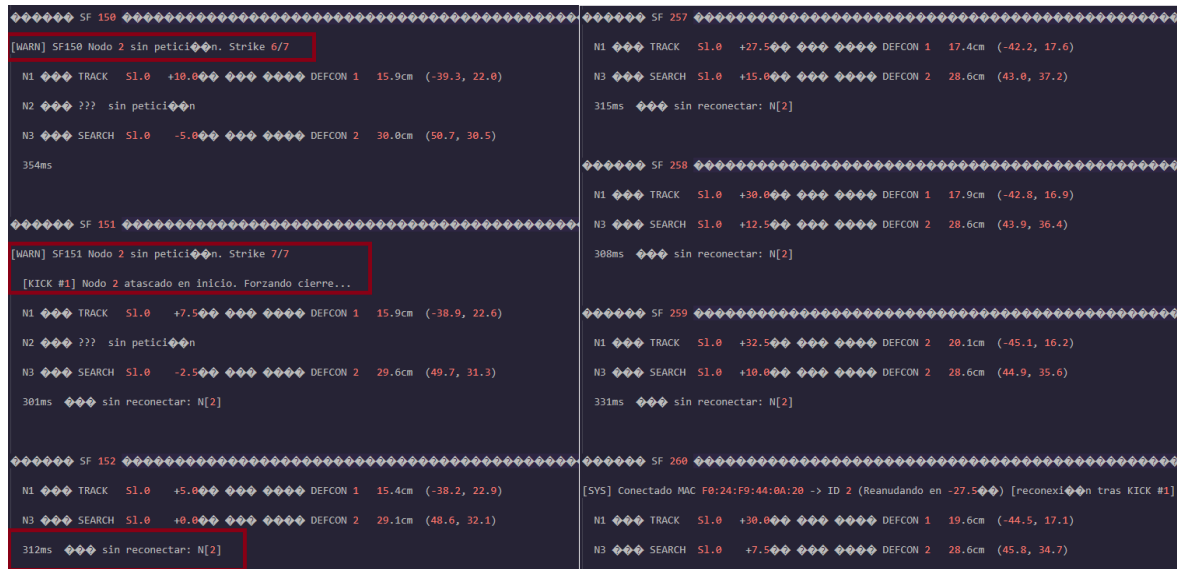


Figura 37: Registro del servidor: desconexión de N2 con strikes 6/7 y 7/7 más mensaje de expulsión (izquierda) y reconexión en SF 260 con ángulo restaurado a $-27,5^\circ$ (derecha).

La Figura 38 confirma visualmente el estado del sistema durante la ausencia de N2: la tar-

jeta del nodo muestra *DESCONECTADO / SIN DATOS*, mientras que N1 y N3 continúan operando con normalidad sin ninguna interrupción en sus ciclos de barrido.

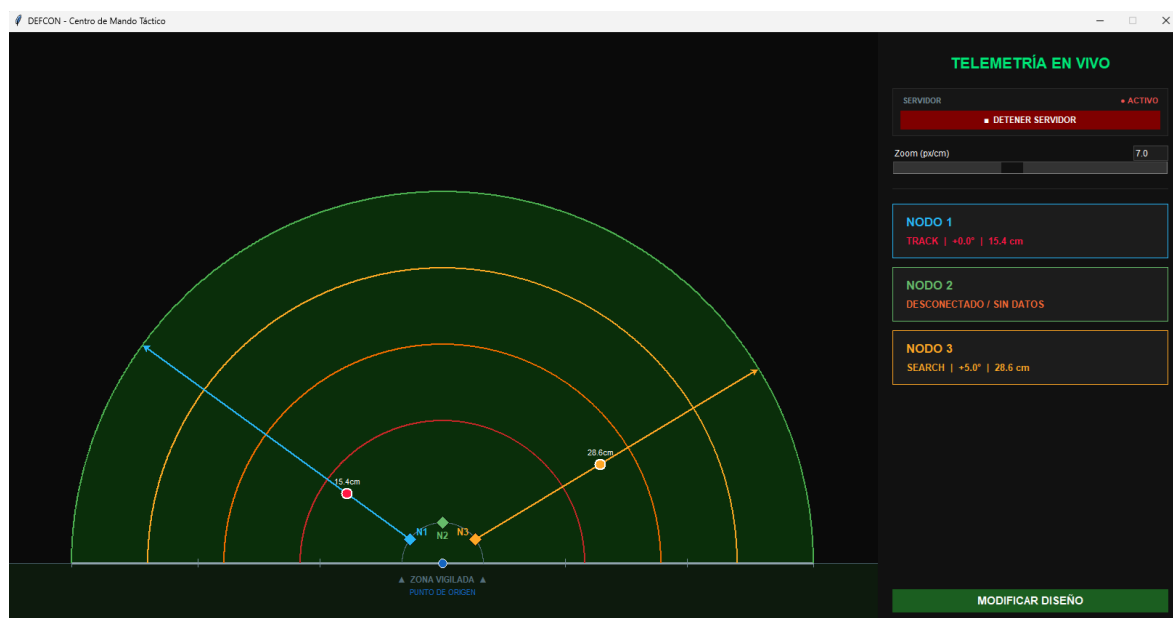


Figura 38: Panel de visualización con N2 marcado como desconectado y N1 y N3 operando sin interrupción.

Recuperación del ángulo (RF02): Como acredita la línea de reconexión del registro, el servidor comunicó a N2 el ángulo $-27,5^\circ$ en el mensaje de presentación, correspondiente a la última posición registrada antes de la desconexión. N2 adoptó ese ángulo como punto de partida sin necesidad de realizar un homing completo desde el extremo del sector. La Figura 39 muestra el sistema completamente restablecido: los tres nodos aparecen activos en el panel, con N2 retomando el barrido desde el ángulo restaurado y avanzando progresivamente por su sector.

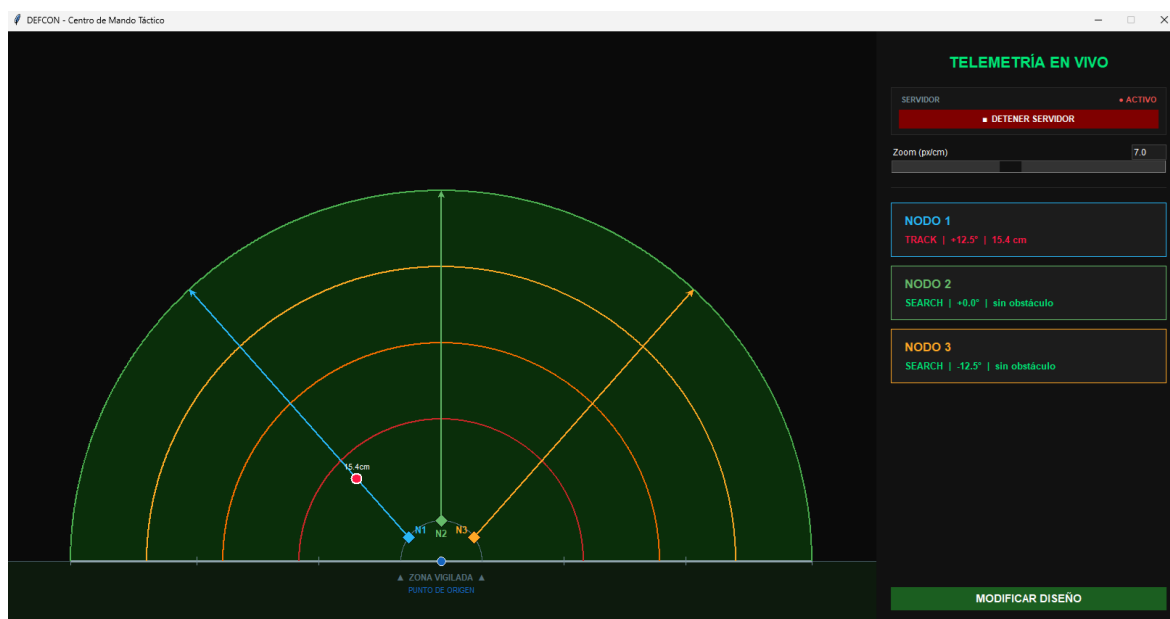


Figura 39: Panel de visualización con los tres nodos activos tras la reconexión de N2, que retoma el barrido desde el ángulo restaurado.

6.3.6 Evaluación de Resultados Obtenidos

La evaluación confirma que el sistema es capaz de tolerar la desconexión inesperada de un nodo sin perjudicar el funcionamiento de los otros, y puede reintegrarlo de manera autónoma con su estado angular restaurado. La identificación de la desconexión ocurre dentro de un número limitado y predecible de supertramas, la restauración del ángulo elimina la necesidad de un barrido completo de inicialización, y el periodo de gracia asegura una reincorporación estable. Ambos requisitos evaluados son cumplidos.

CAPÍTULO 7 | Conclusiones

Tras finalizar el diseño, la implementación y la validación experimental del sistema, se presentan las conclusiones obtenidas, evaluando el grado de cumplimiento de los objetivos iniciales y analizando el impacto de las decisiones técnicas adoptadas.

7.1 Conclusiones Generales

El presente Trabajo de Fin de Grado ha logrado diseñar, implementar y validar con éxito un sistema de vigilancia distribuida compuesto por una red de tres nodos radar cooperativos sobre plataformas embebidas de bajo coste. El prototipo resultante es capaz de cubrir de forma continua un área perimetral interior, coordinar el acceso al medio acústico para evitar interferencias entre nodos adyacentes y estimar cooperativamente la posición de los objetos detectados mediante trilateración.

La principal aportación del proyecto ha sido demostrar que es posible resolver el problema del *crosstalk* acústico en redes de sensores distribuidos de bajo coste mediante una solución íntegramente software, sin requerir hardware analógico especializado ni infraestructura de posicionamiento externa. La implementación de un protocolo TDMA dinámico centralizado [22, 18] ha permitido que los tres nodos operen de forma simultánea sin interferirse, y la arquitectura de firmware basada en FreeRTOS [23] sobre el ESP32 de doble núcleo ha garantizado la separación determinista entre las tareas de red y las de control físico. Ambas decisiones de diseño, adoptadas durante las fases tempranas del proyecto, han resultado ser las que mayor impacto han tenido sobre la fiabilidad y la robustez del sistema final.

7.2 Grado de Cumplimiento de los Objetivos

El objetivo principal de este Trabajo de Fin de Grado era diseñar, desarrollar y validar un sistema de vigilancia distribuido formado por una red de nodos radar cooperativos sobre plataformas embebidas de bajo coste, capaz de cubrir un área perimetral de forma continua, coordinar el acceso al medio acústico para evitar interferencias entre nodos y estimar la posición de los objetos detectados de forma conjunta. Para evaluar el éxito del proyecto de manera objetiva, se procede a verificar el cumplimiento de los requisitos formales establecidos en el capítulo de Análisis del Problema.

7.2.1 Validación de Requisitos Funcionales

La Tabla 25 detalla el grado de cobertura de las funcionalidades operativas del sistema.

Cód.	Descripción resumida	Estado	Validación / Observaciones
RF01	Arquitectura distribuida con gestión autónoma de barridos y eliminación de interferencias.	Cumplido	El servidor coordina el ciclo TDMA de forma autónoma sin intervención manual. Verificado en Prueba 1.
RF02	Restauración del ángulo de apuntamiento al reconectar.	Cumplido	El servidor persiste el ángulo en <code>radar_state.json</code> y lo restituye en el <code>handshake</code> . Verificado en Prueba 3.
RF03	Filtrado de lecturas fuera del rango útil antes de reportar.	Cumplido	El firmware descarta mediciones con distancia cero o negativa y no las incluye en el reporte al servidor.
RF04	Asignación dinámica de <i>slots</i> TDMA: disparos simultáneos para nodos no interferentes y <i>slots</i> separados para pares con solapamiento.	Cumplido	El algoritmo de asignación detecta conflictos de zona sucia y separa temporalmente los pares N1-N2 y N2-N3. Verificado en Prueba 2.
RF05	Control de latencia bidireccional: desconexión por ausencia de respuesta y reinicio lógico del nodo por inactividad del servidor.	Cumplido	El servidor acumula <i>strikes</i> y cierra la conexión al superar el umbral. El nodo cierra el socket si no recibe mensajes en 5000 ms. Verificado en Prueba 3.
RF06	Trilateración cooperativa cuando varios nodos detectan el mismo obstáculo en zona de solapamiento.	Cumplido	El servidor calcula la posición combinada por intersección geométrica cuando ambos nodos del par están en zona sucia. Verificado en Prueba 2.
RF07	Interfaz gráfica con actualización inferior a 1 segundo, orientación angular en tiempo real y nivel de criticidad del obstáculo.	Cumplido	El visualizador recibe el datagrama UDP al final de cada supertrama y actualiza la representación con latencia inferior a un periodo de supertrama. Verificado en Prueba 1.
RF08	Barrido concentrado en modo seguimiento al detectar un obstáculo en el perímetro de criticidad máxima.	Cumplido	La transición SEARCH-TRACK orienta el sensor hacia el objeto detectado y mantiene el seguimiento. Los niveles DEFCON se representan visualmente en la interfaz.
RF09	Chasis modular estandarizado para alojar e inmovilizar los componentes electrónicos.	Cumplido	El diseño 3D iterativo en cuatro versiones alcanzó un ensamblaje sin holguras ni vibraciones parásitas.

Tabla 25: Grado de cumplimiento de los Requisitos Funcionales (RF).

7.2.2 Validación de Requisitos No Funcionales

Del mismo modo, la Tabla 26 analiza el cumplimiento de las restricciones de diseño y calidad del sistema.

Cód.	Restricción	Estado	Justificación
RNF01	Optimización del consumo energético operativo.	Parcial	FreeRTOS bloquea tareas inactivas previniendo ciclos de CPU innecesarios, pero no se realizaron mediciones energéticas con instrumentación de laboratorio por los plazos del proyecto.
RNF02	Alta cohesión y bajo acoplamiento en firmware y servidor.	Cumplido	Arquitectura por cinco capas funcionales en el servidor y cuatro módulos C++ en el firmware con interfaces bien definidas entre capas.
RNF03	Hardware COTS exclusivamente.	Cumplido	El prototipo se construye íntegramente con componentes comerciales de catálogo: ESP32, HC-SR04, NEMA 17, A4988 y LM2596.
RNF04	Código fuente y protocolo publicados bajo licencia abierta.	Cumplido	El código fuente del firmware, el servidor y el visualizador está disponible públicamente en el repositorio del proyecto.
RNF05	Latencia de red optimizada para no desestabilizar la sincronización temporal.	Cumplido	Se deshabilita el algoritmo de Nagle y el modo de ahorro de energía WiFi del ESP32. La referencia temporal t_0 se fija en la recepción local del pulso de sincronización, eliminando la dependencia del sesgo de red.
RNF06	Documentación técnica suficiente para la replicabilidad del prototipo.	Cumplido	La memoria incluye esquemas de conexión, tablas de componentes con especificaciones, diagramas UML y planos del diseño 3D.
RNF07	Guía de despliegue con dependencias, compilación del firmware y configuración del servidor.	Cumplido	La memoria documenta la cadena de compilación del firmware sobre ESP-IDF/Arduino Core y las dependencias Python del servidor y el visualizador.
RNF09	Escalabilidad económica del coste unitario por nodo.	Cumplido	El coste de los componentes electrónicos de cada nodo se mantiene en el rango de electrónica de prototipado de bajo coste, compatible con la escalabilidad de la red.
RNF10	Integración electromecánica sin soldadura SMD ni maquinaria de precisión industrial.	Cumplido	El ensamblaje emplea conectores estándar, protoboard y piezas impresas en 3D, sin requerir herramientas especializadas.

Tabla 26: Grado de cumplimiento de los Requisitos No Funcionales (RNF).

7.2.3 Conclusión sobre los Objetivos

A la vista de los resultados, se puede afirmar que el objetivo principal se ha alcanzado satisfactoriamente. Los nueve requisitos funcionales se han cumplido y los nueve no funcionales se han cumplido en su totalidad, con la excepción de RNF01, cuyo cumplimiento se justifica cualitativamente mediante la elección de un RTOS y hardware de bajo consumo, pero no ha podido validarse con mediciones instrumentadas en el marco temporal de este proyecto. Las tres pruebas de validación han verificado los escenarios operativos más relevantes—funcionamiento en condiciones normales, gestión de conflictos de zona sucia con trilateración cooperativa y recuperación ante desconexión de nodo— sin que ninguna de ellas haya producido un fallo en los requisitos evaluados.

7.3 Retos Técnicos y Limitaciones

7.4 Retos Técnicos y Limitaciones

Durante el desarrollo surgieron desafíos que condicionaron decisiones de diseño no evidentes en la especificación inicial, y el sistema presenta en su configuración actual limitaciones que conviene delimitar con precisión para contextualizar el alcance real de los resultados.

Retos técnicos

- **Referencia temporal en entornos WiFi con jitter variable.** El protocolo de sincronización ancló inicialmente la referencia t_0 al valor de tiempo transmitido en el payload del pulso de sincronización. En las pruebas tempranas esto produjo desincronizaciones de hasta 200 ms entre nodos, causando colisiones acústicas esporádicas. La solución fue descartarlo y registrar el instante de recepción local como t_0 , eliminando por completo la dependencia del sesgo de reloj y del jitter de red. Con mayor tiempo de desarrollo, la implementación de un protocolo de sincronización de relojes distribuidos como PTP permitiría además garantizar la coherencia temporal entre nodos sin depender de la estabilidad de la red.
 - **Criterio de desempate en conflictos de zona sucia.** Definir una política de arbitraje que fuera justa, determinista y libre de inanición requirió varias iteraciones de diseño. La solución final, que combina inmunidad temporal por bloqueo de seguimiento con el instante de primera detección como criterio de desempate, no era evidente en la especificación inicial y emergió de la experimentación con escenarios de conflicto repetido. Con más tiempo de iteración, sería posible evaluar políticas de arbitraje más sofisticadas basadas en la calidad de la señal o en la posición angular relativa de los nodos.
-

- **Latencia introducida por el algoritmo de Nagle y el modo de ahorro WiFi.** La pila TCP del sistema operativo acumulaba los mensajes de control en búferes antes de enviarlos, introduciendo retardos de hasta 200 ms que desestabilizaban el ciclo de supertrama. La desactivación explícita del algoritmo de Nagle y del modo de ahorro de energía del módulo WiFi del ESP32 resolvió el problema, pero requirió un diagnóstico prolongado dado que el comportamiento era no determinista y dependía de la carga de la red.
- **Pérdida de pasos del motor y calibración del reed switch.** En las primeras versiones del firmware, el motor acumulaba errores de posición angular de forma silenciosa al perder pasos por carga mecánica excesiva durante el homing rápido. La reducción de la velocidad durante la rutina de homing y el ajuste fino del umbral de corriente del driver A4988 resolvieron el problema. Con mayor presupuesto, la incorporación de un encoder incremental sobre el eje del motor permitiría cerrar el lazo de control y eliminar esta fuente de error por diseño.

Limitaciones actuales

- **Entorno de validación controlado.** El sistema ha sido validado en un entorno de laboratorio con condiciones acústicas óptimas: superficie plana, ausencia de corrientes de aire y temperatura estable. En condiciones reales, las variaciones de temperatura afectan a la velocidad del sonido, las corrientes de aire desvían los pulsos acústicos y las superficies irregulares generan reflexiones parásitas indistinguibles de ecos legítimos. La extrapolación del comportamiento del sistema a entornos exteriores o industriales requeriría una campaña de validación específica.
 - **Limitaciones relacionadas con el objeto detectado.** El cono de emisión del HC-SR04 de aproximadamente $\pm 15^\circ$ impone restricciones sobre los objetos detectables. Los objetos de dimensión inferior al ancho de haz pueden no devolver un eco suficientemente intenso, las superficies no perpendiculares al eje del sensor reflejan el pulso en dirección contraria al receptor generando falsos negativos, y los materiales blandos o porosos absorben parcialmente el pulso reduciendo el alcance efectivo. El rango muerto de dos centímetros impide además la detección de objetos muy próximos al sensor.
 - **Limitaciones de hardware.** El paso de barrido de $2,5^\circ$ por supertrama implica que un objeto angularmente estrecho situado entre dos pasos consecutivos puede no ser detectado en un ciclo determinado. El sistema de alimentación centralizado a 24 V restringe el despliegue a localizaciones con acceso a corriente alterna. Con mayor presupuesto, la sustitución del HC-SR04 por un sensor LiDAR de un solo punto aumentaría la precisión de las medidas y reduciría la apertura angular del cono de detección sin modificar el protocolo ni el firmware.
 - **Limitaciones de software.** El algoritmo de localización cooperativa calcula la posición del objeto como el promedio aritmético de los puntos estimados por cada nodo, en lugar de una intersección geométrica de circunferencias. Este enfoque introduce un sesgo
-

proporcional a los errores angulares que no puede eliminarse aumentando el número de mediciones. El servidor se ejecuta sobre Python en un sistema operativo de propósito general, sin garantías de tiempo real, lo que introduce una variabilidad en la latencia del bucle de supertrama que el protocolo mitiga pero no puede eliminar por completo.

- **Limitaciones de firmware.** El control del motor paso a paso se realiza en lazo abierto sin realimentación de posición. Si el motor pierde pasos por sobrecarga, el error angular se acumula de forma silenciosa hasta el siguiente ciclo de homing. El watchdog de inactividad TCP, configurado en 5 000 ms, puede provocar desconexiones en redes WiFi con alta latencia momentánea, incrementando artificialmente la tasa de reconexiones.

7.5 Líneas de Trabajo Futuro

El diseño modular del proyecto permite plantear futuras ampliaciones que exceden el alcance de este Trabajo de Fin de Grado pero que aportarían un valor significativo:

1. **Incorporación de sensores de mayor resolución.** La sustitución del sensor ultrasónico HC-SR04 [24] por un módulo LiDAR de un solo punto o por un sensor de tiempo de vuelo óptico mantiene la compatibilidad con el protocolo y el firmware actuales, pero permite aumentar la precisión de las medidas de distancia, reducir la apertura angular del cono de detección y ampliar el rango de aplicaciones a entornos con mayores exigencias de precisión o con condiciones acústicas desfavorables.
2. **Mejora del algoritmo de localización con filtrado de Kalman.** La estimación cooperativa actual promedia las posiciones calculadas por cada nodo de forma independiente. Un primer paso de mejora sería sustituir este promedio por una intersección de circunferencias basada en las distancias medidas, eliminando el sesgo de los errores angulares individuales. La incorporación posterior de un filtro de Kalman permitiría seguir la trayectoria de objetos en movimiento, estimar su velocidad y mejorar la estabilidad de los marcadores en el visualizador.
3. **Escalado de la red a más nodos.** El protocolo de registro automático permite incorporar nuevos nodos sin modificar el firmware ni el servidor, salvo la actualización del archivo de configuración topológica. Una validación experimental con cuatro o más nodos permitiría contrastar el comportamiento del algoritmo de asignación de *slots* con múltiples pares de conflicto simultáneos y evaluar el efecto del incremento del tiempo de supertrama sobre la latencia del sistema.
4. **Sustitución de WiFi por un protocolo de radio de largo alcance.** El empleo de módulos de comunicación de bajo consumo como LoRa o Zigbee en lugar de WiFi abriría la posibilidad de desplegar el sistema en entornos exteriores o en instalaciones sin infraestructura de red preexistente. Este cambio requeriría adaptar la capa de transporte del protocolo a las restricciones de ancho de banda y latencia propias de estas

tecnologías.

5. **Diseño de un sistema de alimentación autónoma.** La alimentación centralizada a 24 V mediante un bus de potencia es adecuada para un banco de pruebas estático, pero impide el despliegue en localizaciones sin acceso a corriente alterna. El diseño de un sistema de alimentación basado en baterías recargables con gestión activa de energía, combinado con los modos de bajo consumo del ESP32, permitiría un despliegue autónomo de varios días para aplicaciones de vigilancia en exteriores o en zonas sin infraestructura eléctrica.
-

Referencias

- [1] Edward A. Lee. Embedded software. In *Advances in Computers*, volume 56, pages 55–95. Elsevier, 2002.
- [2] Espressif Systems. *ESP32 Series Datasheet*. Espressif Systems, 2023.
- [3] I. F. Akyildiz, W. Su, Y. Sankarasubramaniam, and E. Cayirci. Wireless sensor networks: a survey. *Computer Networks*, 38(4):393–422, 2002.
- [4] Qian Liang and Yunjie Liang. Design and analysis of distributed radar sensor networks. In *International Conference on Wireless Communications, Networking and Mobile Computing (WiCOM)*, 2007.
- [5] Robert Buder. *The Invention That Changed the World*. Simon & Schuster, 1997.
- [6] Merrill I. Skolnik. *Introduction to Radar Systems*. McGraw-Hill, 3rd edition, 2001.
- [7] Mark A. Richards. *Fundamentals of Radar Signal Processing*. McGraw-Hill Education, 2005.
- [8] Chee-Yee Chong and Srikanta P. Kumar. Sensor networks: Evolution, opportunities, and challenges. *Proceedings of the IEEE*, 91(8):1247–1256, 2003.
- [9] Tesla, Inc. Tesla Autopilot and full self-driving capability. <https://www.tesla.com/autopilot>, 2024. Accedido: 2026-03-16.
- [10] Jennifer Yick, Biswanath Mukherjee, and Dipak Ghosal. Wireless sensor network survey. *Computer Networks*, 52(12):2292–2330, 2008.
- [11] Johann Borenstein and Yoram Koren. Ultrasonic sensor interference rejection in mobile robotics. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 1995.
- [12] M. Moebus and A. M. Zoubir. Crosstalk cancellation in ultrasonic sensor arrays. *IEEE Transactions on Ultrasonics, Ferroelectrics, and Frequency Control*, 55(4):863–879, 2008.
- [13] J. Chandrasekar. *Radar Design and Management in a Networked Environment*. PhD thesis, Colorado State University, 2008.
- [14] Michael Barr and Anthony Massa. *Programming Embedded Systems: With C and GNU Development Tools*. O’Reilly Media, 2nd edition, 2006.
- [15] Anish Arora, Prabal Dutta, Sandip Bapat, et al. A line in the sand: A wireless sensor network for target detection, classification, and tracking. *Computer Networks*, 46(5):605–

- 634, 2004.
- [16] Kay Römer and Friedemann Mattern. The design space of wireless sensor networks. *IEEE Wireless Communications*, 11(6):54–61, 2004.
 - [17] Guoqiang Mao, Barış Fidan, and Brian D. O. Anderson. Wireless sensor network localization techniques. *Computer Networks*, 51(10):2529–2553, 2007.
 - [18] Sinem Coleri Ergen and Pravin Varaiya. TDMA scheduling algorithms for wireless sensor networks. *Computer Communications*, 33(2):192–202, 2010.
 - [19] Valentino Braitenberg. *Vehicles: Experiments in Synthetic Psychology*. MIT Press, 1984.
 - [20] O. A. Oyelade and A. E. Ezugwu. A review of existing farmland intrusion detection systems. *Agricultural Sciences*, 12(11):1278–1305, 2021.
 - [21] D. M. Anderson. Virtual fencing — past, present and future. *Computers and Electronics in Agriculture*, 56(2):113–117, 2007.
 - [22] Wei Ye, John Heidemann, and Deborah Estrin. An energy-efficient MAC protocol for wireless sensor networks. In *Proceedings of the 21st Annual Joint Conference of the IEEE Computer and Communications Societies (INFOCOM)*, pages 1567–1576, 2002.
 - [23] Richard Barry. *Mastering the FreeRTOS Real Time Kernel — A Hands-On Tutorial Guide*. Real Time Engineers Ltd., 2016.
 - [24] Elecfreaks. *HC-SR04 Ultrasonic Ranging Module — User Guide*. Elecfreaks, 2011.
 - [25] Usongshine Technology. *NEMA 17 Stepper Motor Datasheet*. Usongshine Technology Co., Ltd., 2020.
 - [26] Allegro MicroSystems. *A4988 DMOS Microstepping Driver with Translator and Over-current Protection*. Allegro MicroSystems, 2021.
 - [27] Texas Instruments. *LM2596 SIMPLE SWITCHER Power Converter 150-kHz 3-A Step-Down Voltage Regulator*. Texas Instruments, 2020.
 - [28] Espressif Systems. *ESP32-WROOM-32D & ESP32-WROOM-32U Datasheet*. Espressif Systems, 2024.
 - [29] W. Richard Stevens. *TCP/IP Illustrated, Volume 1: The Protocols*. Addison-Wesley Professional, 1994.
 - [30] Stuart Cheshire and Marc Krochmal. Multicast DNS. RFC 6762, Internet Engineering Task Force (IETF), February 2013.
-

Este capítulo describe el proceso de construcción de un nodo radar periférico, desde el montaje de la estructura mecánica hasta la grabación del firmware. Los pasos deben seguirse en el orden indicado.

A.1 Montaje de la Estructura Mecánica

El diseño estructural del nodo se compone de piezas impresas en 3D que deben ensamblarse de forma secuencial antes de instalar la electrónica.

1. **Placa inferior.** Colocar la placa inferior sobre una superficie plana e insertar los pilares de separación en las esquinas para elevar la placa intermedia.
2. **Motor paso a paso NEMA 17.** Fijar el motor en su soporte sobre la placa intermedia, alineando los orificios de tornillo. El eje del motor debe quedar centrado y perpendicular a la placa. Ajustar los tornillos de forma uniforme para evitar desalineaciones que perjudiquen el movimiento rotatorio.
3. **Soporte del reed switch.** Instalar el soporte magnético en su posición fija sobre la placa intermedia, adyacente al motor. El orificio central debe permitir el paso del cableado sin interferir con el giro del eje.
4. **Soportes del ESP32 y la protoboard.** Encajar a presión los soportes del microcontrolador y de la protoboard en los compartimentos reservados en la placa inferior. Verificar que los componentes quedan inmovilizados sin holgura.
5. **Módulo regulador LM2596.** Fijar el regulador en su posición dentro del compartimento de potencia. Antes de continuar, ajustar el potenciómetro hasta obtener exactamente 5,0 V de salida, verificado con multímetro.
6. **Cabezal rotatorio.** Acoplar el cabezal al eje del motor mediante encaje a presión. Verificar que el ajuste es firme y que el cabezal gira sin oscilación lateral ni holgura axial.
7. **Sensor HC-SR04.** Insertar el sensor en el alojamiento del cabezal mediante encaje a presión. El sensor debe quedar centrado y orientado perpendicularmente al plano de barrido.

8. **Imán permanente.** Fijar el imán en la posición marcada en el cabezal, en el lado opuesto al sensor. Este imán activará el reed switch durante la rutina de calibración angular.
9. **Placa superior.** Cerrar el chasis colocando la placa superior sobre los pilares y fijándola con los tornillos. Antes de cerrar, organizar el cableado interno para que no interfiera con el movimiento del cabezal.

A.2 Conexiones Electrónicas

El diagrama completo de conexiones del nodo, incluyendo el ESP32, el driver A4988, el sensor HC-SR04, el reed switch, el regulador LM2596 y la fuente de alimentación, se encuentra en la Figura 2 del Capítulo 5. A continuación se recoge la asignación de pines de referencia:

- **HC-SR04:** VCC → 5 V; TRIG → D14; ECHO → D27; GND → GND.
- **Reed switch:** Lado 1 → D26 (pull-up interno activado por firmware); Lado 2 → GND.
- **A4988 (lógica):** STEP → D19; DIR → D18; MS1, MS2, MS3 → 3 V3; VDD → 3 V3; GND → GND; EN → GND.
- **A4988 (potencia):** VMOT → V+ del bus de 24 V (a través del regulador de tensión); GND → GND.
- **LM2596:** VIN → Bus 24 V; VOUT → 5 V ajustados; GND → GND.
- **ESP32:** VIN → 5 V (salida LM2596); GND → GND.

Las Figuras 40 y 41 muestran el nodo completamente ensamblado y cableado desde dos ángulos complementarios.

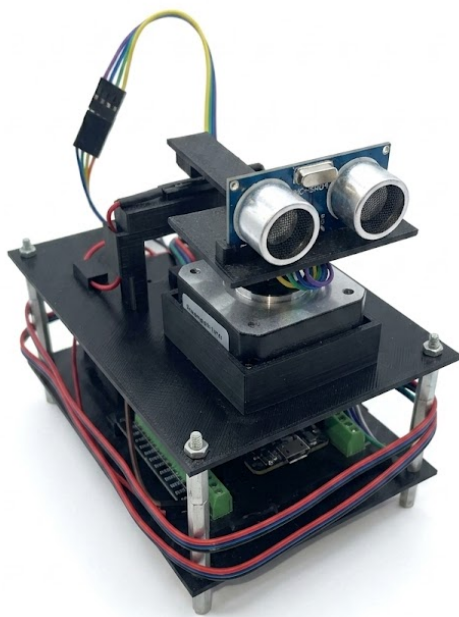


Figura 40: Nodo radar completamente ensamblado, vista frontal: sensor HC-SR04 en el cabezal rotatorio y electrónica de control en las placas del chasis.

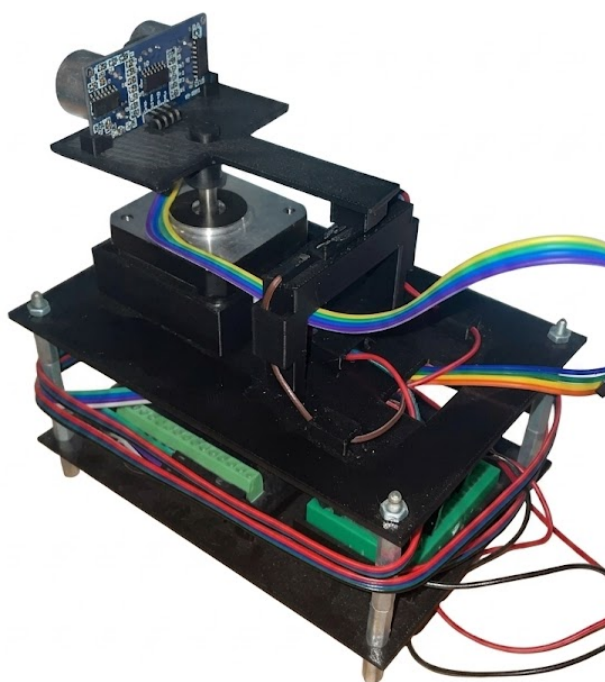


Figura 41: Nodo radar completamente ensamblado, vista trasera: cableado de los componentes electrónicos y estructura del chasis.

A.3 Grabación del Firmware

El firmware se compila y despliega con PlatformIO integrado en Visual Studio Code.

1. Abrir la carpeta raíz del proyecto en VS Code. PlatformIO detectará el archivo `platformio.ini` y configurará el entorno automáticamente.
 2. Conectar el ESP32 al ordenador mediante cable USB.
 3. Editar el archivo `src/config.h` y localizar el array `KNOWN_NETWORKS`. El proyecto incluye dos redes preconfiguradas; para añadir redes adicionales basta con agregar nuevas entradas en el mismo formato, indicando el SSID y la contraseña. El nodo intentará conectarse a la primera red disponible de la lista al arrancar.
 4. Ejecutar la acción **Build** desde la barra de PlatformIO para compilar el proyecto y verificar que no hay errores.
 5. Ejecutar la acción **Upload** para grabar el firmware compilado en la memoria Flash del ESP32. Tras la grabación, el nodo arrancará, realizará el homing angular y quedará a la espera del servidor.
-

CAPÍTULO | Instalación y Configuración B del Sistema

Este capítulo describe cómo preparar el entorno software, desplegar físicamente los nodos y configurar el sistema para una sesión de vigilancia.

B.1 Requisitos de Software

El sistema requiere Python 3.8 o superior en el equipo que actúa como servidor. Las únicas dependencias no estándar son:

```
pip install zeroconf ifaddr
```

La interfaz gráfica utiliza exclusivamente librerías de la distribución estándar de Python (`tkinter`, `socket`, `json`, `threading`) y no requiere instalación adicional.

B.2 Arranque del Servidor y el Visualizador

El servidor y el visualizador se lanzan conjuntamente ejecutando `radar_visualizer.py` desde la carpeta `tools/` del proyecto:

```
python radar_visualizer.py
```

Al ejecutar este comando se abre el panel gráfico en Modo Diseño. El servidor no arranca hasta que el operador pulsa **GUARDAR Y EJECUTAR**, momento en el que el sistema guarda la configuración y comienza a aceptar conexiones de los nodos.

B.3 Ubicación Física de los Nodos

Los nodos deben colocarse sobre la superficie de vigilancia con el sensor HC-SR04 orientado hacia el interior del área a monitorizar, tal como ilustra la Figura 42. Con una cinta métrica

se miden las distancias entre nodos para obtener sus coordenadas cartesianas en centímetros respecto al punto de origen del sistema.

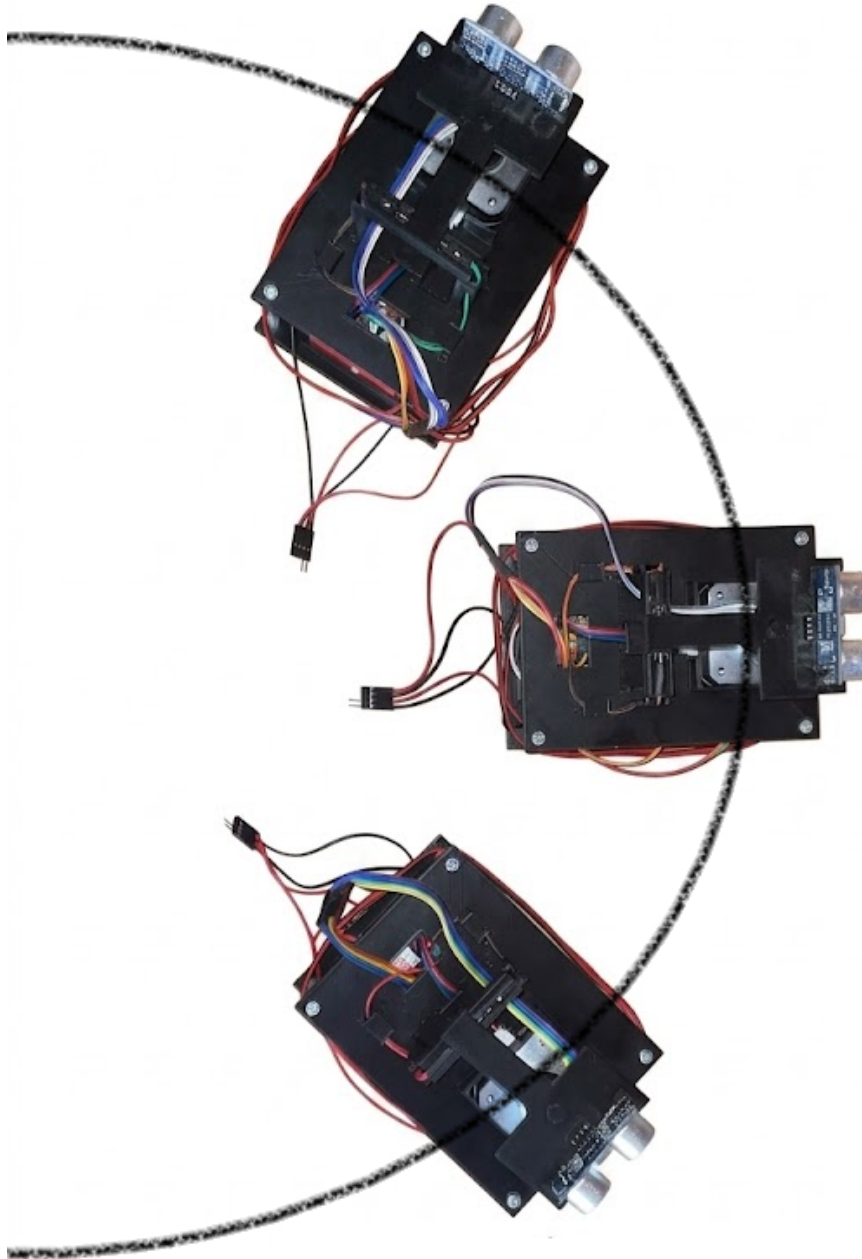


Figura 42: Despliegue físico de los tres nodos radar sobre la superficie de vigilancia en disposición triangular.

B.4 Configuración de Coordenadas y Zonas de Alerta

Las coordenadas, ángulos de orientación y zonas de alerta de cada nodo se configuran en el Modo Diseño del visualizador mediante el panel lateral, como se muestra en la Figura 25. Los nodos pueden reposicionarse arrastrando su icono sobre el canvas; las zonas DEFCON se ajustan con los controles deslizantes del bloque inferior. La primera vez que un nodo se conecta, su dirección MAC queda registrada automáticamente en `grid_config.json` con valores por defecto; el operador debe actualizar las coordenadas reales antes de iniciar la sesión operativa. Al pulsar **GUARDAR Y EJECUTAR**, toda la configuración se persiste en disco y el servidor arranca.

La interfaz gráfica del sistema se describe en detalle en el apartado 5 de esta memoria. A modo de referencia rápida, se recogen aquí los aspectos operativos esenciales.

C.1 Modo Diseño

El Modo Diseño es el estado inicial del visualizador. Permite configurar la geometría del despliegue y los umbrales de alerta antes de iniciar la vigilancia. Una vez ajustados todos los parámetros, pulsar **GUARDAR Y EJECUTAR** guarda la configuración, arranca el servidor y cambia al Modo Telemetría.

C.2 Modo Telemetría

El Modo Telemetría es el estado operativo. El canvas muestra las detecciones en tiempo real con puntos de color según la zona de alerta alcanzada, y un círculo amarillo cuando se activa la localización cooperativa. El panel lateral muestra el estado de cada nodo; si un nodo pierde la conexión, su tarjeta indica *DESCONECTADO / SIN DATOS* hasta la reconexión automática. Para finalizar la sesión, pulsar **DETENER SERVIDOR**.

C.3 Reanudación de Sesión

Al iniciar el visualizador, si existe un estado de sesión anterior en disco, el sistema ofrece la opción de reanudarlo, recuperando posiciones y ángulos de todos los nodos, o de iniciar una nueva sesión descartando el estado previo.